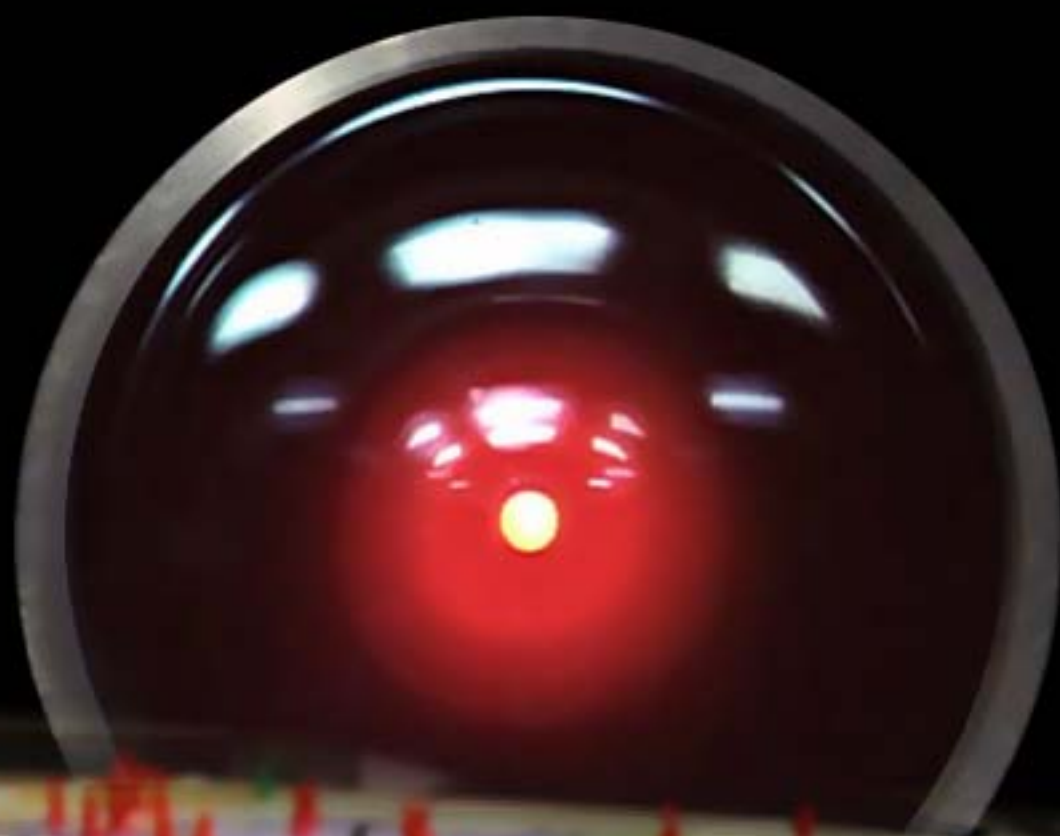


artificial intelligence in computer games



Authors:

Pelle Coltau, Jens Juul Jacobsen and Brian Jensen

TITLE:

Artificial Intelligence in Computer Games

THEME:

Master Thesis E05 - Machine Intelligence Group

PROJECT PERIOD:

September, 2005 - January, 2006

TERM:

DAT5

PROJECT GROUP:

d542a

GROUP MEMBERS:

Coltau, Pelle
Jacobsen, Jens Juul
Jensen, Brian

ADVISOR:

Bangsø, Olav

NUMBER OF COPIES: 5

NUMBER OF PAGES: 140

APPENDIX PAGES: 33

TOTAL NUMBER OF PAGES: 173

ABSTRACT:

This report presents a framework for comparing different AI techniques for computer games. The board game RISK is selected as the case study. An existing implementation of RISK (JRISK) is found suitable for testing the different techniques. A layered, modular structure for the AI is found to be suitable for the AI framework. The tasks of an AI for RISK is identified and the modular framework is explored. A study of the most common AI techniques used in computer games is made. The AI techniques covered are: Decision trees, genetic algorithms, artificial neural networks, Bayesian networks, naive Bayes classifiers, and scripting. Each technique's usage in each module in the framework is analyzed and combined. A full design of the framework is then developed. Finally, numerous topic proposals are explored for future work, such as implementing the framework, testing and comparison of the examined techniques, and examining the generality of the techniques when used in computer games.

Aalborg, January 26, 2006

COLTAU, PELLE

JACOBSEN, JENS JUUL

JENSEN, BRIAN

Foreword

This project was developed by students on the 9th semester in the Machine Intelligence Group at the Department of Computer Science, Aalborg University, Denmark.

This report is a broad study of artificial intelligence techniques used in computer games. The goal of the project is to perform research for use in the upcoming 10th semester where we will be writing our master thesis. The conclusion of this report is meant as a project proposal.

Thanks to Thomas D. Nielsen for friendly discussions on technical questions. Also thanks to Yura Mamyrin for the development of JRisk and for answering API questions.

A digital version of the report can be found through the link below.

URL: <http://www.cs.aau.dk/library/>

Contents

1	Introduction	1
1.1	Project Goal	1
1.1.1	AI Demands	1
1.1.2	Real-Time vs. Turn Based Games	2
1.1.3	The Chosen Game	2
1.1.4	Generality	2
1.2	The Rules of RISK	3
1.3	Implementations of RISK	3
1.3.1	JRisk	3
1.3.2	A Few Definitions	3
2	Analysis	5
2.1	Building a RISK AI	5
2.1.1	Planning the Game	5
2.1.2	Structuring The AI	8
2.2	Information Givers	10
2.2.1	Estimating Opponents' Missions	11
2.2.2	How Close An Opponent Is To Winning	12
2.2.3	The Opponents' Next Moves	12
2.2.4	How Close An Opponent Is To Owning A Continent	12
2.3	The Past	13
2.4	Master Prioritizer: "What to do?"	13
2.4.1	The Complete Set Of Goals	14
2.5	Round Planner: "How to do it?"	15
2.5.1	Overview	17
2.5.2	Coordinating the AIs	17
2.5.3	The Planner	18
2.5.4	The Reinforcement AI	18
2.5.5	The Attack AI	20
2.5.6	The Fortify AI	22
2.5.7	Tasks	22
2.6	Initial Army Placement	31
2.7	Framework Overview	31
2.8	AI Techniques	34
2.8.1	Scripting	35

2.8.2	Decision Trees	37
2.8.3	Genetic Algorithms	40
2.8.4	Neural Networks	43
2.8.5	Bayesian Networks	51
2.8.6	Naive Bayes Classifier	55
2.8.7	Path-finding	55
2.8.8	Influence Maps	59
2.8.9	The Board State	60
2.8.10	Scoring the Board	61
2.9	Discussion on Learning	62
2.9.1	Supervised Learning vs. Reinforcement Learning	62
2.9.2	Training Examples	63
2.9.3	Winners and Losers in Training Data	63
2.9.4	Training Neural Networks	64
2.9.5	Training With Naive Bayes Classifiers	66
2.10	Combining AI Techniques with our Framework	67
2.10.1	Common Techniques	69
2.10.2	Information Givers	69
2.10.3	Master Prioritizer	75
2.10.4	Round Planner	76
2.10.5	Initial Army Placement	83
2.10.6	The Past	84
2.11	Replacing the Whole Framework	84
3	Design	87
3.1	Overall Framework Design	87
3.1.1	Planner Class	87
3.1.2	CashCards Class	88
3.1.3	PlaceArmies Class	88
3.1.4	Attack Class	88
3.1.5	TransferArmies Class	88
3.1.6	InitialArmyPlacement Class	88
3.1.7	Framework Class	88
3.2	Dataflow in Framework	89
3.3	Training Examples	90
3.4	Prerequisites	90
3.4.1	Estimated Army Cost Function	90
3.4.2	Probable Outcome of a Battle	91
3.4.3	Conquering a Continent	91
3.5	Information Givers	92
3.5.1	Estimating Opponents' Missions	92
3.5.2	How Close The Opponent Is To Winning	97
3.5.3	The Opponents' Next Moves	105
3.5.4	How Close The Opponent Is To Owning a Continent	106
3.6	Master Prioritizer	108
3.6.1	Goal Weighting	108
3.7	Round Planner	110
3.7.1	Make Attack Plan	110
3.7.2	Calculate Attack Plan Cost	111
3.7.3	Prioritize Attack Plan	112

3.7.4	Discard Attack Plan	116
3.7.5	Calculate Defense Cost	118
3.7.6	Prioritize Territory Needing Defense	122
3.7.7	Cash Cards	126
3.7.8	Place Armies	127
3.7.9	Transfer Armies	129
3.8	Initial Army Placement	130
3.9	Finding the Best Combination of AI Modules	132
4	Conclusion	135
	Bibliography	139
A	The Rules of RISK	141
A.1	Cashing in RISK Cards Phase	143
A.2	Defend Phase	143
A.3	Attack Phase	144
A.4	Fortify Phase	144
A.5	RISK Activity Diagram	144
B	Analysis Appendix	147
B.1	Scripting Example	147
B.2	Artificial Neural Network Example	150
B.3	The A* Algorithm	150
B.4	EM Learning Example	153
B.4.1	Step 1: Expected Number of Occurrences	154
B.4.2	Step 2: Maximum Likelihood Estimate	155
C	Design Appendix	157
C.1	Full Design of Dataflow in Framework	157
C.1.1	Data Structures	157
C.1.2	Communication Between Modules	159
C.2	Training Examples	161
C.3	HamPath Algorithm	161
C.4	Scripting for the IG “Estimating Opponents’ Missions”	163
C.5	Scripting for the IG “How Close the Opponent is to Winning”	164
C.6	Scripting the Master Prioritizer	165

Chapter 1

Introduction

Computer games being released today are very complex and advanced. However, many of them seem to focus much on graphics and pace over complexity of the computerized players, the AIs. Many (or much) of the AIs are scripted, meaning that they generally do not learn from their mistakes and will most likely do the same mistake again. Many learning algorithms have been developed, but only rarely are they used in computer games. But the question is whether this is a bad thing or not and whether a learning AI performs better than non-learning. In our opinion, this is worth investigating further since learning AIs might be neglected in cases where it would actually be beneficial to have them.

1.1 Project Goal

The purpose of this project is to compare AIs for computer games using different learning and non-learning techniques. We will try to subdivide the behavior of the AI into small segments or tasks, and apply both learning and non-learning methods to each these tasks. This will lead to having various AIs using different combinations of techniques. We will let them play each other numerous times to find the AI that performs the best.

1.1.1 AI Demands

There are different criteria for the performance of the AIs. One of the most important ones is the time the AI uses to solve a task. It cannot take too long or the player will find the game to boring, and stop playing. However, limiting the AI in time often means giving it less calculations to solve its tasks. This may pose a problem in the sense that it may degrade the AI's performance.

The second criteria is the opposition the AI gives. It cannot be too easy to beat. If this is the case, the player will also lose interest in the game. If it is too easy to beat, it would most likely mean that the AI is too predictable. However, the AI cannot be impossible to beat either. This would also kill the players' joy of playing the game.

These two criteria are often dependent. Meaning that the time criteria may influence how much opposition the AI gives. Lowering the time used is often done by taking "shortcuts" in the calculations which might lead to poorer performance opposition-wise.

Therefore, the goal is to find a reasonable trade-off between the performance of the AI and the time it takes to perform.

A third criteria is AIs memory requirement. It cannot take up too much physical memory since this will cause an overall slowdown of the computer due to other processes need to be swapped to disk while the AI works. However, the memory requirement is not a big problem, since the amount of memory in a standard office PC often has up to 1 GB of memory. But some AI techniques use a great deal of memory, so it should not be overlooked.

1.1.2 Real-Time vs. Turn Based Games

When selecting a game in which an investigate into the development of an AI is made, there are two different types of games that can be chosen: real-time games and turn-based games. Real-time games is where the AI needs to make decisions as fast as the human player, or at least the heavier part of the decision making process should be located before the actual game starts. Real-time games sets a much higher restriction on the time an AI can use when making decisions. In turn based games each player make decisions in turn, similar to most common board games. This gives the AI much more time to make decisions, up to many seconds, which is in fact a lot. In the later part of the turn based strategy game Civilization, it can take up to a minute for the AI to finish, which probably is the limit for which most players would tolerate.

But choosing to investigate AIs in a turn based game would result in the ability to investigate much more computationally heavier AI techniques. The AI can concentrate on doing good decisions, instead of concentrating on making fast decisions.

1.1.3 The Chosen Game

The game chosen for this project is the board game RISK. This is a turn based game meaning that it will lower the requirement of time severely compared to a real time game. RISK has a fairly limited amount of rules, but it still has very varied game play, making the development of an AI much more interesting.

1.1.4 Generality

When the investigation into how learning and non learning algorithms has been completed for the game of RISK, it should be interesting to see if some conclusion on the performance of the algorithms, regarding their usage in computer games in general, can be made. Perhaps the usage of each AI technique could be classified to types of games or types of behavior.

1.2 The Rules of RISK

There exist many different rules for the game of RISK. Different rules have both been published by the developers of the original game, but also many “house-rules” have over the years become widely used.

The rules of RISK used in this project can be found in Appendix [A](#).

1.3 Implementations of RISK

There are quite a few different implementations of the RISK game available, both commercial versions and some free. Some of these have been examined throughout this project to determine if any of them could be of any use. What is needed is an Open Source version of the game that is somewhat flexible in the AI part. Meaning that the AI can somewhat easily be replaced by an AI of our own.

1.3.1 JRisk

We found a game called JRisk on the Internet that meets all our requirements. It is written in Java and implements a design that will let us, somewhat easily, replace the AI part. The game is open source and all source code is available. JRisk can be found at [\[jri\]](#).

The only difference between the rules in JRisk and our rules mentioned earlier, is that when another player kills your target color, you win instead of getting the “Occupy 24 territories” mission. Also, the armies gained from cashing RISK cards is different from our rules: each time a set is cashed in, the armies gained is increased from 4 armies for the first set, 6 for the second, 8, 10, 12, 15, 20 and so on.

1.3.2 A Few Definitions

Throughout this report we will use a few terms that needs to be defined here. When we use the word *turn*, it means what happens when a player holds the dice. A players turn is what goes on within the four phases mentioned earlier. A *round* is goes from when a player ends his turn until he gets the turn again. This means that the statement “*an opponents wins 0 rounds from now*” means that the opponent will win before the player gets his next turn.

When a player *occupies* a territory it means that the player owns it, and will be the one defending it from attacks. When a player attacks a territory he tries to *conquer* it.

When referring to “an occupied territory”, it generally means that the territory is occupied by the AI.

Chapter 2

Analysis

Having found JRisk as a usable platform for testing the different AI techniques, the different issues with building an AI framework needs to be analyzed. This chapter is structured as follows: First it will be analyzed which things a Risk player needs to consider when playing the game, then a structure for an AI will be presented. Next, a possible framework for the AI will be discussed and the AI techniques we want to test will be examined. Finally, it will be discussed how to combine the AI techniques with the different parts of the framework.

2.1 Building a RISK AI

To build a functioning RISK AI, it must be analyzed which things it should consider when deciding what to do in each phase.

2.1.1 Planning the Game

When playing the game, there are different things the players need to take into consideration when planning how to complete their missions. These are:

Completing your own mission. If your mission is to destroy another player, you have to make sure that he does not hold a complete continent and thereby obtains more reinforcements than you every round. Also reinforcing near him would be a smart move.

If your mission is to get different continents, it is wise when placing armies at the start of the game to place armies on or near the continents of your mission. If possible, it could be wise to focus on a continent that is easy to keep and then move on to other continents.

If your mission is to occupy 18 territories with 2 armies each it could be tempting to place at least two armies in each territory. However, doing this would not be a good idea because it would reveal your mission to your opponents right from the start.

Also if you have the missions where you have to get X countries, it would be wise to focus on getting a continent so that you get more reinforcements every round.

What are the opponents' missions? To win the game does not only mean fulfilling your own mission, but also preventing other players from fulfilling theirs. Therefore it is a requirement that the player constantly pays attention to the game and notices what the other players are doing. Hopefully it is possible to see some pattern in their movements and thereby end up with a usable estimate on what their missions are.

Obstructing the opponents' mission. It is expedient to obstruct the opponents' missions in order to prevent them from winning. If you think a player is close to winning the game, it could be wise to disregard your own mission for a while in order to obstruct the winning opponent's mission. This can be by spoiling the continents he owns, by killing the player he needs to kill or just by taking some of his territories.

Conquering continents. Conquering continents is a natural part of RISK. A player needs to come up with a plan on how to do this. He needs to figure out which continents are easy to conquer and hold; which ones that are mission specific and also how to conquer them. This involves having an army large enough to conquer the continent in one round and at the same time defending it the next. If he only needs the one continent for winning the game, he does not need to be able to hold it the next round.

How close are the opponents to winning? It is useful to gather information on how close an opponent is to fulfilling a mission since you then would be able to counteract this in time.

Obstructing opponents' continents. Obstructing an opponents continent means conquering one territory in a continent held by one opponent. There are two ways of obstructing. One where the territory is just conquered and left with a minimum number of armies in it, and one where the territory is conquered and reinforced where the goal is to keep it from being re-conquered.

It is necessary to do this either to keep an opponent from winning or just keeping him from getting too many armies when defending. The obstruction can also be done even before the opponent gets the complete continent. If you know what the opponent's mission is and it involves a continent where you have a territory, the obstruction can be done by reinforcing in this territory.

Trading cards. The player needs to consider when it is best to trade in his RISK cards and when it is better to wait.

Placing armies in defend phase. In the defense phase of the round armies can either be placed defensively or offensively. It is up to the player to decide how to use his armies.

Attacking to get a RISK card. Gaining a Risk card will always be a valuable thing since they can be cashed in order to receive extra reinforcements. So if you can see that you cannot do anything useful in a round, just attack somewhere where you are almost 100% certain that you will win. You do not necessarily need to keep the territory for more than the round you take it.

Attack where? The player must decide where to attack. This can either be mission specific or just to get a RISK card. Generally the player should make some kind of “attack strategy” on which territories to attack, and also in which sequence to do the attacks.

Attack from where? The player must choose which territory he will attack from.

Moving armies. The player should decide if any of his territories require reinforcements. This could for instance be continent borders. If it is possible, he can choose to move armies to the territory needing reinforcements from any occupied neighbor territories.

Considering the opponents’ next moves. The player needs to anticipate an opponent’s next moves in order to counteract them. This can be done by evaluating the board state to see where the most likely attack will come. It is very likely that if some of a player’s continent borders are undefended, they will get attacked. Therefore the knowledge of the opponents’ next moves can be used to give you an idea on what to do yourself.

Gathering armies in one territory. If a player’s larger armies are scattered over more territories after a successful attack phase, it can be wise sometimes to use the next few rounds for gathering the armies in one place by moving armies from one territory to another. An example of this is if the player have been designated two of the territories in Australia from the beginning of the game. To keep them from being conquered, he has reinforced in both. After successfully having taken the rest of Australia, he might need to use the next few rounds to gather the armies in the border territory.

Attacking in the right sequence. When wanting to attack a sequence of territories, you need to consider in what order you attack the territories. If you choose a bad sequence, your armies might end up in a territory they cannot exit in any way which might lead to other territories being undefended. In Figure 2.1, part A shows a poor way to conquer Africa, while part B shows a better way.

At least the attack sequence in part A actually ends up conquering Africa, although with its armies far away from the border. It could have been worse: If it had gone from Egypt through East Africa, Congo, and South Africa to Madagascar it would have been stuck there with no possibility of getting to North Africa and conquer the continent. A better situation would be the one in part B, where Africa is conquered and the armies end up in North Africa and can defend the border.

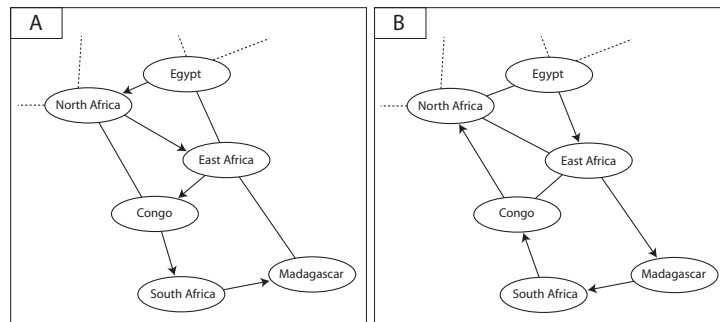


Figure 2.1: Part A and B illustrates a poor and a good attack sequence for conquering Africa, starting in Egypt.

When to stop? When should a player stop his round? Of course the wisest time to stop will be when, if you do something, you will end up in a less beneficial board state. Also, you should not attack a mission specific territory unless you believe you can keep it.

Preventing an opponent from getting one more army in defend phase. If an opponent will receive one less army in reinforcement if he loses one territory, you can benefit from conquering a territory occupied by him. Your benefit from him getting one less army when defending in his next turn of course needs to be weighted against the expected loss you get from trying to take the territory.

Retreat. When should the player give up on attacking a territory? It is possible to retreat in the middle of battle, and this is useful when you are simply unlucky. When you have lost to many armies to either keep the territory you are trying to occupy or the loss starts jeopardizing your ability to keep the territory you are attacking from, it might be wise to stop the attack.

Number of troops in an invasion. When an attack is successful, the player needs to decide how many armies to move into the newly acquired territory. This number is at least the number of dice thrown in the last battle. The number should be decided based on what the future plans are. If they should simply be used to defend the territory or should be used in future battles.

All these aspects of the game are the ones that the players need to consider. While aiming at having an AI taking a player's place in the game, it too must consider the exact same things while making its moves.

2.1.2 Structuring The AI

Besides the different aspects that need to be considered by the AI, the structure of the AI is also important. It may not be important for the functioning of the AI but it will be important that a structure is chosen that allows easy testing of different AI techniques.

Non-modular

Having all AI tasks modeled in one big cloud of interacting functions is possible as seen in Figure 2.2. However since the goal of this project is to compare AI techniques, it seems fitting to have a more modular structure. This should ease the task of replacing specific components and add clarity to the AI structure.

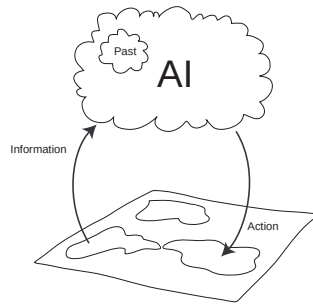


Figure 2.2: *The non-modular design of the AI.*

A way of obtaining a modular structure is by making the AI structure layered. In a layered AI model, the AI is divided into logically coherent parts. The parts send data to each other which ensures that the AI still works as a whole. The benefits of using this kind of model is that it is highly modular since any of the parts can be replaced by others as long as their interface is the same. The downside of this model is that it can be quite hard to find the logically coherent parts and thereby obtain the structure. Since, as previously explained, the project goal would be easier to obtain by having a modular structure, we will put this extra effort into obtaining a layered AI structure.

Modular

A fitting way to split the AI would be by having the tasks that directly modifies the board and therefore logically belongs together in one part of the AI structure and the rest in another. This structure is seen in 2.3. The parts that do not modify the board and only give information to other parts, makes sense to have in their own layer. The parts that modifies the board and uses information from the tasks in the first layer can then be put in a layer of its own.

The estimates based on information from the board and the past and uncertain information such as the RISK cards owned by an opponent can be evaluated and delivered as a list of goals to the lower part. A goal could for example be "Conquer Africa" or "Attack Red Player". Making an abstract version of the information as list of goals will make it easier for the lower layer tasks to handle it compared to receiving separate information from each of the task in the upper layer. The list of goals could have an attached priority telling how important it is to complete. Along with the goal list it may also be useful if the lower layer could still access the information from the tasks in the upper layer directly, because some information will get lost when converting it to a prioritized list of goals.

The upper layer of the AI will from now on be denoted *Master Prioritizer* or simply *MP*. The AI part that is in direct contact with the board is called the

Round Planner or *RP*, since it should be responsible for making a plan based on the list of orders issued every round from the MP. This means that all tasks used to make a plan for the current round, such as where to attack from and to, are placed in the RP. The ones used to specify the general goals are placed in the MP.

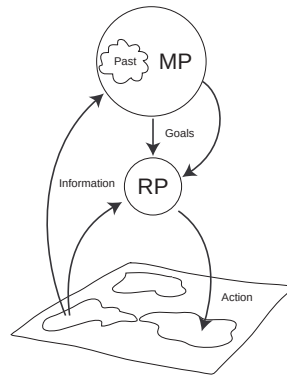


Figure 2.3: A modular design of the AI divided into MP and RP.

Some of the tasks in the MP only outputs information used internally in the MP. A task such as *How close are the opponents to winning?* should output an estimate. This estimate is however the same as indirect information. Therefore the MP can be split into two parts where one part is responsible for calculating indirect information, and the other is responsible for using the information to come up with a list of goals based on the information provided.

To illustrate even further how to split the MP in two, take for example a task like *What are the opponents' missions?*. It does not make sense to have this task output anything else than an estimation on what an opponent's mission is. It would not make sense to have this task output a description of how to prevent the opponent from fulfilling his mission. This issue must therefore be dealt with further down in the AI structure. This new range of AI parts in the structure are called *Information Givers* or simply *IG* since giving information to the MP are their main tasks. The MP will then evaluate the information received from the information givers and, based on this, issue a prioritized list of goals to the AI part that modifies a the board. In Figure 2.4 the whole model for the AI can be seen.

Having this modular structure should make it possible to exchange any part of the structure with a completely different approach.

2.2 Information Givers

The IGs are placed in the upper layer in the AI structure and are the basis of all information provided to the MP. They deliver information on various aspects of the game, such as estimates on opponents' mission and other useful information. The following sections will cover the different IGs. Each task in the upper layer is an IG and it will be described what input and output are needed.

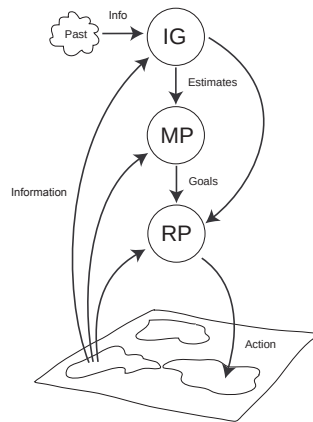


Figure 2.4: A modular design of the AI divided into IG, MP and RP.

2.2.1 Estimating Opponents' Missions

This deals with delivering information on the opponents' missions. This can be done by evaluating what moves the opponents make. By looking at which territories and/or continents he attacks more often than others, it should be possible to make some estimate on what mission he is trying to accomplish. If a continent is attacked more often than others then it might be a part of his mission. Also, if an opponent attacks one player more frequently than others, this may be seen as an indication of that he has one of the "Kill X Player" missions. This means that this IG needs some representation of what have happened in the past as input. Also if an opponent tends to leave two armies on many of his occupied territories it could be a sign of the mission "Conquer 18 territories with 2 armies on each".

When the estimate on the opponents' missions are passed to the MP, it should be accompanied with some kind of certainty value. This value should state how certain the IG is that its estimate is correct. This could be either a percentage value or some sort of fuzzy value like "very sure" or "not sure". Either certainties on the estimate for each are passed to the MP, or only the "most likely" estimate.

Of course the certainty of the estimate also relies on how much data that have been gathered. If only one attack is made by a player in the first round, for example Black attacks Blue, the careless IG would pass the information that Black has to defeat Blue with 100% certainty to the MP which will then, even though it is incorrect information, act on this. To avoid this, there needs to be a certain threshold on the amount of data needed to make "good" estimates. Below this threshold, the estimate needs to be altered either by disregarding it completely or by multiplying it with some factor between zero and one. However, the estimating can still start at the beginning of the game when the armies are placed. If the opponent places more armies in one territory than others, it could mean that he might have this territory as part of his mission. But then again, this estimate should not count as a "good" estimate.

Estimating Opponents' Missions

Input: The Past.

Output: List of mission with an attached certainty on each.

2.2.2 How Close An Opponent Is To Winning

This IG passes information on how close the opponents are to completing their missions. This is of course closely related to the estimate on what the opponents' missions are. The estimate is made by looking at what mission the opponent has (which is of course only an estimate) and looking at the actual board state. From this, you can estimate what the probability is that some opponent wins in the next round, the probability for him to win in the second round from now, and so on, up to some threshold (e.g. 5 rounds from now). If for example the winning probability is 90% 3 rounds from now it means that there is a 90% probability that the opponent has won the game by that time - either after having his turn this round, the next round, the round after that (round two from now), or round three from now.

It is necessary to take what other players will do into consideration since their actions will probably slow the opponent even further which is good. Again, there needs to be some sort of threshold so the AI does not get paranoid in the earlier part of the game. This could for example be the certainty of the estimate on which mission the opponent has.

How Close An Opponent Is To Winning

Input: Opponent's missions, the board, RISK cards owned by the opponents.

Output: A list of probabilities, stating whether each opponent is going to win in the next round, the next round again and so on.

2.2.3 The Opponents' Next Moves

If you already have some estimate on what an opponent's missions is, it is possible to put yourself in his place and see what you would do given the board state, their mission, and how many RISK cards the opponent has (to take any possible extra reinforcements into consideration). Also the information from the other IGs will be useful. One kind of information delivered by this IG, could be a list of actions that represents what the opponent is going to do in his round - this can be an estimation only of course.

The Opponents' Next Moves

Input: The output from the other IGs, the board, the opponents' RISK cards.

Output: A list for each player of actions made by that player.

2.2.4 How Close An Opponent Is To Owning A Continent

Since owning an entire continent is a huge advantage because of the extra reinforcements it gives each round, it is useful to know how close an opponent is to owning a continent. This information can be used by the MP to know when it should intervene.

To estimate how close an opponent is to owning a continent, this IG can take such things as how big a percentage of the territories in that continent is owned by the opponent, how many armies the opponents has in and around the continent, how the opponent's armies are placed in that continent (if he has trapped some of them and they are unable to attack), how many armies the other players have in that continent, and also how many reinforcements the opponent is expected to receive next round into account.

The estimate on how close the opponent is to owning a continent could be given as a list of probabilities for owning it in each round (up to e.g. 5 rounds from now) - the same way as in the IG "How Close An Opponent Is To Winning".

How Close An Opponent Is To Owning A Continent

Input: The board, RISK cards owned by the opponents.

Output: A list of probabilities, stating whether each opponent is going to conquer each continent in the next round, the next round again and so on.

2.3 The Past

The past is a collection of information that keeps track of how the game is progressing. This might very well have been modelled as an Information Giver, but since nothing is estimated upon, it is unfit for our definition of Information Givers. The past instead delivers information to the IGs.

The information from the past round that seems relevant to the IGs is:

List Of Actions: A list in chronological order of actions performed by each player throughout the game might be very useful. Such a list could be used in many aspects of the IGs, for instances where an estimation of an opponent's behavior is to be made, his past behavior might be useful in such an estimation.

Activity Maps: It could be useful to have different maps corresponding to the RISK board, but with different information such as: where a given player has attacked continuously, where a given player has placed reinforcements and where he has moved his armies. These maps would show where a given player has focused his attention throughout the game, and thereby indicate what mission he has. Such maps could either be derived from the list of actions explained above, or they could be maintained throughout the game. These maps are in themselves not estimations or reasonings, but they can be used in some estimation or reasoning on the players' behavior.

2.4 Master Prioritizer: "What to do?"

The main task for the Master Prioritizer (MP) is to deliver a list of objectives to the Round Planner (RP). This set is a list of goals that the RP will use to evaluate which specific goals must be achieved in this round. The MP has the

responsibility that the mission is completed. It must tell the RP which continents to conquer or which player to attack. It is then up to the RP to decide how to fulfill the MP's goals. The goals of the list should be accompanied with a weight specifying just how important a specific goal is. This will be used in the RP to decide what is most important to do.

In principle the MP's only task is to put weights on main goals such as "Conquer African continent", "Attack Red player", and "Defend Australian continent". These weights are decided by using the information given by the Information Givers (IG). If for example an IG is nearly 100% sure of red player's mission and calculates that the red player can win the game next round by conquering the last non-red territory in Africa, a goal such as "Obstruct Africa" is given a higher weight than for example "Defend Europe", urging the AI to do its best to hinder the red player in fulfilling his mission. If there is no risk of any other players ending the game in the near future, the MP should weight the mission specific goals higher such as "Conquer and hold North America" or "Attack Blue Player" or even non-mission specific goals such as taking a continent that is not in your mission. The MP tells the RP what to do, not how to do it. Also, the MP does not take into account the difficulty of a given goal, only the importance.

2.4.1 The Complete Set Of Goals

The following will describe in detail the complete set of goals that the Master Prioritizer should decide upon.

Conquer and defend continent: This goal is meant as conquering a given continent and holding that continent a complete round, such that the AIs reinforcements increase, and perhaps complete a part of his mission. This is done by leaving sufficient armies on the border-territories towards neighboring continents. The difficulty of such a goal varies a great deal throughout a game. A player might only be a single territory away from completing this goal, or the given continent might belong completely to another player. There is one goal of this type for each continent in the game, a total of six goals.

Conquer continent: This goal is a lighter version of the above. It has no requirement of defending the conquered continent in the following round. This goal is most likely to be used in the completion of a mission, where the capturing of a continent is important, not the following defence. There is also one goal of this type for each continent.

Defend Continent: This goal can only be given if the continent has already been captured. The importance is weighted according to the importance of the continent: how many reinforcements does it pay, is it in my mission, etc. In most cases defending a captured continent is always important. There is also one goal of this type for each continent.

Obstruct continent: The meaning of this goal is: obstruct an enemy-controlled continent by capturing a territory in that continent. If an enemy player has captured a complete continent, it is in most cases important that he is not allowed to keep that continent. So capturing a territory in that

continent should be prioritized as important. If some information giver has calculated that the given player is close to completing his mission, then it is even more important to obstruct one of his continents. There is also one goal of this type for each continent.

Obstruct continent and defend obstruction: If it is of importance that a player does not recapture a lost continent, or if another player is simply close to capturing a continent, it should be important that the AI reinforce armies in an occupied territory in that continent, so that the continent is not easily captured - thereby defending his obstruction. There is also one goal of this type for each continent.

Attack player: This goal is mainly used, when the AI's mission is to kill a given player. If he has captured some rewarding continent, and is well fortified, it might be time to begin hunting down his enemy and winning the game. This goal could also be used as a vengeance action. If a given player has annoyed him throughout the game, it might result in an interesting game play, if the AI suddenly starts to retaliate. The goal can also be used if the AI discovers that a weaker player has many RISK cards. Prioritizing this goal high would encourage the AI to attack and destroy this player to get his RISK cards. There is one goal of this type for each player in the game, and since there can be up to six players this gives six goals.

18 territories with at least 2 armies: This goal is only used, when the AI has the mission *Occupy 18 territories with at least 2 armies on each territory*, and the MP has calculated that the AI is close to fulfilling this mission.

24 territories: This goal is only used, when the AI has the mission *Occupy 24 territories*, and the MP has calculated that the AI is close to fulfilling this mission. It is implied, that there is no restriction of 2 or more armies on each territory - at least 1 army on each territory is needed.

In Figure 2.5 one can see a depiction of the information flow from and to the Master Prioritizer. The MP corresponds to the Master Prioritizer, and IG_{1-4} is the Information Givers, that provide indirect information to the MP. The outward edges from MP are the different weights given to each goal. The *Co&De* is the *Conquer and defend Continent* goal, the *Co* is the *Conquer Continent* goal and so on.

For consistency, the inputs and output is the following:

Master Prioritizer

Input: Each of the Information Givers

Output: A prioritized goal distribution.

2.5 Round Planner: "How to do it?"

The Round Planner receives a prioritized list of goals from the Master Prioritizer, and it is the RP's job to determine which of the goals it should meet and how it should do it. As seen on Figure 2.4 on page 11 the RP is the lowest part

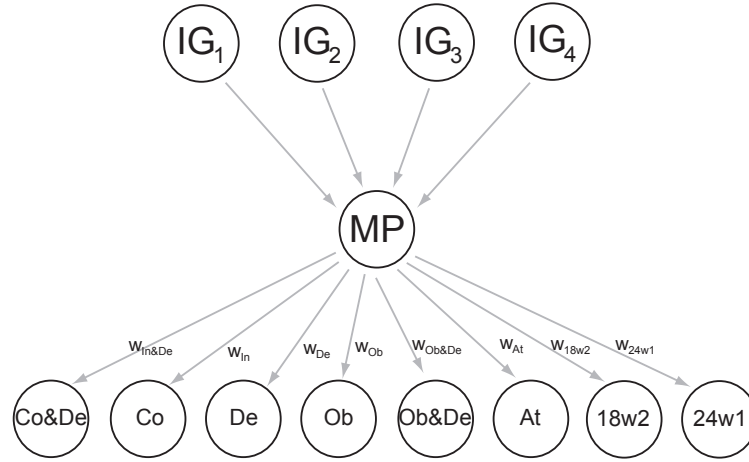


Figure 2.5: The Master Prioritizer MP, receives some information from the Information Givers IG_{1-4} and outputs a weight to each of the goals.

of the AI. Whereas the upper parts only observe the world, this part is able to affect the world by performing actions on the board.

This gives the picture of the Round Planner, shown in Figure 2.6. The input to the RP is the goal list, the current board state, the output from some IGs, and the output from the RP is some action to perform on the board. The goal list is represented by goal nodes (one for each goal) and their associated priority. Each territory node (one for each territory on the board) contains who its owner is and how many armies are placed there. Together, the territory nodes represent the board state.

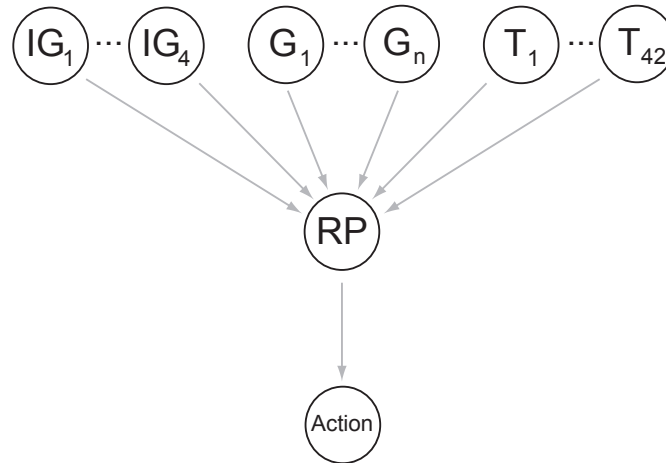


Figure 2.6: The Round Planner receives information from the Information givers I_{1-4} , a list of prioritized goals G_{1-n} from the Master Planner, and the state of each territory on the board T_{1-42} . From this the RP decides which action to perform in the game.

2.5.1 Overview

Each round of the game consists of four phases, and depending on which phase the game is in, the set of legal actions available to the RP is different. The set of legal actions available in each game phase can be seen in the following list:

Phase 0 - Initial army placement:

Place X armies in *Territory*.

Phase 1 - Getting and placing new armies:

Place X armies in *Territory*.

Phase 2 - Attack:

Attack *Territory A* from *Territory B*.

Move X armies from attacker territory to conquered territory.

Phase 3 - Fortify:

Move X armies from *Territory A* to *Territory B*.

This means that an action such as "Place X armies in Greenland" for example cannot be performed when the game is in the attack phase. Because the set of available actions is not the same in each phase, and the purpose of every phase differs, the same RP AI cannot be used for all phases - it needs to be divided. Dividing the RP AI into three or four modules responsible for each specific game phase allows to test different AI techniques that are specialized to exactly that module's purpose. It is obvious that phase 2 and 3 need their own RP AI while phase 0 and 1 may use the same RP AI since their action sets are identical.

2.5.2 Coordinating the AIs

Since the RP is split up in different modules, some way is needed to make them work together towards the same goal. While the AI in phase 0 works independent of the other, the AIs in phase 1-3 are quite dependent on each other's decisions. Without coordination with the Attack AI in phase 2, both the AIs in phase 1 and 3 will have problems determining what to do, since their tasks depends on it. The Reinforcements AI needs to determine whether to trade cards or not and also where it should place the reinforcements. The Fortify AI in phase 3 needs to know whether to move armies from one territory to another or not and if so, find out which territories should be involved. Deciding where to transfer armies from one territory to another cannot be done in a good manner without knowing where the armies will be needed next round. This depends on where the Attack AI will use them next round. To make sure each AI does not make its own isolated decisions, the AIs need some form of coordination.

If the AIs shared a plan on what should happen this round, the coordination problems would not exist. The AIs could then read from this plan and determine what they should do to follow it. Based on the goals from the MP the plan could contain a list of territories to attack and also a list of territories

to defend. If the plan was built like this, by looking at the plan the Reinforcement AI can determine where extra armies are needed, and the Fortify AI can determine where to move the armies.

2.5.3 The Planner

The unit responsible for making the plan is called the Planner. The Planner's task is to interpret the goals from the MP and transform them into concrete territories to attack or defend. This gives two lists:

1. A collection of attack plans
2. A list of territories to defend

Each attack plan in the first list consists of a sequence of territories to attack. The sequence is a path from a territory already occupied by the computer player through a number of enemy territories to a target territory. The target territory is picked based on the goals from the MP, for example if a goal was "Conquer Europe" a target territory could be one of Europe's border territories. If the MP goal "Conquer 18 territories with two armies on each" is a high priority, the Planner should specify to leave two armies instead of one behind when moving armies to a newly conquered territory. A plan will be built for each possible way to attack a target territory. This will result in several plans which cannot all be carried out. The Planner therefore needs to give each attack plan a priority. It would make sense if this priority was related to the priority of the MP goals it helps satisfy. Also, the cost of carrying out each plan should be computed by the Planner. Based on the expected loss of armies and the amount of armies needed to be left behind when moving armies to a newly conquered territory, it is for each attack plan computed how many extra armies would be needed to carry it out.

The list of territories to defend consists of a number of territories. Taking the goals from the MP into account, the Planner decides which territories need extra armies in order to defend them against opponent attacks. This list of territories to defend will also need to be prioritized since there may be many territories that could need defense but there will not be enough reinforcements for every one. This could be done the same as before by giving each territory a priority related to the MP goals that would be satisfied by placing armies there. An indicator on which territories need defense could be if the opponent's next moves contain an attack on an occupied territory. For each territory on the defense list, the Planner should also estimate how many armies would be needed to defend it.

Making a list of attack plans and a list of territories to defend for each and every goal from the MP might be too big a task. To make the task easier, the Planner could for example only take goals with a priority over 10% into consideration.

2.5.4 The Reinforcement AI

The list of attack plans and the list of territories to defend are then received by the Reinforcement AI. It also receives how many armies it has gained by

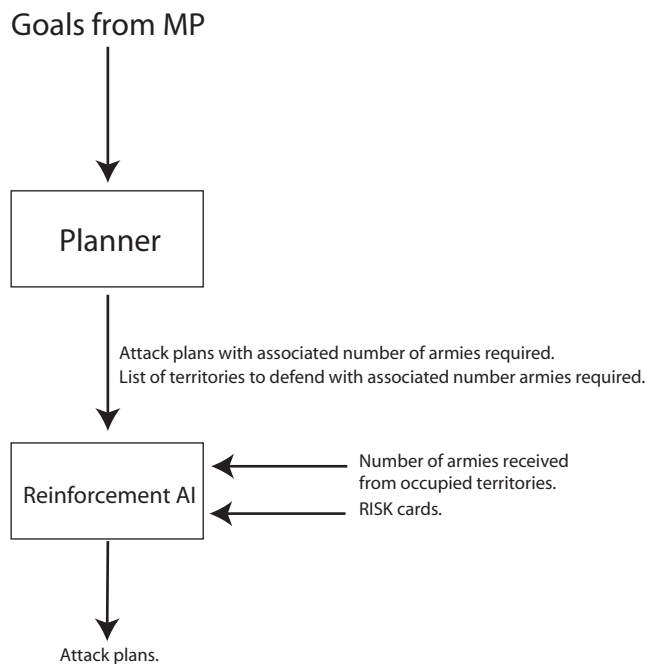


Figure 2.7: Based on the goals from the MP, the Planner makes two lists: One containing a number of different attack plans and one with territories needing defense. Each entry in the lists is labeled with a priority and the number of armies required for completing the task represented by each entry. Based on these lists, the Reinforcement AI decides if it should cash its Risk cards and where it should place the extra armies gained. The Reinforcement AI decides which entries in the list should receive extra armies. Knowing where the armies will be placed, the attack plans which still require more armies can be discarded and only the ones possible to complete will be output, and can be used by the Attack AI.

occupying a number of territories on the board, and which Risk cards it owns. From this information the Reinforcement AI decides whether to cash its Risk cards and where the gained armies should be placed. The relationship between the Planner and the Reinforcement AI can be seen on Figure 2.7.

When looking at an attack plan, the enemy territories are of no importance because the AI can only place armies in territories occupied by itself. Only the starting territories need to be considered. From the attack plans a new list of only the starting territories for each attack plan along with its priority and costs can be made. By merging this list with the list of territories needing defense, a list of candidates for army placement can be made.

Looking at this list, it must decide whether it is necessary to cash its Risk cards this turn. If cashing the cards makes it possible to pay the cost for a high priority objective it would be a good idea to do it. On the other hand, if the extra armies gained by cashing cards does not make a big difference it may be better to postpone it till the next round. For example if the computer player has a joker and two cannons he would receive four armies in reinforcements if the cards were traded in this turn. On the other hand, if he won a card by conquering a territory this turn, and the card was not a cannon, he could get ten reinforcements (e.g. joker-cannon-infantry) next turn instead.

When placing the armies gained from occupying territories and possibly from cashing cards, the Reinforcement AI can use different approaches when choosing what objective it should pay the cost for and thereby make possible to carry out. It has to decide whether an objective with high priority but also a high cost is better to carry out than a low-cost objective with lower priority. It might be possible to carry out several low-cost low-priority objectives compared to a single high-cost high-priority objective.

When the Reinforcement AI knows where the armies will be placed, it can revise the attack plan. Entries in the plan that still require more armies can be discarded and only the ones possible to complete will be kept. It might still make sense to make a high priority plan survive this revision even though it would require more armies - for example if it was an essential plan that could win the game then it would make sense to take the chance although the odds might be less favorable. This list of possible attack plans can then be used by the Attack AI when deciding where to attack.

2.5.5 The Attack AI

With the list of prioritized attack plans in hand, the Attack AI can pick the highest prioritized plan and try to carry it out. It can be directly read in the attack plan what occupied territory should be the one the attack should originate from and what opponent territory should be the target.

Knowing where to attack from and to, the attack can begin. It has to be decided whether to attack with 1, 2, or 3 armies at a time. Attacking with 3 armies will always give the best probability of winning, the only downside is, according to the rules, that the AI is then forced to move at least 3 armies into the target territory if all the opponent's armies are eliminated. However being forced to move at least 2 more armies than the one that has to be moved into the target territory anyway, does not seem like a big price to pay for having the best probability of a positive outcome when attacking.

When the dice have been rolled there are three possible outcomes of the attack: Either the attack has been won (all opponent armies in target territory eliminated), the attack has been lost (all computer player's armies eliminated), or the attack has neither been won or lost. In the last case the Attack AI must determine if it is feasible to continue the attack. This could be done by estimating the probable outcome of the attack based on the number of armies in its own and the opponent's territory. If the calculation shows the AI will lose the attack it would make sense to stop. It might be feasible for the Attack AI to attack even if the odds are not on its side, if the attack plan has a high priority, for example if winning the attack would make the computer player win the game. If not, the attack should be aborted. If, at some point in the attack plan, the computer player has lost more armies than expected according to the plan, it might not be possible to carry it out and so the attack plan needs to be revised. If the attack has been won, the Attack AI must decide how many armies need to be moved to the target territory. This can be directly be read from the attack plan as it corresponds to the attack plan's cost.

After each won or lost attack, the Attack AI can compare the number of armies lost with the estimated number of armies that should be lost according to the attack plan. If more armies were lost than estimated, the Attack AI should make the Planner produce new attack plans based on this situation. If

the actual number of armies lost was lower or equal to the estimated cost, the Attack AI does not necessarily have to poll the Planner for new attack plans since the basis of following the current attack plan has not gotten worse. If an attack plan has been completed and there are spare armies compared to what was estimated in the plan, there could be a need for new attack plans since the extra armies could make it possible to follow new plans with a higher cost. But using the Planner to build new attack plans every time an attack has been made should be done as seldom as possible, since this can be a very calculation intensive job. This is because the Planner bases its decision on the MP goal list, and the MP bases its goal list on the IGs. This means that all IGs and the MP would potentially need to be run again each time the Planner is building a new list of attack plans. So a compromise between always having attack plans that accurately fits the current situation meaning new attack plans after each attack and a potentially slow AI and the need for a fast AI but with attack plans made on a not completely accurate basis, has to be made. With this in mind it seems reasonable to only update attack plans if the actual cost of following a plan was higher than what was estimated. But exactly how big a difference it makes to always construct new attack plan vs. constructing them only when it is really needed, should be investigated further when the AI has been implemented.

All that remains to be decided is when the Attack AI should stop carrying out its given plans and let the Fortify AI take control. The easiest way of doing it would be by letting the Attack AI do everything possible, meaning that it will keep attacking until it cannot do anything else. This however is not always a clever decision. Doing this will ultimately mean that the AI will traverse the map in order to fulfill as many plans as possible. This gives the possibility that the AI will end up having a lot of territories with only one army defending. Therefore the AI needs to be stopped earlier.

It might be wise to use the importance measure as a mark for when to stop. There are some tasks that are always given some importance from the RP: "Attack player X" missions. Giving these task some priority is a way of making it possible for the AI attack if nothing else (more important) can be done. This is because every successful attack is rewarded with a Risk card. If there is nothing better to do, getting a Risk card is a good way to spend the time. Because these tasks can almost always be done they pose a problem: Since they usually can be done from almost everywhere on the map, the RP will be able to calculate a lot of attack plans based on these tasks. Therefore there needs to be some sort of limitation on how many of these low priority task that can be performed in a round. This may either be a constant number or one calculated in the RP. A rather different approach would be having a score for how good one's position on the board is. Calculating such a score is not very simple though. It would involve a way of representing how well positioned one's armies are (or will be after a performed attack) in relation to the enemies' attack plans for the next round. Finding a natural way of representing this score might be a too complex task in contrast to the easier obtainable method described earlier. However, having such a representation would have the benefit that it would provide a natural way to answer the question when the attack AI should stop. The answer would be: When performing any attack will give the AI a lower expected score than at the current time.

2.5.6 The Fortify AI

When the Attack phase is over, the Fortify AI is responsible for transferring armies between two neighboring occupied territories. It must be decided what territory should be destination for the transfer, what territory the armies should be transferred from, and how many armies should be transferred. If no territory needs to get armies transferred, the Fortify AI should decide to do nothing.

A territory can need armies either because it is threatened by armies in enemy territories or because the territory would be a good starting point for an attack next round. The best basis of decision for the last case would be if it was known how the board would look next round. If this was known, that board could be given to the Planner which then could give attack plans and a list of territories needing defense next round. An alternative to this could be to assume that the board configuration will not have changed dramatically next round and so the basis of decision will be the same. To do it this way, the Reinforcement AI could make the MP update its goals based on the board at the end of the turn and then make the Planner give attack plans and a defense list based on this. The destination territory can then be picked as being the highest priority territory with a neighbor having enough armies to pay the associated army cost. If the cost is higher than any neighbor have excess armies but the priority of the destination territory is also high, it may still be feasible to transfer armies though - better have *some* armies instead of none when it *really* counts. Before the transfer is done from a source territory it must be analyzed if the source territory can afford to give away the armies. It would not be wise to take armies from a neighbor who needs them more than the destination territory, for example if keeping the source territory occupied has a higher priority than keeping the destination territory.

When the Fortify AI has made its move, the computer player's turn ends and passes on to the next player.

A picture of the RP and its modules can be seen on Figure 2.8.

2.5.7 Tasks

By looking at the four parts of the RP, it can now be analyzed how the RP can be further divided and split into tasks. Each task can then work as different modules that can be implemented in completely different ways without affecting the other. This makes it possible to test AI techniques for each module independently of the other.

In this section the different tasks of the RP will be identified and then it will be found which elements form the basis of each task's decision. For each module corresponding to a task, there will first be a description of the elements needed by the module. This will be followed by a summary of the module's input and output.

The Planner

The first part of the Planner that seems obvious for a module itself is the task of picking a target territory. The purpose of the module would be to pick an opponent territory that should be the destination of an attack plan. One way of doing this would be to only consider some of the opponent territories was as

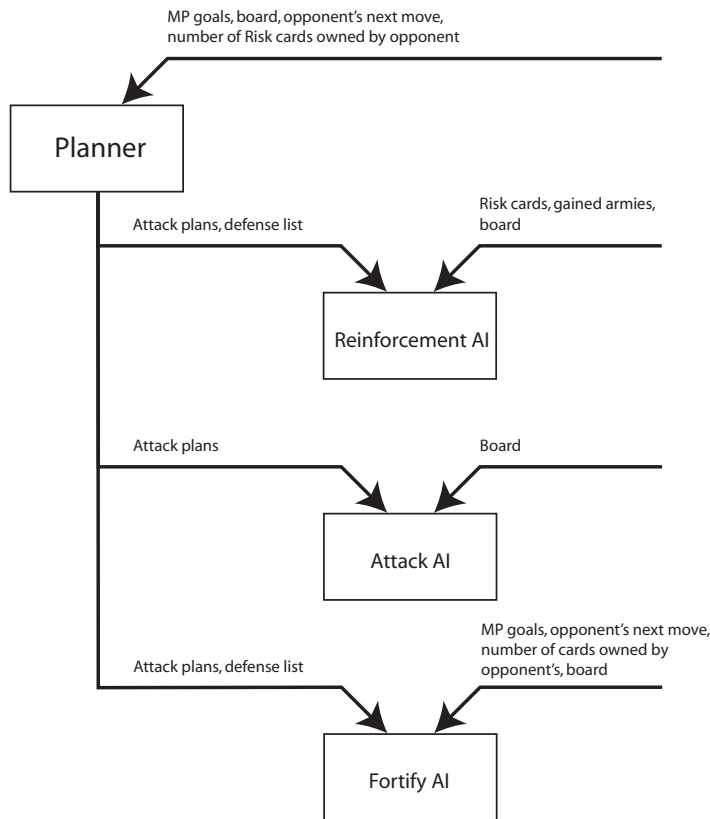


Figure 2.8: The Round Planner consists of four main modules: The Planner, the Reinforcement AI, the Attack AI, and the Fortify AI. The arrows indicate the information needed by each module.

candidate target territories. However, if just a few of the many opponent territories on the board were candidates and only attack plans for these were made would rule out attack plans for all the rest of the territories. This means that potentially valuable attack plans involving these other territories will never be made. So instead of discarding potentially good attack plans before they have been made it would be better to make attack plans for all opponent territories and then discard poor plans afterwards. Each opponent territory on the board should then be picked one at a time and an attack plan should be made using that as target territory. Thus a module for picking a target territory is not needed. Calculating attack plans for each and every combination of occupied and enemy territories might be a calculation intensive job, but the result is that no attack plans gets discarded.

The task of picking a target territory was not suitable for a module. But the rest of the Planner modules found in this section can be seen on Figure 2.9.

Make Attack Plan

Having picked a target territory, it next needs to find a path that leads to it from an occupied territory. This is also a task that is well-suited to be put

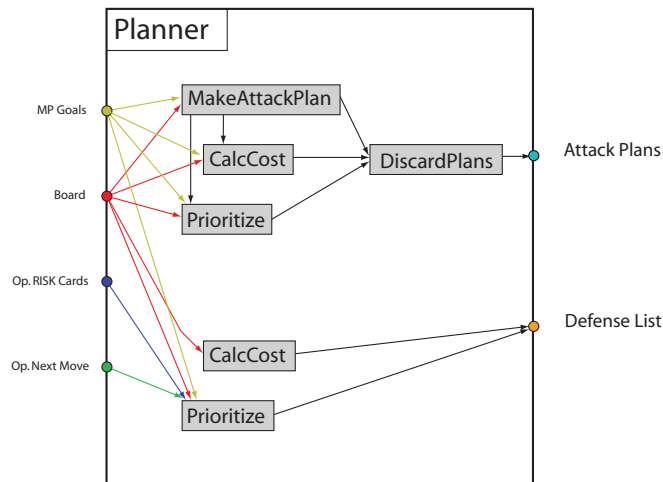


Figure 2.9: On basis of the four inputs on the left, the Planner’s modules computes a list of attack plans and a list of territories needing defense.

into its own module. This path is a list of territories that, when conquered in sequence, will make the computer player occupy the target territory. When making the plan, the target territory and the board should be known. Knowing from what occupied territory the attack should start is also needed. If each of the computer player’s occupied territories is given to the module one at a time as start territory, the result will be many possible attack plans for conquering the same target territory. In addition to knowing the start territory, the MP goals could be of use. For example if there is the choice of attacking either of two territories occupied by Red Player and Blue Player, both have the same number of armies, and the goal “Kill Red Player” is a high priority, choosing Red’s territory would be the best choice.

Make attack plan

Input: Occupied territory, target territory, MP goals, board.

Output: Attack plan.

Calculate Attack Plan Cost

The possible cost of each attack plan measured in the number of lost armies will be calculated. This calculation will fit into its own module. The loss of armies when attacking a territory will be at least one, since one army needs to be left when moving to the newly gained territory. In addition to that comes the extra army to leave behind if the MP goal “Conquer 18 territories with 2 armies on each” is highly prioritized. The minimum army cost for following an attack plan is therefore equal to the number of territories that need to be conquered times two if the mentioned MP goal has high priority. One thing is the minimum army cost for following the attack plan, another and more realistic number is the estimated cost. The estimated army cost takes into account the probability of losing armies each time an opponent territory is attacked. By simulating each attack throughout the attack plan the estimated cost will be

the number of armies lost on the way. Outside the module the estimated army cost can then be associated with the given attack plan.

Calculate attack plan cost

Input: Attack plan, board, MP goals.

Output: Minimum army cost, estimated army cost.

Prioritize Attack Plan

To prioritize an attack plan is also a task suitable for its own module. If the attack plan brings the computer player closer to satisfying an MP goal, the attack plan should be given the priority of that goal. This way attack plans satisfying high priority goals will also get a high priority. In the case of an attack plan satisfying more than one MP goal the priority should reflect this by combining the priorities for the satisfied goals into one. To determine if an attack plan brings the computer player closer to complete an MP goal such as "Kill Player", it would be useful to look at the board and count how many of the target player's armies will be killed if the plan is followed. If many armies are killed then the plan should be given a higher priority according to that MP goal compared to a plan that kill only few or none of the target player's armies. The priority given by the module can then be associated with the attack plan outside the module.

Prioritize attack plan

Input: Attack plan, MP goals, board.

Output: Priority.

Discard Attack Plan

The attack plans build by the Planner may contain plans that are impossible to complete this turn, for example if the minimum cost is much greater than what will be possible to pay this turn even when taking into account the armies received from owning territories and from cashing Risk cards. If these plans were passed on to the other modules, for example the Attack AI, then it would start an attack with no chance of success. On the other hand, discarding all attack plans that cannot be completed this turn would make the computer player incapable of following plans that require more than one round to complete. This could mean that the computer player would never get closer to meeting a comprehensive goal such as to conquer a continent because it would probably never be possible to complete in one turn. To complete such a goal it could be needed for some turns to place reinforcements in a territory from which a conquering attempt could start, although these armies would not be used in the same turn they were placed. If the goal was a part of the computer player's mission, the MP would have given it a high priority. Therefore it would make sense to keep high priority attack plans whose cost is greater than what will be possible to pay this round, but lower priority plans with a non-payable cost should be discarded. Partially paying the cost for a high priority plan this round will bring the computer player closer to be able to pay the total cost next round.

If the plan takes a big number of rounds to complete, like 50, and although the plan had very high priority it could make sense to discard it. Spending armies on a plan that lies so far ahead in the future would be waste of armies, since it may keep the computer player from completing less, but still high prioritized plans.

Discard attack plan

Input: Attack plan priority, minimum army cost, estimated army cost, number of reinforcements without cashing Risk cards, Risk cards.

Output: Yes/No.

Together the modules *Make attack plan*, *Calculate attack plan cost*, *Prioritize attack plan*, and *Discard attack plan* will result in a list of attack plans, each plan with its associated army cost and priority. This information can then be used by the other AI modules such as the Reinforcement AI to determine where extra armies will be needed.

In some situations the described way of making attack plans has its limitations since an order in which they should be followed cannot be defined. Imagine the situation in Figure 2.10.

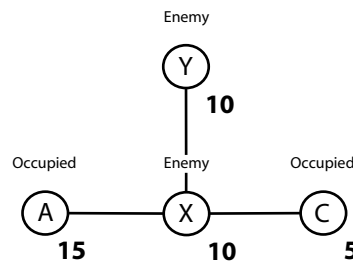


Figure 2.10: A situation where territories A and B are occupied by the player and X and Y are occupied by the enemy.

Territories A and B are occupied by the player while X and Y are territories occupied by the enemy. If the player wanted to use the armies in A as his main attack army to attack X and Y and he did not care about occupying B, a wise move would be to attack X from B to soften up X and then attack with the main army force from A through X and Y.

The attack plans that would be made for this situation would be: $A \rightarrow X$, $A \rightarrow X \rightarrow Y$, $B \rightarrow X$, and $B \rightarrow X \rightarrow Y$. The plans starting with $B \rightarrow X$ would then probably be discarded because these attacks would be estimated to be lost. The consequence of this would be that the AI would attack X from A, and if the chance of conquering Y with the remaining armies is low, it would then not attack Y this round, but move the armies from B to X in the fortify phase. Y could then be attacked next round using the armies gathered in X. So, overall the result will be the same but the attack on Y s to wait till next round. This is of course a limitation but it is not a severe one so using attack plans the current way should not hamper the AI significantly.

Calculate Defense Cost

Like all territories were potential starting territories for an attack plan, all territories should be potential recipients of reinforcements. The only territories which may not be a good idea to place reinforcements in, are the ones already surrounded by occupied territories. It would be a better idea to place the reinforcements in its neighbors.

However, this is not true for all situation. Imagine the situation depicted in Figure 2.11 where a continent is occupied and the territory *D* next to the continent border is also occupied and has an opponent territory as neighbor. 1 army is placed in *B* and *C*, 10 in *A*, 2 in *D*, and 15 in *X*.

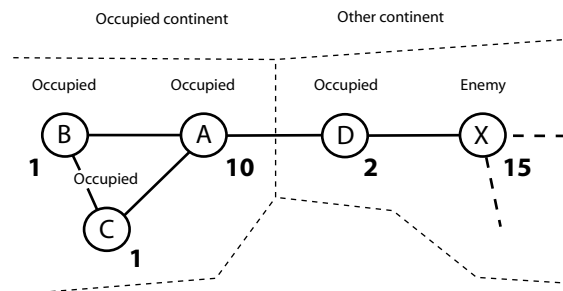


Figure 2.11: The occupied territory *A* has 10 armies placed in it, *B* and *C* 1, *D* 2, and the opponent territory *X* 15. Although *A* has no opponent territories as neighbors it could still be a good idea to place armies in it.

Placing armies in *A* would still be beneficial since they will protect the continent from enemy attacks, especially if the player's amount of armies in its neighbors outside the continent is low, like the situation is here. So not considering a territory without an enemy territory as neighbor (like *A*) a candidate for reinforcements will not be a good idea. All territories should be considered.

Like calculating the expected army cost for an attack plan, the task of calculating the army cost of defending a certain territory can be put in its own module. By looking at how the opponent armies are placed on the board it can be estimated how many armies will be required in order to keep the territory from being conquered by an opponent this round. The surrounding territories have an impact on how many armies are needed to make the territory a bad choice for the opponent's to attack. It may not be needed for the territory to be defended fully against an attack as long as there are other territories around it that are easier targets for the opponents and thereby cheaper to conquer. It may only need an amount of armies that is just sufficient for making the territory uninteresting to conquer compared to its neighbors.

Calculate defense cost

Input: Territory, board.

Output: Estimated army cost.

Prioritize Territory Needing Defense

The last of the tasks in the Planner that can be a module for itself, is the task of prioritizing a territory needing defense. If defending a territory will satisfy an

MP goal, the priority of the territory should reflect the priority of the MP goal. The opponent's next move will also be useful information and can be received from the appropriate information giver. For example, if it is likely that an opponent will attack an occupied territory this round, defending the territory should add to the priority. However, the opponent's next move will always be an estimate and to base how threatened an occupied territory is solely on this could be dangerous. So it would make sense to take the board as input as well. Using the board it can directly be seen which territories are threatened and have few armies in it compared to its opponent neighbors. Also, if the opponent has for example five Risk cards on hand, he is sure to receive extra reinforcements in his turn, so the armies in territories occupied by him represents a potentially greater threat. Like the attack plan, if defending a territory satisfies more than one MP goal the priority should reflect this by combining the priorities for the satisfied goals into one.

Prioritize territory needing defense

Input: Territory, MP goals, opponent's next move, board, number of cards owned by opponents.

Output: Priority.

From the two aforementioned modules, *Calculate defense cost* and *Prioritize territory needing defense*, a list of territories needing defense can be made, each territory with its associated army cost and priority. Like the attack plans, this information can then be used by the other AI modules to determine where extra armies will be needed. No module is needed for discarding a territory from a defense list since this it will be the task of the Reinforcement AI to decide. The reason why the module *Discard attack plan* exists is that the attack plans produced are used by more than one module and it therefore makes sense to do this one time for all modules.

Reinforcement AI

The tasks of the Reinforcement AI are first to decide whether to cash its Risk cards this turn and then place the armies possibly gained from the cards and the armies gained from owning territories and continents on the board.

When placing armies in occupied territories on the board it should do it with respect to both which attack plans need armies and also which territories in the defense list need armies. When placing armies to satisfy the need of an attack plan, they can only be placed in the attack plan's start territory. Therefore the rest of the territories in the attack plan are of no relevance in relation to the Reinforcement AI.

A new territory list consisting of the start territories in each attack plan and each territory in the defense list can be made. Each territory in the list can then be associated with a priority and a minimum and estimated cost according to the attack plans' and the defense list's original priorities and costs. The minimum cost for a territory in the defense list can be set equal to the estimated cost. This association between territories, priorities, and costs should still be stored. The priorities and the costs could for example be stored in each their

lists - then the X'th entry in the priority list could contain the priority associated with the X'th territory in the territory list.

Cash Cards

The module for the task of deciding whether to cash the Risk cards on hand or not will need to consider a number of things. The associated priority of a territory in the territory list along with its cost could both influence the decision. Since the module does not need to know where the territory is located on the board, only the priority and cost is needed in the decision making. Besides from cashing Risk cards, the armies needed to pay the costs are gained from occupying territories and continents on the board. By considering both these numbers, the module can decide if there is need for cashing Risk cards in order to pay for territories with high priority.

Cash cards

Input: List of priorities, list of minimum and estimated number of armies needed to satisfy each priority, number of armies gained from occupying territories and continents, Risk cards on hand.

Output: Yes/No.

Place Armies

Placing armies can be done by making use of the territory list with the associated lists of priorities and costs. The module's task is then to pick which territories from the list should have their cost payed. Territories not needing any extra armies can safely be removed from the lists (including priority and cost lists) before this module is used.

In the situation where two territories have the same priority and cost but there are only armies enough to pay for one of them, the following actions can be taken: Pay for one of the territories, pay for the other, or pay both half. It may be best to pay the full cost for one of the territories and not for the other, since only paying half of the cost will not give the territories what they really need. On the other hand, paying half of each of the costs may still help a little and could be sufficient, for example if the difference between the cost and what can be payed is small and only is a matter of a few (one, two) armies. It is still worth something to have two priorities almost payed for compared to paying the cost for only one and thereby dissatisfy the other priority.

Another interesting case is if there is a high priority territory which cost cannot be payed fully and at the same time possibly more than one low priority territory whose costs can all be payed. The actions that can be taken are then: Pay most of the high priority territory's cost, pay all of the low priority territories' costs, or pay a some of the high priority territory's cost and some of the low priority territories' cost. One could say that making sure many lower priority territories gets their cost payed is better than paying the cost for only a single high priority territory. But if the single territory has a high priority compared to the others, then it must be worth paying for.

It would be a simple way of paying the costs for more than one territory, if the module was called one time for each army received as reinforcements and

placed one army each time. Even though there might not be armies enough for paying the whole cost for two territories, interleaving the placement would give the capability of partially paying the full cost for both. From the list of territories needing reinforcements the module then chooses one in which one army should be placed.

Place armies

Input: List of territories needing reinforcements, list of priorities, list of minimum and estimated costs, number of reinforcements left.

Output: Territory.

Attack AI

The Attack AI does not have any tasks that are suitable for a module. For example, to estimate the probable outcome of an attack based on the number of armies in the computer player's own and the opponent's territory is a question of a simple calculation and the result will always be the same given the same input. So just making a table of the possible outcomes given 1, 2, or 3 attacking armies vs. 1 or 2 defending armies, will be a simple solution to this problem. The resulting table would be 3x2 large with the number of attacking armies along the row and number of defending armies down the column. To determine the outcome of an attack with 500 attacking armies against 300 defending armies, the attack could be simulated by looking up in the table and iterating that way until no armies are left on one of the sides will give the most probable outcome. A more effective alternative to the table would be to devise a formula that calculates the outcome directly.

Another example on the simplicity of the Attack AI shows when picking which attack plan to follow. It is simply a question of picking the one in the list with the highest priority because the unusable attack plans has already been discarded by the Planner.

When the computer player has conquered a territory it must decide how many armies to move into it, or rather how many armies to leave behind. This can be seen from the minimum cost of the attack plan since this number corresponds to the number of armies to leave behind if conquering every territory in the plan. To find out how many armies to leave behind the minimum cost can simply be divided by the number of territories to conquer in the attack plan. Doing it this way could seem to make it impossible to leave a lump of armies behind midst of an attack plan if for example a territory in the middle of the attack plan needs it to be able to defend it. But if the attack plans are rebuilt by the Planner each time a territory has been conquered this would not be a problem, since the situation would be reconsidered and the lump of armies would then not be moved away from the territory needing defense.

Fortify AI

The task of the Fortify AI is to transfer armies between two neighboring occupied territories and it is a task that fits a module.

Transfer Armies

It must decide a source territory from which the transfer should be made, a destination territory for the transfer, and finally how many armies should be transferred. The territories needing reinforcements along with their priorities and costs and the total list of occupied territories have influence on this module's decision.

Transfer armies

Input: List of territories needing reinforcements, list of priorities, list of minimum and estimated costs, list of all occupied territories.

Output: Source territory, destination territory, number of armies to transfer (0 = no transfer).

2.6 Initial Army Placement

Before the game actually starts, each player needs to put armies on their given territories one after another. This placement is something neither the MP nor the RP takes care of. This is done only in the beginning of the game, and therefore it would not make sense to have this functionality as a part of either of the mentioned AI parts. Therefore this gets a part of its own which is run before any of the others, and only when it has finished the others can start playing the game. The AI part responsible for doing the initial army placement will from now on be denoted as the *IAP*.

The placing of armies could be done randomly across occupied territories, but it would make good sense to have a more effective way of placing them since the rest of the AI will use the outcome of this part as the basis for the rest of the game. Since the rest of the AI is working towards winning the game following some specific mission, the IAP should also base its placements on this mission. Besides taking the mission into account, it is always good to get control over a continent. So if the AI has been given a lot of territories in one of the easy defendable continents, it is good to place armies in this. Even if it is not a mission specific continent, the constant extra reinforcements gained every round make it easier for the AI to fulfill its' mission. Also, if an opponent has been given many territories in a continent, it is important that the AI will try to prevent that opponent in easily conquering that continent.

Place army

Input: Board, Mission.

Output: Territory.

2.7 Framework Overview

The framework for an AI in RISK has now been fully defined. This section will give a quick overview. Figure 2.12 is a full picture of how the information flow is between all the different modules in the framework, not including the IAP since it is not related to the other modules in-game.

At the top, the different information directly available (the past, board and opponents' number of RISK cards) is located. These feed information to many different modules in the framework. Below the direct available information, the *Information Givers* are located. They mainly feed information to the *Master Prioritizer* below. The MP delivers a goal distribution to the different parts of the *Round Planner*. These parts, called the *Planner*, *Reinforce*, and *Fortify* are located at the bottom of the figure. The colors on the arrows are merely for making the figure more readable and each gray box represents a module. Each dot on the left side of each of the four bottom parts represents an input and the type of input can be read next to each dot. Depending on which input a module uses the arrows connecting it to an input will be different.

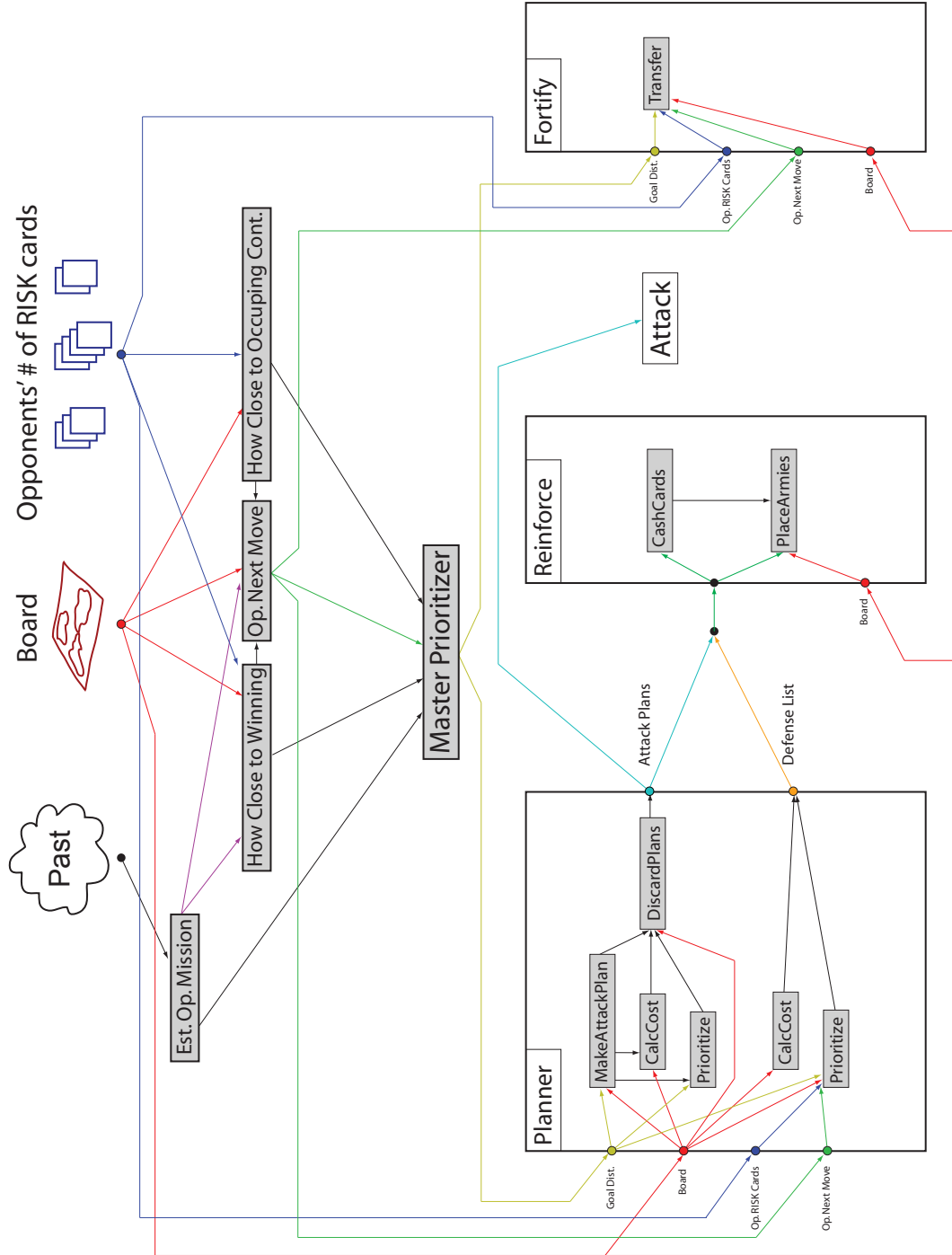


Figure 2.12: The information flow in our framework. At the top, the different information directly available (the past, board and opponents' number of RISK cards) is located. These feed information to many different modules in the framework. Below the direct available information, the Information Givers are located. They mainly feed information to the Master Prioritizer below. The MP delivers a goal distribution to the different parts of the Round Planner. These parts, called the Planner, Reinforce, and Fortify, are merely for making the figure more readable. The colors on the arrows are merely for making the figure more readable. All gray boxes are modules.

Each module could correspond to a class in the object oriented paradigm, where the functionality of a module can be replaced with another, as long as the input and output is the same. The modules in the framework can be replaced by different AI techniques, but not all modules can be replaced by any AI technique.

The following section will describe the AI techniques used in this project. Following that, will be a section that discusses which AI techniques can be used in which module.

2.8 AI Techniques

The following sections will cover a description of the following AI techniques, which can be used by the modules in this project. Each technique will be introduced and it will be described how it is used in computer games. After this, each technique will be examined in details.

Scripting: This is the most used techniques when constructing an AI for a computer game. The behavior of the AI is directly dictated through a set of scripts and if the behavior of the AI should evolve, the script themselves needs to be rewritten. This is so widely used because the scripts can be tailored to the very specific case of each AI, and no time should be spend on teaching or evolving the AI. Scripts can be used to specify most AI behavior using the degree of complexity needed. But on the other hand, improving the AI is tedious and in most cases difficult to do.

Decision Trees: Decision trees learn the benefit of doing specific tasks and taking specific decisions. They learn this benefit by looking at what the benefit of that specific decision (or series of decisions) has been in the past. Decision trees can be used to retrieve the general benefit of taking some decision. But the past will always limit the “intelligence” of decision trees. It can not be evolved to make decisions that are more intelligent than the decisions made in the past.

Genetic Algorithms: These are used to optimize or search for a solution to some problem. They use randomness and merging of the best performing solutions. But the problem has to be represented as a binary string of 1s and 0s. Not all problems can, in a practical way, be reduced to such a string.

Artificial Neural Networks: These are used in a wide variety of applications, which also include computer games. The power of artificial neural networks (ANNs) is that they base their immediate behavior on past behavior, but can also be trained to perform beyond the limits of the past. This is useful in computer games, when the AI needs to simulate human behavior - but also in most cases challenge any human player. This is difficult to achieve since the past needs to somehow include the best playing behavior. ANNs can then be used to evolve such behavior. But ANNs have the disadvantage that the in- and output of the network has to be a discrete set, and methods in where the in- or output may vary in size can not be directly simulated by ANNs.

Bayesian Networks: These are not used in any (commercial) games at the moment to the best of our knowledge. They are used to describe how different objects in the real world interact causally with each other. Methods exist that distribute knowledge from observed events in model throughout the entire model. Bayesian networks do not necessarily supply 100% concrete knowledge on whether a specific event occurs, but mainly gives a distribution on how likely an event is to occur given that other events have occurred. The major downside of Bayesian networks is that their size grows rapidly as the number of nodes in the model grows. It can then get too big for a computer to handle and take long time to compute. Another downside of Bayesian networks is that a lot of probability tables need to be manually specified for the network to output useful information. This can be overcome by using a learning Bayesian network technique which makes it possible for the network to learn the distributions in the probability tables.

Some other tools are also useful, but these are more specific tools, than general AI techniques. Since these are only specific tools, they will only be used in scripting - they can not be used to replace entire elements of the AI. Those tools are the following:

Path-finding: This is the problem of going from one point to the other on some optimal path. The shortest-path problem is most commonly solved by the *A* algorithm*. Another problem is to conquer a series of territories in a specific order to satisfy some goal without getting trapped halfway in a territory from which the attack cannot continue.

Influence Maps: Influence maps are used to provide all kinds of strategic disposition information to an AI. In a RISK game, it can be used to extract abstract information from the board such as the influence of a given player on a continent or the potential attacking radius of a given player etc.

2.8.1 Scripting

Scripting is a very intuitive way of creating an AI. By analysing what situations that can emerge in the game, the AI designer designs different behavior into the AI to handle these situations. Even though this may sound very simple, it might be a rather complicated task since in most games the number of possible scenarios to handle can get very numerous. Hard coding scripts for an AI is one type of scripting. We define hard coding as designing the AI to work in one *specific* environment. The downside of hard coding the AI is that you cannot change anything in its working environment. In Figure 2.13, an AI has a hard coded representation of path from 1 to 4. If it is told to move from 1 to 4, it has been hard coded to know that it needs to go from 1 to 2, from 2 to 3 and from 3 to 4 to get to the destination. However, if the environment was changed with the one in Figure 2.14, the hard coded AI would not see the connection from 1 to 4 with the old code. Therefore, if anything is changed in the environment the hard coded AI needs to be changed too. This little example was used to show that it is not a good idea to hard code everything. Rather it is a good idea to examine more dynamic ways of designing the AI so it can be used in different environments.

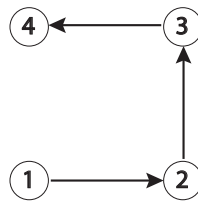


Figure 2.13: A graph representing how the AI can move from 1 to 4.

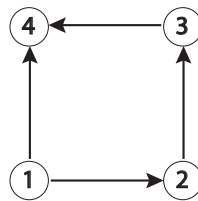


Figure 2.14: Another graph representing how the AI can move from 1 to 4.

Simple Scripting

A script is build up by having some *conditions* which needs to be met for the script to be run, and some *actions* which is the actual output of the script. This is illustrated in Figure 2.15. As seen, scripting is merely a set of *if-then* statements where the state of the AI and observations of the world determines what needs to be done.

```

if
  CONDITIONS_HERE_ARE_MET
then
  DO_THESE_ACTIONS
else
  DO_THESE_ACTIONS
  
```

Figure 2.15: A basic script it divided into conditions and actions.

In Appendix B.1 a practical example of how scripting can be used in a small first person shooter game is found.

Advantages and Disadvantages of Scripting

The major benefit of scripting is also one of its drawback. This is that the AI designer has full control over how the AI behaves in every situation in the game. This is also a drawback because the designer must make sure that the AI *can* in fact handle every situation. If some situations overlooked, the AI might behave irrationally or even badly when the situation occurs.

Another drawback with the scripted AI is that it is not adaptive. It will

never get better than its designer made it. This means that once a player has learned how to beat it in the game, he will most likely beat it every time since it does not change. Of course this last statement may not always be true since in most games luck plays a certain role, but in general when you figure out how an AI works, beating it becomes relatively easy. There are some measures that can be taken to avoid the AI becoming too predictable such as letting the AI choose randomly from more actions in different situations.

One major advantage concerning scripted AIs is that they are quite easy to understand and design. This is of course dependent of the size of the game, but in general this is true. Also a scripted AI does not need any training data to work and is therefore not dependent of the trustworthiness of such training data.

2.8.2 Decision Trees

A decision tree (which is also known as a classification tree) is used for making decisions based on a set of training data. There are two types of decision trees. One which is learned through training data and one which is manually build. We will first cover the learned trees and later we will describe the second type of tree. The tree structure is learned from training data. The use of this is that the tree can be used to tell what to do when the world looks in a certain way. The world is represented through attributes having different values. Basically the final result of building the decision tree is a set of *if-then* statements used to decide on a specific attribute.

The Training Data

The training data required for building the tree is a range of attributes with associated values that describes certain observations in the world. The number of different values that an attribute can have needs to be discrete and finite, meaning that they cannot just be e.g. integers. One of the attributes is the *target attribute* which is the attribute that the tree outputs, being the decision taken by the tree. In the decision tree, each attribute will be a node with a number of edges leaving it. This number is determined by the number of values the attribute can have.

Building the Decision Tree

An approach to building the decision trees is using the *ID3* algorithm [Tom97]. In the *ID3* algorithm all attributes are tested to see which one gives the highest *information gain* on the target attribute. The information gain is a measure stating how much information there is gained on the target attribute from the attribute being examined. Calculating the information gain requires calculation of the *entropy* for the attributes. The entropy is a measure of how “dirty” the training data is. If all examples of a certain attribute in a specific state results in the target attribute being in only one specific state, the entropy is zero. The further the entropy moves from zero the dirtier the data is. The link between entropy and information gain is that the lower the entropy, the higher

the information gain.

Calculating the information gain is the main part of the ID3 algorithm. The attribute with the highest information gain on the target attribute, is used as the root node of the decision tree. This node has a number of children corresponding to the number of possible values the attribute has. Which node is placed on a branch is determined by re-calculating entropy and gain based on the value of the edge. This is done for all edges going from the parent.

This procedure continues until all examples has zero entropy or all attributes have been entered into the tree from the root and down. Attributes are only considered once in any branch of the tree meaning that going from a leaf to the root, any attribute will only be met once. This makes sense since an attribute already entered into the tree cannot supply any more information further down the tree that it has already done once.

The result is a tree based on classification examples where every attribute node branches into the number of values possible for that attribute, until the target attribute is reached as the bottom leaf. The leaf is the decision the tree decides. This corresponds to the action output from for example a script. An example of a tree is seen in Figure 2.16.

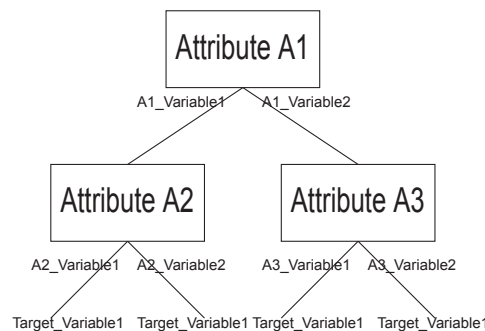


Figure 2.16: An example of a decision tree with three attributes and a target attribute (the decision that is output). Each attribute has two different values.

A more concrete example with training data can be found in [Tom97] from page 59.

At the nodes of the decision trees, we keep track of how many observations there are of a specific path. This ensures that the tree can be used for making decisions on events that have not previously been observed. If a new unknown event is observed, it is tracked through the tree until the unhandled event occurs. The tree is then traversed further through the “most likely” events, meaning the ones that has occurred most frequently.

Learning

It has already been noted that the decision tree learns by letting the ID3 algorithm classify the different attributes in the tree. However, it needs to be determined what training data to use. Since the tree needs to learn to be good at the game, it makes sense to only use training examples from games that have been won. One might say that there is still something to learn from lost games,

but in the case of RISK, there is only one winner. This means that there is up to five losers, and therefore letting their moves influence the tree will possibly build a tree that performs badly. It is not possible to, in decision trees, say that when someone loses, everything he did was bad and everything he did not do would be good. The reason for this is that the way the tree is built requires that the precise state of an attribute is known, which it would not be if “everything else” is the state of the attribute.

Converting the Tree to Rules

After the tree has been build, it can be converted to a set of rules. This is done by tracing each possible path from the root node to the leafs.

These rules are simple *if-then* statements and are therefore both easy to grasp and fast to work with.

Multidecision Trees

Another approach to decision trees is given in [Jen02]. It is based on the construction of a *decision graph*. A decision graph is a graphical representation of a decision problem. A decision graph can be converted to a decision tree with the same expressional power and vice versa.

The main difference from the decision trees explained so far is, that the trees described in [Jen02] can make more decisions before reaching a leaf node. Also, they are build by hand and not learned. To avoid misunderstandings these decision trees are from now on denoted *multidecision trees*.

In the multidecision tree, the non-leaf nodes can be chance variables or decisions. Chance variables can be seen as dice rolls, there is a *chance* that either of the events in the node occurs which is specified as a probability distribution over the possible states of the node. The decision nodes are decisions made when traversing the tree. Decision nodes have a *utility* attached. Utilities are measurements of the benefit of a specific decision being in a specific state. This utility can for example, in the sense of the RISK game, be specified as how well your armies are positioned at the board. Utilities need to be measured on a numerical scale.

Multidecision trees are solved by calculating expected *utilities*. They are calculated from the leaf nodes and up to the root. For all chance nodes, the probability for the node being in a specific state is multiplied by the utility connected to this node and the result is put on the link between the node and the utility. If the next node in the tree (moving up) is a decision node, the sum of the former mentioned links are put on the link between the nodes. If instead of a decision node, the next node is a chance node, the sum is multiplied by the chance of outputting the value resulting in the link.

Decision Trees vs. Multidecision Trees

It is not easy to evaluate at the current time which type of trees will be most beneficial to use in the sense of the RISK game. Decision trees generated by

already observed data have the benefit that they are automatically generated whereas utilities needs to be specified in the case of multidecision trees. However, some benefits by using multidecision trees are that they actually will provide information on whether it is clever to do something given that something else have been done before. This however, also depends on the decisions that needs to be made. If a series of decisions needs to be made, the use of multidecision trees becomes apparent, but if only one isolated decision is needed the multidecision tree might be wasted.

Another discussion concerning decision trees is, how much the training data can be trusted and also how much training data there is needed to make even simple decisions. Likewise, some question emerge for multidecision trees. For example whether it is possible to specify the needed utilities in such a way that it makes sense to use them as the basis for decisions.

The cost of updating the tree may also influence the decision on whether to use one type of tree over another. If decision trees are only re-calculated “offline” then there is no problem with it being time consuming. However if for some reason it turns out that it might be beneficial to re-calculate the tree while playing the game, then decision trees might not be an obvious choice of solution, depending of course on the size of the training data.

The main problem with using multidecision trees for this project is specifying the utilities attached to both decision and chance nodes. Doing this by hand will probably be far to comprehensive. The question on how good a specific decision is, can be reduced to finding a score on how well the armies are placed on the board given the specific mission. If a score for this task is available, it is most likely manageable to specify the utility for a specific decision. However finding a way create a score for this may turn out to be too large a task for this project. This discussion will be continued later in the report.

We will leave it open for the time being whether we will be using decision trees or multidecision trees to solve the tasks of the AI. It might even turn out to be task-specific what the choice should be, meaning that decision trees will be used for certain situations and multidecision trees for other.

2.8.3 Genetic Algorithms

A genetic algorithm (GA) is a technique used to search for an optimal solution to some problem. Candidate solutions for the problem are encoded in a *genome* (also called *chromosome*), and then evolved toward better solutions [Dav89]. A genome traditionally consist of a binary string of 1s and 0s, but different representations are possible. The evolution starts out with a population of random genomes, which is called the first generation. A *fitness score* is then calculated for each genome. A batch of the best performing genomes (those with best fitness) are selected and modified (mutated and combined) to form a new population. For this second generation, the fitness is then calculated again, and the whole thing is repeated until some condition is met (e.g. the genome performs satisfactory).

The Genome

A genome is an encoding of a solution to a problem. For a GA to be used in finding an optimal solution to a problem, the solution has to be able to be encoded as a string of 1s and 0s, or at least as a series of floating point numbers, which make up the genome. Examples of a genome can be seen in Figure 2.17.

Figure 2.17: Two examples of a genome. First a binary genome string of 1s and 0s. Second a genome of floating point numbers.

The Fitness Score

When a new genome has been formed, it is given a fitness score. The fitness score is a measure of how well that particular genome has performed in solving the problem. How such a fitness score is calculated depends wholly on the application.

Combining Genomes and Mutation

When a fitness score has been assigned to each genome, the pool of genomes is sorted according to fitness. The genomes are then paired to form new genomes. The selection of the genomes are biased toward the best performing genomes, but that does not rule out any of the poorer performing genomes to be selected. There are different selection methods. One of the most common are *roulette selection wheel* and *tournament selection*. Roulette selection wheel uses the fitness of a genome as the probability for it to be selected. This ensures that poor performing genomes also has a chance of being selected. Selecting only the best performing genomes might make the population converge to a sub-optimal solution - e.g. some part of the poor genome might be performing optimal, but the rest of it drags it down, and it is thereby eliminated and the optimal solution might never be found. Leaving room for such genomes to be selected is therefore important.

Tournament selection is where the probability that a genome is selected, is based on their fitness. The genomes are selected at random, ensuring that all genomes can be selected. But the probability of a genome being selected increases as its fitness increases.

When a batch of the genomes have been selected, they are paired and combined (bred) to make new genomes. This is done through *crossover* (or *recombination*) and mutation. Typically there is a probability (e.g. between 0.6 and 1.0) that a pair of genomes will actually breed. If the pair are to be bred, each of the genomes are split in two and recombined to two new genomes. An example of a recombination can be seen in Figure 2.18. The point in which the two genomes are split is called the *crossover point* and can be selected at random.

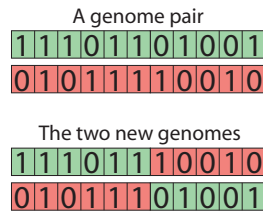


Figure 2.18: A genome pair is combined at a random crossover point and the result is two new genomes.

Mutation is when a single bit in the newly created genome is flipped (1 becomes 0 and 0 becomes 1). The probability for mutation to occur is typically quite small. Mutation helps a population that might be stuck in a sub-optimal solution to hopefully find a more optimal solution. Since mutation is a small change in the whole genome, the otherwise good performing genome should not be affected greatly. But mutation is still a change that might ruin the genome's performance, so the probability for mutation occurring should be very small (e.g. 0.01). When the genome is of floating point numbers, the mutation method should pick a random number in the genome, and then randomly change that number.

Genomes are paired until the size of the new population reached some pre-set value.

Termination Condition

When new genomes have been created and possibly mutated, the algorithm is repeated by calculating the fitness of the new genomes and repeat the selections method. At some point, when many generations of genomes have been created and removed, some termination condition is met. Such a termination condition is one of the following:

- A fixed number of generations reached,
- Some limited computation time has been reached,
- A genome has been found that satisfies some minimum criteria,
- The highest ranking genome's fitness is reaching or has reached a plateau such that successive iterations do no longer produce better results,
- Manual inspection conclude that the genomes are performing satisfactory,
- Combinations of the above.

Any of these conditions could result in the algorithm terminating. Which one is selected highly depends on the application.

2.8.4 Neural Networks

An artificial neural network (ANN) is a rough way of simulating the human brain electronically.

Generally speaking ANNs are used to derive a meaning from a complicated or noisy set of data, such as character and face recognition. It is also applicable to problems for which symbolic representations are often used, such as different elements (e.g. the board) in a RISK game. A trained ANN can be seen as an “expert” giving his opinion on a given dataset.

An ANN takes some input (e.g. real numbers) and produces some output (e.g. real numbers again). Whether the network produces the correct output is determined by the human modeler or some other benchmark. If the output is incorrect, the network is adjusted and an output is produced again. If the network produces a correct output the network is not adjusted. This is called *training the network*. Training is repeated until the modeler thinks the network performs satisfactory. However, repeated training on an ANN may result in *overfitting*. Overfitting is where the network is adjusted to respond to the training data only, and not to the general case as it was intended to do. Overfitting ANNs is also discussed in Section 4.6.5 in [Tom97].

Generally speaking, ANNs are networks made of artificial neurons connected to each other. Such networks can be connected in many different ways to form an ANN, but this section will only cover the *feedforward* type of network. But before discussing artificial neural network architecture, it is useful to discuss these artificial neurons.

The Artificial Neuron and the Perceptron

An artificial neuron (also called a *neurode*) can be modeled in different ways, depending on its usage. One type is called a *perceptron*. A perceptron receives an arbitrary number of inputs and produces a boolean (1/−1) output. Each input has an associated *weight* which determines the contribution of that input. Such a perceptron can be seen in Figure 2.19.

The perceptron calculates a linear combination of this input and outputs a 1, if the result is greater than some *threshold* and -1 otherwise. So, given input x_1 through x_n , the output $o(x_1, \dots, x_n)$ computed by the perceptron is

$$o(x_1, \dots, x_n) = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i > 0 \\ -1 & \text{otherwise} \end{cases} \quad (2.1)$$

where n is the number of inputs and w_i is the weight on input x_i .

The Sigmoid Unit

The binary 1/-1 output from a perceptron is not always useful. It is at times more useful to have a real numbered output. In such a situation one can make use of the *sigmoid unit* instead of the perceptron. It will also become apparent later, that the sigmoid unit is sometimes better when it comes to training the network.

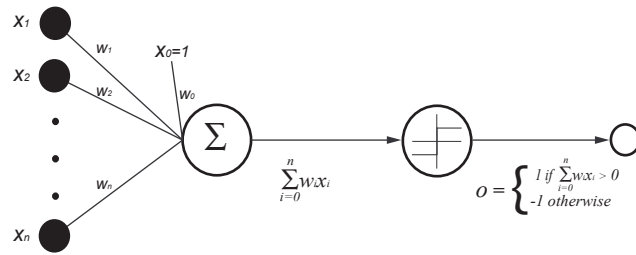


Figure 2.19: A perceptron receives an arbitrary number of inputs, calculates a linear combination of these inputs and outputs 1 if the result is greater than some threshold and -1 otherwise.

The input to a sigmoid unit is treated the same way as in the perceptron to get the net input.

The output however is instead the following function of its input

$$o(net) = \frac{1}{1 + e^{-net}} \quad (2.2)$$

The output ranges from 0 to 1, and it maps a very large input domain to a small range of outputs and is therefore often called a *squashing function* of the unit. Figure 2.20 shows the sigmoid function and Figure 2.21 shows the sigmoid unit.

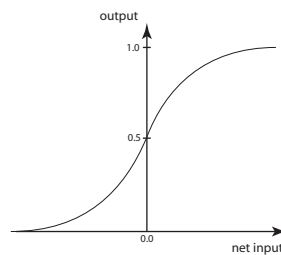


Figure 2.20: The S-shaped sigmoid function takes a large input domain and outputs a real number between 0 and 1.

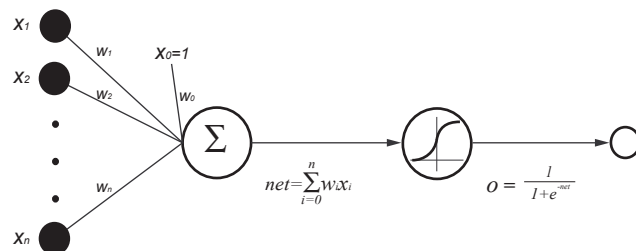


Figure 2.21: A sigmoid unit receives an arbitrary number of inputs, calculates a continuous combination of these inputs and outputs a real number between 0 and 1.

Other derivations of the sigmoid function has been made. For example, the term e^{-net} in the sigmoid function is sometimes replaced by $e^{-k \cdot net}$ where k is

some positive constant that determines the steepness of the sigmoid function. A smaller k results in a sigmoid function looking more like a straight line.

It now becomes clear why the input to a sigmoid unit should lie between 0 and 1. Lower or higher values would simply result in more or less the same output.

Neural Network Architecture

The neurode has now been explained and it is time to look at how one can put together these neurodes to form a network. This section will only cover the *multilayer feedforward* type of network, as it is the only type that applies to this project. Other types are explored in [Rus90].

The multilayer feedforward¹ network has three types of layers (see also Figure 2.22):

1. **Input layer:** Each input node is connected to each node in the hidden layer. The number of input nodes entirely depends on the application for which the network is intended. The more input nodes, the harder it is to train the network successfully, so it is important to try and group some input nodes into a single real numbered input, since this will decrease the number of inputs to the network and thereby decrease calculation time, when the network is to be trained. But as discussed on page 39 in [Rus90]: “...resist the urge to combine parameters before presentation to a neurode. It will be more efficient (...) to allow the network to take a little longer to train successfully, than if it fails to train at all.”. The importance of normalizing the input data to being between 0 and 1 is discussed throughout [Rus90]. Also it is a good idea to be general in one’s representations, in order to make the model more understandable. As long as the resolution of the input is not lost in the normalization.
2. **Hidden layer:** There can be an arbitrary number of hidden layers. Each neurode in a hidden layer is connected to each neurode in the next hidden layer. The number of hidden layers and the number of neurodes in each hidden layer also entirely depends on the applications. Too few hidden neurodes and the network will not be able to learn the training data, but too many neurodes and the network will take forever to train and also overfitting may occur. Finding the right number of hidden neurodes has been compared to being an art instead of science².
3. **Output layer:** The output neurodes take the output from neurodes in the last hidden layer and calculates an output. This is the output from the network. Again, the number of output neurodes depends on the application.

An example of using a feedforward network is character recognition. Such an example can be seen in Figure B.5 on page 152 in the Appendix.

¹In some literature called a *back-propagation model*, since the *back-propagation algorithm* is often used on this type of network. But to avoid confusion, we will only refer to this type of network as a feedforward network.

²[Rus90] recommends on page 42 to have the number of hidden neurodes be the square root of the input nodes plus output neurodes, and then add a few.

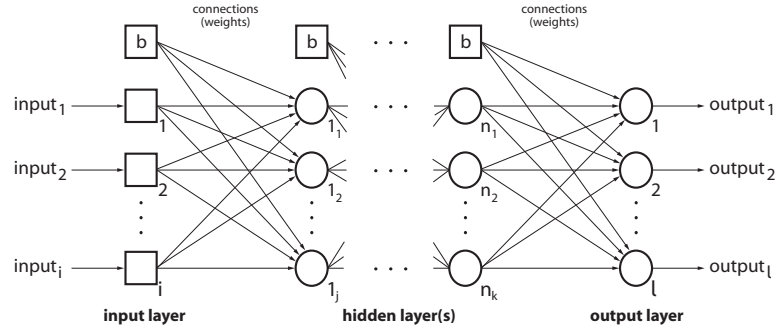


Figure 2.22: The multilayer feedforward model. Each circle is a neurode and each square is an input node. The network has i input nodes, n hidden layers and l output nodes. Hidden layer 1 has j neurodes and hidden layer n has k neurodes. The bias nodes b always input a 1.

Training Neural Networks

Training neural networks is simply a matter of tweaking the weights in the network in order to get a satisfactory output. There are different ways of doing this. In this report, two different approaches will be discussed: the *back-propagation algorithm* and a method using a collaborative combination of neural networks and genetic algorithms. The latter method is explored amongst others in [J. 92].

The Back-propagation Algorithm

Essentially, the back-propagation algorithm (BPA) learns the weights for a multilayer network with a fixed set of neurodes and interconnections. BPA attempts to minimize the difference between the actual output of the ANN and the expected output given by a set of training examples. A training example is an example of what the ANN should output when it is given a specific input. With many different training examples, the ANN should hopefully conform to the general case that they in combination describe. It should not conform to each training example separately, since there in some cases could exist two training examples with the same input, but with different output. The ANN should in such a case produce some average between the two.

In this relation, it is useful to have some measurement of how well an ANN has been trained, this is called the *error measure*. The following formula is commonly used to calculate the error measure E :

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2 \quad (2.3)$$

where *outputs* is the set of output units in the network, t_{kd} and o_{kd} are the expected output (target output) value and the actual output value associated with the k th output unit for training example d . D is the full set of training examples. E is characterized as a function of $\vec{w}$ because the output o depends on this weight vector. The aim of BPA is to minimize E .

In Figure 2.23 one can see a visualization of the *hypothesis space* H of possible weights and their associated error E . In this context the hypothesis space is the set of all possible weight values in a ANN, and the resulting error E for each combination of weight values. This produces a hypothesis space with n number of dimensions, where n is the number of weights in the network plus 1. The extra dimension is the resulting error E . In Figure 2.23 there are only two weights, which results in a 3-dimensional hypothesis space H .

Minimizing E is a matter of finding the point on H with minimum error. It has been compared with letting a small ball roll down H and where it comes to rest, is the point of minimum error. But H might have a local minimum where the ball also could come to rest - a small dent in H . The BPA is unfortunately only guaranteed to converge to such a local minimum, but in practice, it has shown to produce excellent results. Testing whether a local minimum has been reached, one could perform the BPA again from a new starting point and see if any lower error is reached.

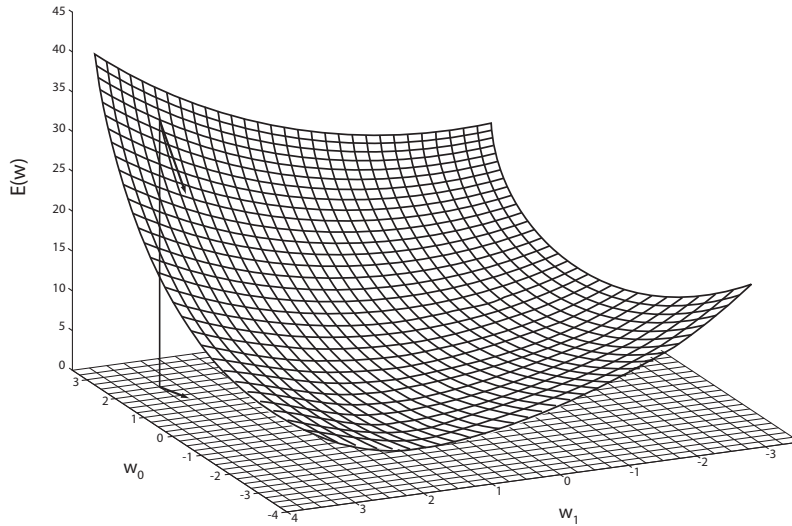


Figure 2.23: For a linear neurode with two weights, the hypothesis space H is the w_0, w_1 plane. The vertical axis indicate the error E for a given weight vector hypothesis. The arrows shows the negated gradient in a particular point, indicating which direction produce the steepest descent along the error surface.

The arrows in Figure 2.23 shows the direction in which the weights should be tweaked to produce a smaller E . Finding such a direction is a matter of calculating the derivation of $E(\vec{w})$. This is also called the *gradient* of E with respect to $\vec{w}$, written $\nabla E(\vec{w})$.

Finding a minimum of E using its gradient is called *gradient descent*. There are however some practical difficulties in applying gradient descent such as converging to a local minimum can be slow and take many gradient descent steps, and if there are many local minima in the error surface, there is no guarantee that the global minimum will be found. So a common variation on gradient descent is often used, called *stochastic gradient descent*. Stochastic gradient

descent approximates the gradient descent search by updating weights incrementally, and calculating the error for each individual training example. The BPA also applies stochastic gradient descent.

As stated previously, tweaking a given weight in an ANN is a matter of changing the weight according to the gradient of the error measure E . The change in a weight Δw_{ji} , which is the weight on the connection going from unit i to unit j (e.g. from hidden node i to output node j), is:

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \quad (2.4)$$

where η is a positive constant called the *learning rate* and E_d is the error on training example d , summed over all output neurodes in the network

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

The term $\frac{\partial E_d}{\partial w_{ji}}$ in Equation 2.4 is the *partial derivative* of E_d with respect to w_{ji} . The result of $\frac{\partial E_d}{\partial w_{ji}}$ is found in two cases: 1) where j is an output neurode, and 2) where j is a hidden neurode. The latter case is different from the first, because the change in a weight in a hidden layer influences the following layers' weights, and thereby also the weights in the output layer. In [Tom97] Δw_{ji} is shown, in the first case, to be

$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad (2.5)$$

where

$$\delta_j = (t_j - o_j) o_j (1 - o_j) \quad (2.6)$$

Here x_{ji} is the input value to neurode j , t_j is the expected (target) output and o_j is the actual output of neurode j .

In the second case, where j is a hidden neurode, Δw_{ji} is shown to be

$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad (2.7)$$

where

$$\delta_j = o_j (1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj} \quad (2.8)$$

Here $\text{Downstream}(j)$ is the set of neurodes whose immediate inputs include the output of neurode j .

Equation 2.5 and 2.7 are the same and called the *general stochastic gradient descent rule*.

Both equations contain the term $o_j(1 - o_j)$ which is the derivative of the sigmoid function. This means that, if another threshold function is to be used in the neurodes, then it has to be differentiated, and it is an advantage if the function is easily differentiated.

Until some termination condition is met, do

- For each $\langle \vec{x}, \vec{t} \rangle$ in *training_examples*, do
 1. Input $\vec{x}$ to the network and compute the output o_u of every unit u in the network.
 2. For each network output unit k , calculate its error term δ_k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k) \quad (2.9)$$

3. For each hidden unit h , calculate its error term δ_h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in \text{outputs}} w_{kh} \delta_k \quad (2.10)$$

4. Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

where

$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad (2.11)$$

Figure 2.24: The back-propagation algorithm for a feedforward network with a single hidden layer. The term unit covers input nodes, hidden neurodes and output neurodes. Each training_examples is a pair of the form $\langle \vec{x}, \vec{t} \rangle$ where $\vec{x}$ is a vector of input values and $\vec{t}$ is a vector of (target) output values. The input from unit i into unit j is denoted x_{ji} , and the weight from unit i to unit j is denoted w_{ji} . η is the learning rate (e.g., .05).

Now that we have found a the derivative of E_d , we continue with the general description of the back-propagation algorithm.

The algorithm can be seen in Figure 2.24. Before applying the BPA, a network should of course be constructed with n_{in} input nodes, n_{out} output neurodes and n_{hidden} hidden neurodes. All weights in the network should be initialized to a small random number³ (e.g. between -.05 and .05), but any value is permitted, which correspond to starting a random place in the weight space.

The BPA then starts out by calculating the output of the network given the input from the first training example. It then calculates the error δ in all outputs of each neurode in the network for this example. The weights are then updated using the general stochastic gradient descent rule. The algorithm then starts over again with a new training example. The set of training examples are repeated again, unless some termination condition is met. This could for instance be when the error measurement $E(\vec{w})$ has become sufficiently close to zero. But one should also be aware of any overfitting with repeated training. Overfitting could be continuously monitored by calculating a *generalization accuracy* which states how well the network conforms to a given generalization

³Finding the initial values of weights should be done intuitively. In [Rus90], it is recommended to use random values between -.3 and .3 for the initial weights.

example.

When looking at the BPA in detail, Expression 2.9 is easily recognized as Equation 2.6 for the output neurodes.

Expression 2.10 correspond to Equation 2.8, but since Figure 2.24 is an example of the BPA with only a single hidden layer, the term *downstream* is replaced with *outputs*, since the neurodes following any neurode in the hidden layer, is in the set of output neurodes.

In Expression 2.11 the general stochastic gradient descent rule is used and all the weights in the network is updated, in accordance with each training example.

The Combination of a Genetic Algorithm and a Neural Network

When the training examples for a ANN does not contain an expected output value t , or even when no training examples are present at all, but one still is able to tell whether an ANN is doing something right or wrong, the use of a genetic algorithm (GA) for updating weights is a possibility (also called *neuroevolution*). In the context of this project, it might become apparent, that there is nor time or facility to produce thousands of training examples with *good* playing RISK AIs, for the ANN to learn from.

The method consist of take the weights in the network and encode them into a genome, as in Figure 2.25.

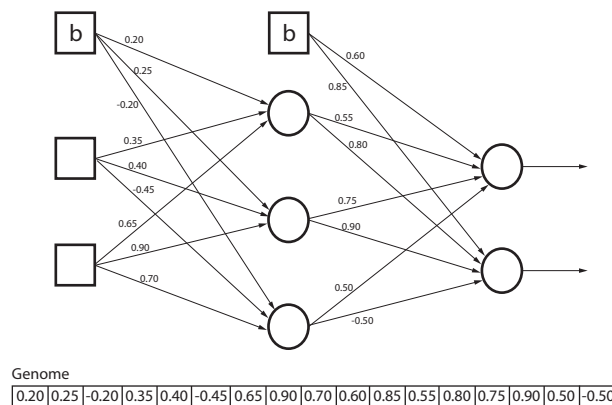


Figure 2.25: The weights in a network is encoded as a genome for a genetic algorithm.

At first a batch of random genomes is constructed and placed as weights in an ANN. A result is then computed and the fitness of each genome is increased if the performance of the given ANN is satisfactory, and decreased if not. If there is some way to graduate the good and bad results, one can also graduate the fitness score by giving a higher fitness to the best performing ANNs and a lower positive fitness to an only average performing ANN.

The best performing genomes are then selected, paired and combined, where mutation occurs. The new genomes are then placed as weights in the ANN

again, and the algorithm is repeated until the ANN performs satisfactory.

Compared to the BPA, this approach is useful when it is troublesome to calculate the exact error of a network, or when the exact error is unavailable. But also, because of the random mutation, applying GAs to ANNs potentially produce a global search of weight space and thereby avoid local minima.

The drawback of this approach is however, that it is generally slower than BPA. In [J. 92] optimization is discussed in an example, where a small population size was used (e.g. 50 individuals) with a much higher mutation rate than those used by traditional GAs. In some applications, this optimization has been shown to produce results superior to BPA.

It is also important to mention, that there exist other ways of combining GAs and ANNs. Combinations can both be supportive (they are used sequentially) or as in this described case, collaborative (they are used simultaneously). Supportive combinations typically involve using one method to prepare data for the other method. Collaborative typically involve using the GAs to determine the network topology and as in this case, determining the weights in the network. This latter method is the only GA/ANN combinatory method found to be relevant in this project, and is thereby the only one described in detail. For a more in depth description on the other, see [J. 92].

2.8.5 Bayesian Networks

A Bayesian network, or simply BN is a model of the relations between different events in a real world situation. It is useful for supplying information on a specific event based on observations on other events. This information is expressed in certainties. As an example, take a car that will not start. You see two possible causes for this: you are out of gas or the spark plugs are dirty. Both causes influences whether or not the car will start. If you *know* that the car is out of gas, you believe this is the cause of the car not starting. This will of course lower your belief that the cause is the dirty spark plugs (even though they may be dirty too).

The BN Structure

The following is the definition of a BN [Jen02]:

- A BN is a set of nodes interconnected by directed edges.
- The nodes are variables with discrete set of mutually exclusive states.
- The nodes and edges together form a directed acyclic graph.
- Each variable A with parents $B_1, \dots, B_n$ has a *potential* table attached which specifies $P(A|B_1, \dots, B_n)$.

The *potential* is a probability table for a variable stating how likely a variable is to be in a certain state. This is also referred to as the conditional probability table(CPT) for the variable.

An example of a BN is seen in Figure 2.26. If a variable has been observed to be in a certain state, the variable is said to be instantiated. Another term used in this situation is that evidence has been entered on the variable. When evidence has been entered into a BN, it is propagated. This means that the evidence is distributed to all relevant nodes in the BN.

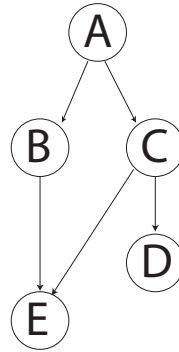


Figure 2.26: An example of the BN structure.

D-separation

In the context of BNs, and causal networks in general, *d-separation* is an important topic. It is used to determine which nodes have causal influence on each other. If for instance variable B is d-separated from A , evidence e entered on A does not effect the variable B .

The definition of d-separation, given in [Jen02], says,

Definition (d-separation): Two distinct variables A and B in a causal network are *d-separated* if, for all paths between A and B , there is an intermediate variable V (distinct from A and B) such that either

- the connection is serial or diverging and V is instantiated
- or
- the connection is converging, and neither V nor any of V 's descendants have received evidence.

Bayesian Learning

Instead of manually specifying the CPTs of a BN, which is often a very difficult task, learning can be used. This learning is done based on training data. The training data, in the case of a BN solving a task in the RISK game, is information gathered on previously played games. From these past games, statistical information is gathered concerning values of each node in the BN. For example for a node A with the states a_1 and a_2 , the occurrences of each state will be respectively $\frac{a_1}{ALL}$ and $\frac{a_2}{ALL}$, where ALL denotes the number of occurrences of A .

The previous method works if training data is available for the states of all nodes in the BN. However, this may not always be the case. Often there may be nodes which are not part of the training data. Therefore their probability distributions cannot be specified through statistics. To specify these, Bayesian learning algorithms can be used. These algorithms take already known information (found as above) and use it to tune the unspecified nodes' probabilities towards usable distributions.

The EM Algorithm

The Expectation-Maximization (EM) algorithm is an example of such a Bayesian learning algorithm. If the training data is missing information on some variables, the algorithm gives an estimate on the most likely state of the variable [Tho]. Each iteration consists of two steps, first the expectation step where the missing values in the data set is filled with calculated expected values, then the maximization step uses the filled-in data set to calculate new maximum likelihood estimate for the variables. This way missing values are filled in and can be used in the next iteration. The algorithm will continue until the difference between the maximum likelihood estimate calculated in iteration n and the maximum likelihood estimate in iteration $n - 1$ is smaller than some predefined threshold value, or has run for some predefined number of iterations. An example of use of the EM algorithm can be seen in Appendix B.4.

This can be useful in the game of Risk because although most variables are always observable there might be some which can never be observed.

Modeling Techniques

In this section some modeling techniques that might be useful when the BNs will be constructed in the Design phase.

Divorcing

The modeling of a BN can be quite tedious to do by hand since the size of the conditional probability table (CPTs) of each node grows exponentially with its number of parents. Thus the amount of probabilities to specify grows at the same speed. Specifying them by hand will therefore take a very long time. Also when computing the probabilities in the BN the sheer size of it will make the calculation time increase rapidly and the BN itself may be too big to handle. To reduce the CPT size *divorcing* can be used when modeling. An example of this can be seen on Figure 2.27.

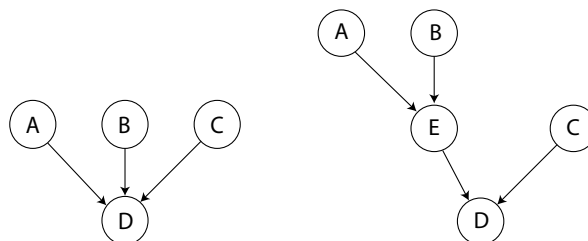


Figure 2.27: The node E divorces A and B and thereby reduces the CPT of D.

E is used to lower the number of states the variable D has to consider. The divorcing results in D having a smaller CPT if the number of states in E is lower than the combined number of states in A and B .

Time Slice

When modeling a domain that changes over time with a BN it can be done using *time slices*. For each time step a local model of the network can be built to represent the domain. This model is called a time slice. This modeling method will for example be useful when modeling the belief on how close an opponent is to occupying a continent in each of the following five rounds. The local model representing how close he is to occupying the continent this round can be reused to model how close he is the following round and the round after that and so on.

An simple example of a BN with time slices can be seen on Figure 2.28.

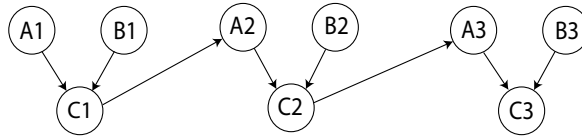


Figure 2.28: A BN with three time slices of the local network model A , B , and C .

A_1 and B_1 both effect C_1 in the model of time = 1. At time = 2, the state of A_2 depends on the state of C_1 in the previous time slice. A_2 again has effect on C_2 together with B_2 . In time = 3 the state of C_2 effects A_3 ... and so on.

A more compact way of representing the time sliced network can be seen in Figure 2.29. The dashed arrow means that C affects A in the next time slice. This is the notation that will be used in this report.

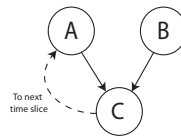


Figure 2.29: A more compact representation of the time sliced BN.

A BN containing many time slices can quickly grow too large for a computer to handle, since the clique size will be very large, because nodes from the first time slice will be parents of nodes in the following time slices. This should be kept in mind when modeling time dependent BNs later.

Development Tool

We will be using a graphical tool called *Hugin* [hug] for modeling BNs. It is both usable for modeling the BNs as well as re-calculating the CPTs after entering evidence into the BN. Also, when the BNs are going to be implemented in our Risk game, Hugin's API can directly be used. It makes it possible to use Hugin's BN calculation and updating methods from within the game.

2.8.6 Naive Bayes Classifier

A naive Bayes network is a special BN. It is one where the observed node is child of all other nodes in the network. This structure is seen in Figure 2.30, where a target attribute is child to all other attributes in the BN. In a network like this, all attributes are d-separated from each other as long as the target attribute is not instantiated. This means that the parents are independent, which is a key-assumption for the naive Bayes classifier: that all attributes in the underlying model are independent of each other. This assumption is to simplify the calculations for the classification since the probability of seeing a specific conjunction of the variables is found by multiplying the probabilities for the attributes. Using a naive Bayes classifier, it is possible to classify new training examples based on experience from training data.

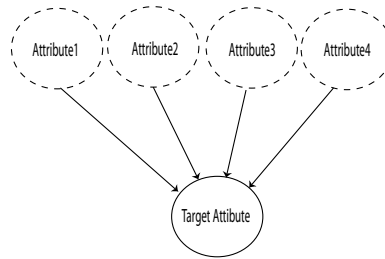


Figure 2.30: A naive Bayes network.

The Training Data

The training data for the naive Bayes classifier (NBC) is much like the one used in decision trees. Meaning that there is one target variable that is decided which is dependent on the other variables.

Learning

Classifying a new instance, which is a new observation of the attributes' states, is done in the following way:

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_i P(a_i | v_j)$$

where v_{NB} is the target value that is output. V is the set of states in the target attribute. a is an attribute from the new observation and v is the target attribute.

When the NBC is presented a new configuration of attributes to decide on it compares it to the training data. For each state v in the target attribute, $P(a_i | v)$ is calculated. Finally the product of all $P(a_i | v)$ and $P(v)$ is found. The result of the classifier is the one where v results in the highest probability.

2.8.7 Path-finding

One issue in many computer games is path-finding. The task of finding the path with the least cost from one point to the other. This is also an important

issue in RISK. If the AI needs to attack a territory T, but it does not occupy a neighboring territory to T, the AI needs to somehow find the path from a territory occupied by itself to the T, but with the least amount cost. In this context cost is the estimated amount of lost armies when attacking a territory. The probability of losing dice roll is approximated to 50/50, meaning that for every army in the enemy territory the AI will lose one army when attacking. So the cost of going from territory A to neighboring enemy territory B is the amount of armies in B.

One important note to make, is when the AI has to go from territory A to T, then any territory occupied by the AI itself cannot be in the path from A to T (except A itself, which is the starting point) - the AI cannot attack a territory occupied by itself.

Also, the fact that the AI has to leave at least one army in A when going from A to B is not a fact that the path-finding algorithm has to take into account. All the path-finding algorithm needs is a measure of the relative cost of going from A to B compared to going from A to C, and leaving one army in A is done in any case.

The A* Algorithm

The A* algorithm is a graph search algorithm used to find the shortest path between two nodes in a graph. Such an algorithm is called *admissible* if it always finds the shortest path. If A* uses a *heuristic* that never overestimates the distance to the target, A* can be proven to be admissible. A* is also *complete*, meaning that if there is a path between two nodes, then A* finds it.

Creating a graph from the territories in a RISK game is straight-forward. Every territory on the board is transformed to nodes. If armies can be moved from one territory to a neighboring one, then a link is made between the two nodes corresponding to territories. Figure 2.31 shows a standard RISK board transformed to a graph (a *RISK graph*). Each node in the RISK graph has information on the owner of the corresponding territory, and how many armies are located in that territory. As argued previously, the cost from going from territory to neighbor B is the amount of enemy armies in B.

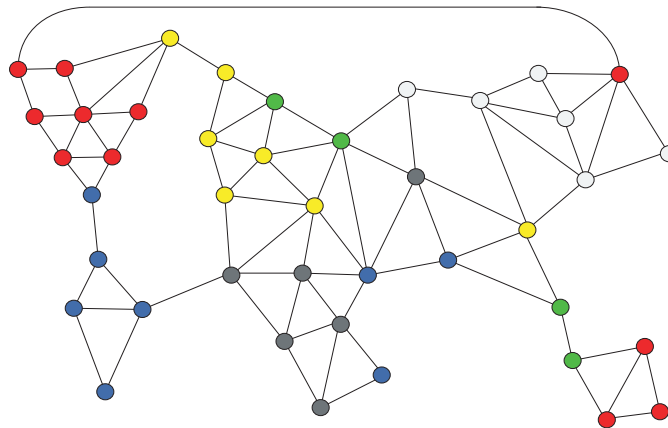


Figure 2.31: The territories of the RISK game as an undirected graph. Each player has his own color-code.

As mentioned previously, A^* uses a heuristic. The heuristic is an estimate on how far away we are from the target from any given point. And as already stated, such an heuristic should not overestimate. One could in fact just set the heuristic to zero, which never is an overestimate, and that would effectively reduce A^* to *Dijkstra's path-finding algorithm* [P. 68], which still finds the optimal path, but not as quickly. Since this is the case, Dijkstra's path-finding algorithm will not be explored further. Common heuristic functions are Euclidean Distance Heuristic (used in tile-based games and other applications where a distance can be calculated as the square-root of the distance in the x-axis squared plus the distance in the y-axis squared) or the simpler Manhattan Distance Heuristic (the distance in the x-axis plus the distance in the y-axis) [sta]. But both these are used in tile based applications, and can not be used for the RISK graph. So in this project, a heuristic function for a RISK graph has been constructed. Since a RISK graph only consists of 42 nodes, it is feasible to precalculate a heuristic table. This only needs to be calculated once and then stored in a file.

The algorithm for calculating the heuristic table for a the RISK graph is shown in Figure B.6 on page 152 in the Appendix. The algorithm does first a *depth-first search* from node A, then node B ect. calculating distances. Every time a new node is visited, the distance from the starting node (for instance A) to this node is entered into the heuristic table.

Every time a new distance $d_{A \rightarrow V}$ (the distance for going from node A to node V) is to be entered into the heuristic table, it is checked to see if a smaller value already has been entered into the table, which means that there already was found a path from A to V that was shorter. If not, $d_{A \rightarrow V}$ is entered into the heuristic table in field (A,V). The algorithm then continues by looking at the neighbors to V, always checking to see if any smaller values already has been entered into the table. This is also why the table is initialized to having values greater than the number of nodes in the graph. In this way, the first time a value $d_{A \rightarrow V}$ is checked against the value $T(A, V)$ in the table, $d_{A \rightarrow V}$ is always smaller.

An example of the RISK graph heuristic can be seen in Table 2.1 which is the heuristic table calculated from Figure 2.32.

s/t	A	B	C	D	E	F	G
A	0	1	2	1	2	2	3
B	1	0	1	1	2	2	2
C	2	1	0	1	2	2	1
D	1	1	1	0	1	1	2
E	2	2	2	1	0	1	2
F	2	2	2	1	1	0	1
G	3	2	1	2	2	1	0

Table 2.1: The heuristic table for the graph in Figure 2.32, which holds the distance of going from node s to node t.

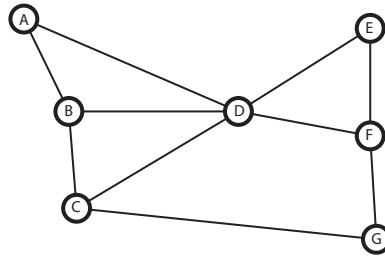


Figure 2.32: An example of a simple undirected graph. The RISK graph heuristic table for this graph can be seen in Table 2.1.

When the heuristic table has been set up, the main A* algorithm can begin. The main steps of the A* algorithm are described in Appendix B.3.

Avoid Getting Trapped

When conquering a series of territories it is important to avoid getting trapped halfway in a territory from which it is not possible to continue the attack. For example if the computer player has a big army in South Africa and he wants to conquer the territories of Congo, East Africa, and Madagascar. If the computer player's attack sequence was first to conquer East Africa and then Madagascar then it would be impossible to conquer Congo this turn, because Madagascar has only already occupied territories as neighbors. This problem will also appear when conquering whole continents - getting trapped halfway through would be a problem. Besides avoid getting trapped it would also be useful if the attack sequence would end in a territory from which the newly conquered continent can be defended, for example a border territory. If the border territory is already occupied then it would also be fine if the path ended with conquering a neighbor to it, then the armies can be transferred to the border in the reinforcement phase.

The problem of avoiding getting trapped is solved by the A* algorithm described earlier. However, the A* algorithm is not suited for making a path covering all territories in a continent. It is only usable for point-to-point paths. When determining how a whole continent can be conquered an other approach must be used. It must be able to solve the problem of finding a path from an occupied territory that leads through all enemy territories in the continent and ends in an occupied border territory (neighbor) or other occupied territory needing defense in that continent.

This problem is similar to the problem of finding a hamiltonian path in an undirected graph as described in [Gar93]. A hamiltonian path is defined as follows:

A *hamiltonian path* is a path in an undirected graph $G = \langle V, E \rangle$ that contains each vertex in V , starts in $s \in V$ and ends in $t \in V$.

All algorithms for finding hamiltonian path has been proven to be NP complete and so no efficient way of doing this has been found. Reducing the search space is therefore important. From the graph representing the board, the sub graph G can be made which only contains the relevant vertices, namely s and

all vertices representing enemy territories in the continent. Irrelevant territories such as occupied territories other than s and all territories that are not a part of the continent will this way not be a part of G .

The hamiltonian path could for example be found using a recursive algorithm traversing the graph using depth-first search. Starting from s the algorithm should continue to recursively visit neighbor nodes until t is reached using a path where all nodes have been visited. The algorithm should stop either when all possible path combinations have been tried or a hamiltonian path has been found.

2.8.8 Influence Maps

Influence maps are used to deduct information from a map. Such a map could be the board in a RISK game or a map in some real-time strategy game. The influence map has the same size as the map from which the information is deducted.

The information retrieved from the maps depends on the application, but the most common is the *unit-influence grid map*, where the influence map is first initialized to zero and the influence of all units on the map is then counted. Friendly units count as positive influence and enemy units as negative. The influence on a tile spread outwards to adjacent tiles - also described in [Woo02]. The decay of influence over time is decided by the designer and is application dependent. Such an influence map can be seen in Figure 2.33, where a friendly unit can use the influence map to sneak around the enemy units or find a good attack point.

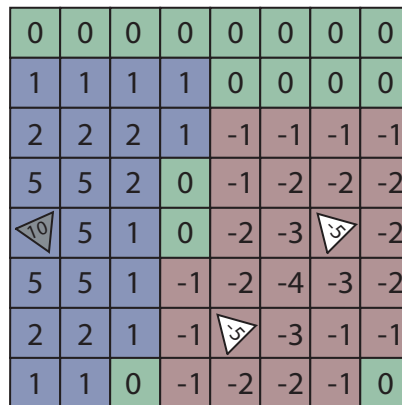


Figure 2.33: The positive influence of a friendly unit and negative influence of an enemy unit.

In RISK, influence maps can be used as a tool for retrieving information on the board. For instance the potential attacking reach of a given player. Using such an influence map is an easy way of determining whether a player has the power to reach a given continent or territory in the successive rounds. In this project an influence map will always have the same representation as the board in RISK. An example of an influence map in a RISK game can be seen in Figure 2.34. We call such an influence map an *Estimated Influence Map*, since this is the influence of a player if lose an estimated amount of armies when attacking a

territory. This estimate could be based on the actual probability of how many armies one on average would lose.

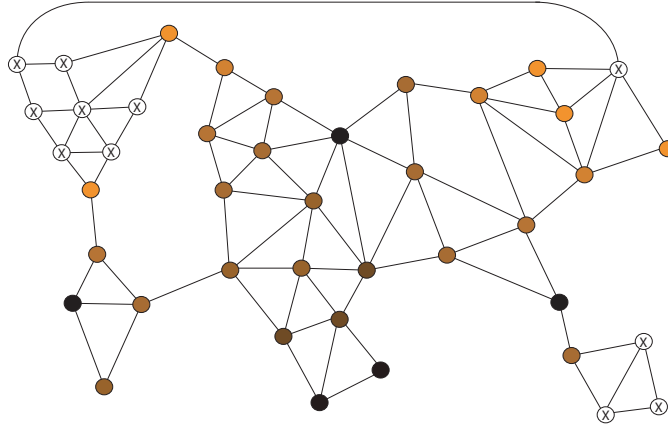


Figure 2.34: The influence of the red player on enemy territories. The territories occupied by the red player are marked with an X. The more orange a territory is, the more influence the red player has on it. Territories occupied by red influence neighboring territories more, if there is many armies in red's territory. If a territory is black, then the red player has no influence on it. The influence is decreased proportionally to the amount of armies in enemy territory, which means this is an influence map depicting the territories that the red player could potentially conquer.

2.8.9 The Board State

Representing the board state is often an infeasible task because of the large number of states most of the variables can have. This is especially a problem when wanting to represent the current board state in a decision tree. The tree generally needs to be rebuilt every time anything on the board has changed. This means that the tree might have to be rebuilt more than once during an AI's turn. Therefore, it would be expedient to have a more abstract way of representing the map, with fewer variables, but still precise enough to base the AI's estimations and planning on.

However, finding a simpler way of representing the variables of something without losing precision might be out of the scope of this project. There are, however, still a few things that can be done to narrow the state space of the board representation. Combining states in groups is a way to narrow the state space. This can of course not be done without losing precision, but by grouping the states in a rational way the loss of precision can be made minimal. As an example take the variable "Number of Armies". This variable can (in theory) have the same number of states as there are positive integers. Through analyzing past games, it should be possible to derive which are the most commonly used distribution of states. These are for instance the integers between zero and ten. These values are then used as a state each and everything else is placed in another state. In this example the variable "Number of Armies" will have the states $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, > 10\}$. Of course, these states need to be carefully derived in order for this to work. If somehow the states are clas-

sified inappropriate this will not work since valuable information is lost. It is important that the “other” state is not within commonly used values.

2.8.10 Scoring the Board

It would be beneficial to have a way to “score” the board state, meaning having a way to determine how well positioned ones armies are in contrast to the other players’ armies. This would be beneficial in the sense that it would make it possible to tell if the action just made was good or bad, meaning did the board state get changed to the better or did it put you in a less beneficial situation. Also it would be beneficial to foresee if an action you are going to perform will end in a better or worse situation.

However it is hard to determine how such a scoring function should be made. There are many factors that influences the score. We will try to list them here:

Number of occupied territories:

This is important since this directly influences how many reinforcements you will receive the next round.

Mission-specific territories/continents:

It is better to occupy mission-specific territories/continents since this will eventually make you win the game.

Continents:

Owning and being able to defend entire continents is good since this will boost the reinforcements received.

How well defended the occupied territories are:

It does not matter if 26 territories are occupied if an opponent conquers them all the following round.

The placement of armies.

It is of no use to have a large army if it is trapped somewhere on the board.

Opponents’ mission-specific territories/continents.

To avoid the opponents win the game, it is good to hold territories/continents from their missions.

From this list the question on how these internally should count against each other in a scoring function. It is obvious that the positive aspects should all count in a positive way. For example owning a lot of territories is good. It is even better if some of them are mission-specific. But then the question emerges: If some of the territories will be lost the following round, how much should losing a non-mission-specific territory count versus a mission-specific? If we say that it is better to loose for instance three non-mission territories over a mission-specific it would have the result that one less army would be available the following round. Also there is a consideration whether it is better to

hold a mission-specific territory over an opponent's mission-specific territory.

Of course it is possible to make approximations on how to do this. One approach is to only consider how the board looks compared to ones own mission. This includes analyzing the board with respect to the mission only considering the number of mission-specific territories and the threat to these. The boards closer to winning will then score higher. A flaw with this procedure is that the AI using it will not consider if it is actually helping another opponent win the game by clearing the opponent's mission-specific territories in order to conquer his own.

Another approximation would be the exact opposite of that just mentioned. A method that takes into consideration only what the opponents' are trying to obtain and will give a higher score if the opponents' are poorly positioned on the board. However, an AI using this method would be far to defensive and would not consider to fulfill its own mission.

Finding a hybrid of these two extremes would be what is needed to have a durable scoring function. However, because of the mentioned problems with conflicting interests, this task of finding this scoring function might be to comprehensive for this project, and we will therefore not look further into it at the time being. Moreover, a scoring function looking at the board configuration at a given time will be of limited use. The board as it looks at a given time does not tell anything about how it would look the round after. The exact same board configuration might be good in one game and bad in an other. It depends on what the opponents are planning to do before it gets your turn again.

A consequence of not being able to give a score on the board is, that we must rule out the use of multidecision trees. These are ruled out since not having a score function gives us no other choice than specifying utilities manually in these. This task is far to complex, and therefore they are no longer considered a usable AI technique for this project.

2.9 Discussion on Learning

Before we go into detail with combining AI techniques with our framework, we will discuss how the training of the learning AIs is performed.

2.9.1 Supervised Learning vs. Reinforcement Learning

Supervised learning and reinforcement learning are two different techniques in machine learning. Supervised learning is most commonly based on *training examples*, where the actions performed by an agent is compared to the training example. Supervised learning use *instructive feedback* stating whether the action performed is correct or incorrect, and what the correct action should have been. In reinforcement learning the action is evaluated by the performance of the action - it uses *evaluation feedback*. The action is given a score (or fitness). The best action in a given situation is only found through the performance of the other possible actions done in the same situation [Ric98], which means that

determining whether a given action is good or not, can only be done by comparing the action's fitness to another action's fitness. The action with the best fitness is then the best action.

However, it has been stated previously, that a score for how well a player is positioned on the board can not be found, which apparently renders reinforcement learning unusable in RISK, since the score for an action is most easily calculated based on how well a player is positioned on the board. However, it has to be investigated further, for each module in the framework, whether or not this is the case.

2.9.2 Training Examples

In many of the learning algorithms, training examples can be used. These training examples will in many cases be previously played RISK games.

For these games to be valid training examples for the AI, it is needed that the players in these games are at least of average skill, at best perfectly playing, which means, that the random player is ruled out, because nothing can be learned from such a player. But the players also need to be varied in their play, so that the AI can learn all the different aspects of the game. But in all cases a big basket of varying training examples for the AI to chew through is needed, and the only way to obtain this, is to have some AI players play all those games.

A way of doing this, is to have all the different AIs found in the following section play against each other. All the combinations of AIs should preferably result in varied ways of playing, but also somewhat competent play. It is a requirement that the different aspects of the AI does not learn at first. This means that no learning algorithms are used to make decisions in any part of the AI. The reason for this is that no training data has been created at this time, therefore the learning AIs have nothing to learn from. When the non-learning AIs have played a number of games, a large set of training examples should have been created. This training data is then fed to the learning AIs. These learning AIs then begin to play each other and possibly also the best non-learning AIs to generate more training examples.

2.9.3 Winners and Losers in Training Data

In these training examples, both winners and losers are kept. It is straightforward to see how learning algorithms can learn from winners - everything winners do are examples of good behavior. But everything a loser does are examples of bad behavior. This is in fact a restriction, since not everything a loser does is necessarily bad - perhaps he has performed perfectly up until a point, but then makes the worst decision ever making him lose the game. Up until that point, he was actually a good example. This also goes for winners - perhaps they were just lucky. But identifying the good behavior in an overall bad example and vice versa is beyond the scope of this project, and losers are therefore deemed bad examples.

But learning from bad examples is in fact not impossible. Turning a bad example into a good example can be done by stating that doing the opposite of the bad example is a good example. This is like saying that everything the bad example did not do, is good. This is not entirely true, since doing "something else" could be even worse. But over time, with many bad examples, the good

examples should be narrowed down, making the behavior of the learning algorithm overall good. Intuitively, this statement is true, but whether or not that is in fact the case, can only be shown by examining the problem closer. As of now, we assume the statement to be true.

Analyzing how to incorporate the training with bad examples in the AI techniques used in this project, is done for each of the techniques separately.

Neural Networks

As stated above, learning from bad examples is a matter of doing the opposite of the example. Training an ANN to do the complete opposite of an example, is a matter of negating the error in the network. This is like changing the gradient of the error to go in the opposite direction. Whether or not this is a better direction of the gradient than the actual gradient is difficult to say, but this has been discussed above.

Bayesian Networks

When calculating the probability distribution of an observable node in the Bayesian networks (recall that the probability distributions are based on counting in training examples), it is possible to learn from both winners and losers. Normally, in won games, the state of a specific variable will get a “+1” count if this state was in the configuration of the training data for the winning player. For all lost games this +1 is distributed evenly over all other states within the variable. This is the same of saying that everything else was better than the choice made in the lost game.

Naive Bayes Classifiers

This can be done in the same manner as for Bayesian Networks.

Decision Trees

There does not seem to be any way to learn a decision tree to do the opposite of a given action. For instance, what if the action where to attack a given territory - the opposite of that could be to attack some other territory or perhaps not attack at all. If there were some way to have the target value in a given training example be a set of all the other actions than the given action, then decision trees could learn from bad examples. But this can not be modeled in a decision tree, therefore only the winners will be used to train decision trees.

2.9.4 Training Neural Networks

Based on the above discussion on supervised training and reinforcement learning, a discussion on training of ANNs is needed.

When the ANNs should be trained through training examples, the common back-propagation algorithm will be used. But when the ANNs has learned the training examples and further training is needed to improve the AI beyond the training examples, two things could be done: 1) training is continued where all the AIs generate training examples for the ANNs (ANNs can not generate

training examples for themselves, since an given ANN can not learn anything from a training example made by the same ANN), 2) neuroevolution (as described in the section “The Combination of a Genetic Algorithm and a Neural Network” in Section 2.8.4) is be used. The first case is supervised learning and the latter case is reinforcement learning. As long as there exist training examples, then supervised learning is used, but after that, the latter of the above cases is used. This is to investigate the performance of neuroevolution compared to learning through training examples. The immediate force of neuroevolution is that it does not require any training examples. The creation of a large enough set of training examples is many times a big hurdle in learning algorithms.

But since neuroevolution is employed, it means that when the ANN for a particular task is discussed in the following sections, it is needed to also analyze how performance of the ANN is evaluated - how the fitness of each genome is determined. For instance, if an ANN should perform the task of placing armies in the reinforcement phase, the performance of such an ANN could be evaluated either through some algorithm that determines whether or not the placement was good or poor immediately after it has been done (*locally*), or it could be evaluated through the outcome of the game (*globally*) - if the game was won, all placements were good, and if the game was lost, all placements were poor. Both of these are in fact supervised learning, except that the correct result is not directly stated. Correct results are kept for mutation and crossover and the incorrect result are not used.

There are pros and cons to both approaches. Evaluation at the end of the game has the advantage that some approach that locally would be evaluated as poor, but necessary for a positive outcome, is correctly evaluated as good. On the other hand though, if a method would locally be evaluated as good, but globally poor because the game was lost, it might still be good, if the context was different (or if the outcome was due to bad luck). Also, the consequences of some poor action might become apparent many rounds later and not picked up by the local evaluation. As illustrated here, the advantages of both approaches are likely to be equal, so the selection of the evaluation method for a given task, will be based on which method is the most useful in the given situation. In the cases where both methods are equally useful and equally feasible to implement, the global evaluation method will probably be chosen, since this is straightforward - if the game was won, the task performance was good, if the game was lost, the performance was poor.

Local evaluation could also run into some local optimal evaluation, that in the long run will not end in a good situation, which is also discussed in [Joh02]. Global evaluation does not have this flaw. But if global evaluation is used, then it will require many more games of RISK to be played, since the algorithms are only evolved once per played games (at the end) - local evaluation will evolve the algorithms many times in a single played game.

Training on Normalized Output

In some of the modules in the framework, the output from the module should be normalized to sum to 1 (this is the case in the IG “Estimating Opponents’ Missions” and the goal distribution from the Master Prioritizer etc.). When

using the back-propagation algorithm, there is some tweaking that needs to be done, for the network to learn from a training example, where the data is normalized. This is best illustrated in an example:

Some ANN will produce the following output when presented some input: $[0.6, 0.6, 0.6]$. The training example that the ANN should be compared to is: $[0.5, 0.25, 0.25]$ - which sum to 1. The output from the ANN should then be normalized to: $[\frac{0.6}{1.8} = 0.33, \frac{0.6}{1.8} = 0.33, \frac{0.6}{1.8} = 0.33]$. The error in the network is then calculated by comparing the actual output and the training example. But before the error can be used in the network, the change in the output by the normalizing-procedure should be converted back in the error, or else the error would be factors smaller in relation to the actual output. So after the error is calculated, it should be multiplied by the sum of the actual output. For instance, the error for the above example is $[0.0376, -0.0177, -0.0177]$ and this should then be multiplied by 1.8. This way, the error can be related to the output of the network and used to change the network weights accordingly. On the other hand, the training example could also be lengthened by a factor of 1.8 and the error then divided by 1.8, but this changes the training data according to whatever output is produced by the ANN and this seems to be untenable, so the first approach is selected.

Extended Training with Neuroevolution

Another matter that need a bit of discussing, is the fact that when neuroevolution of the ANNs starts, the weights in the networks can not be initialized to random. A genome that makes up the weights in an ANN needs to somehow be initialized to represent the weights already learned from the training examples.

This is in essence a matter of finding the trade-off between losing the already learned behavior and potentially climbing out of a local minimum. This means that each value in the first generation of a genome should be initialized to the corresponding weight in an ANN adding a random value between $-\eta, \eta$, where η is some constant. Finding the exact value of η is difficult and might even be dependent on which ANN the genomes should be used in. One could argue that it is closely related to the mutation rate of the genetic algorithm. Further investigation into each of the learned ANN should provide some information on the value of η .

2.9.5 Training With Naive Bayes Classifiers

In the naive Bayesian networks, training data will be available for all nodes. This means that the probability for a specific state within an attribute can be calculated by taking the number of occurrences of the attribute being in that state, and divide it by the number of occurrences of that attribute being in any state. The same method is also usable in other Bayesian networks than the naive, on the nodes that we have information on. The nodes that cannot be specified directly through the training data will have to be learned through some Bayesian learning method.

2.10 Combining AI Techniques with our Framework

Now that our IG-MP-RP framework has been defined and the techniques for implementing AI in games has been described, it is time to combine the two.

The following sections will describe where the AI techniques can be used in the IG, MP and RP. The usefulness of the different techniques will be discussed for each part of the framework. This section will result in the summarizing Table 2.2 that depicts which techniques can be used where.

	Script	Random	Decision Tree	Neural Net.	Bayesian Net.	Naive B. C.
Initial army placement	x	x	x	x		x
Estimate opponents' missions	x	x	x	x	x	x
How close an opponent is to winning	x	x	x	x		x
The opponents' next moves	x	x				
How close an opponent is to owning a continent	x	x	x	x	x	x
MP Goal Weighting	x	x		x		
Make attack plan	x	x				
Calculate attack plan cost	x	x				
Prioritize attack plan	x	x	x	x	x	x
Discard attack plan	x	x	x	x		x
Calculate defense cost	x	x	x	x	x	x
Prioritize territory needing defense	x	x	x	x		x
Cash cards	x	x	x	x		x
Place armies	x	x	x	x		x
Transfer armies	x	x		x		

Table 2.2: Each of the modules in the framework has been combined with the AI techniques. First the Initial Army Placement module, then each of the Information Givers, then the Master Prioritizer and lastly the modules in the Round Planner.

2.10.1 Common Techniques

There are two AI techniques that are common enough that they can be in all the parts of the AI. These are *random* and *scripted* AIs. The reason these two can always be used are that they are flexible enough to solve any task. The *random* technique can be applied anywhere the AI needs to make a choice. The quality of such an AI will, as discussed earlier, probably be poor, but nevertheless it is a technique that needs to be considered among others. The *scripting* technique applies fixed functionality to different tasks, therefore it is always possible to make a script that solves a specific task.

The use of genetic algorithms does not fit into any of the tasks the framework is divided into. These tasks are so specific that they cannot be divided into smaller parts. And since genetic algorithms are used for finding an optimal combination of smaller parts, they are not useful within those used in this framework. They will, in this project, be used for finding the best combinations of AI techniques. More information on this is found in section 3.9.

2.10.2 Information Givers

As explained previously, the IGs provide information to the MP. There are a number of different IGs, that, in the following sections, each will be combined with the AI techniques.

Estimate Opponent's Mission

This IG should deliver an estimation on how likely (represented as a percentage) it is that a given opponent has each mission.

Estimating Opponents' Missions

Input: The Past.

Output: List of missions with an attached certainty on each.

Decision trees:

There should be no problem in building a decision tree for this task. The information needed in the training examples is not just past board states. It must specify which actions have been performed by the different players throughout the game. However, there is a slight problem. The tree cannot output a list of missions but merely one mission. Also the decided mission cannot have an attached certainty. The reason for this is that the tree always outputs its best estimate. It is not easy to obtain a certainty for how good this estimate is using a decision tree.

Neural network:

It is not expedient to directly have the past as input nodes in an ANN. There is possibly too many nodes for the ANN to ever learn anything. Some other representation of the past has to be found for it to be input. If for instance some unique numeric value for each possible instance of the past can be found, then it is possible for it to be input in

an ANN. The number of output neurodes would correspond to the number of missions in RISK for each player. Each output would be a numeric value stating the certainty that a player has a given mission. The collected output neurodes for a player has to be normalized to sum to 1.

There is no need for evaluative feedback in this ANN, since the opponent's actual mission is known. Evaluation of the network can then be done locally during the game by comparing the estimated mission with the actual mission and adjusting the weights accordingly, but this could be considered cheating, since real players would not have access to the others player's mission. Evaluation could also be done at the end of the game: the actual mission is used as a training example for the network. The network can be seen on Figure 2.35.

It seems reasonable to have an ANN learn to estimate the opponents' missions. It is only prohibited by the fact that a good representation of the past needs to be found.

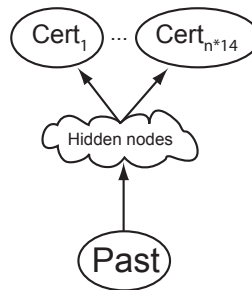


Figure 2.35: Neural network for the task “Estimate Opponent’s Mission”. The input node is *Past*. The output node $Cert_1 \dots Cert_{n*14}$ gives a certainty for each of the n players having each of the 14 missions.

Bayesian network:

A BN is not an obvious choice for solving this task. The problem is representing the past in a model that would not have all possible actions as parents to the output node (the observed node).

Naive Bayes classifier:

As mentioned the past may be represented a single node for all entries in the past. These nodes are parents to the observed node. The parents are all independent which means that this task can be solved using a naive Bayes classifier.

How Close an Opponent Is to Winning

This IG should, given an estimate on what mission a given opponent has, deliver an estimate on how close that opponent is to fulfilling that mission and winning the game (represented as a probability of the opponent winning in the next round, a probability for the opponent winning in the next round again and so on - up to some finite number of rounds).

How Close An Opponent Is To Winning

Input: Opponent's mission, the board, number of RISK cards owned by opponents.

Output: An estimation of how close the opponent is to winning in each of the next five rounds.

Decision trees:

The main problem using a decision tree for this task is that the output is specified over time. This is not handled well by a single tree. Also the number of states in some the input variables may limit the use of decision trees. Also the number of attributes and the number of states within the attributes may limit the use of decision trees. However, we do not know at the current time how large the set of input variables can be before it compromises the use of the technique, but we may choose to group states to keep the possible number of outputs from each variable as small as possible.

Neural network:

An ANN for estimating how close an opponent is to winning, has the estimate on his mission as input, the board and the number of RISK cards each opponent owns. The difficult part of the input is representing the board as input nodes. Some kind of numeric description of the board is needed. This could for instance be 42 nodes each representing the amount of armies in a given territory.

As output, there would be a finite set of neurodes outputting each a floating point number. At first, the ANN will learn from already played games as training examples. But afterwards it has no way of knowing whether the probabilities found are correct or not, so this module can not be trained any further using neuroevolution, which has also been argued previously in the discussion on learning. Another way is saying that if the AI won the game, then everything the IG produced was correct through out that game. meaning that the evaluation is done globally.

Many of the modules in the framework take the board as input. Therefore, a way of inputting the state of the board to ANNs is needed. If such a representation of the board is not found, or found to lose too much information, then ANNs can not be used efficiently for modules that take the board as input. Finding a good representation on the board is explored further in the Design chapter.

Bayesian network:

It should be possible to model this task in a BN. However, there is a problem that needs to be solved considering the board. It may not be efficient to model the whole board into the network and therefore this needs to be handled to make the BN efficient for solving this problem.

Naive Bayes classifier:

The board input is internally independent, meaning that the number of armies in one territory has no influence on how many are placed

in the other. This is naturally not totally correct since a player having many armies in a territory may cause that other players reinforce in the surrounding territories. But this does not matter in the current situation since we are interested in how the board looks at the current time, and not how the territories influences each other through the following rounds. Therefore a NBC is useful to solve this task.

The Opponents' Next Moves

This IG should deliver a list of actions that is an estimation on what the opponent is going to do in the following round.

The Opponents' Next Moves

Input: The output from the other IGs, the board, opponents' number of RISK cards.

Output: A list for each player of his next moves.

The AI can use the information that would be available to the opponent, and plug that in the AI's own MP and thereby estimate what goal the opponent would deliver to its RP, and thereby what move the opponent would make.

The AI could even directly calculate what mission the opponent thinks the AI has, and then use that information to better estimate his next move. The only things that the AI is unsure of, is the opponent's actual mission and RISK cards, this has to be estimated.

One could say that this way of estimating the opponent's next move, will in all cases be the best way. No other AI technique would be better; if there existed a better way of estimating the opponent's next move, the AI would use that technique, when doing its own moves.

There is of course some assumptions that need to hold, for this to be true: It has to be assumed that the opponent and the AI are equals - they use the same reasoning to make a move. Otherwise, if the opponent for instance were inferior to the AI, the AI might estimate a very good move for the opponent, but it probably would not be the correct one, thus making it a very poor estimate. This however is not a problem. If the AI is in fact better than the opponent overestimating it will not matter since the AI will most likely win anyway. This also holds the other way: if the opponent is a better player than the AI, the AI will underestimate it. However, since the opponent was already better, he would most likely win anyway.

Using the AI's own MP-RP to estimate an opponent's next move, is in any case a good suggestion for solving this problem, but other AI techniques should of course also be investigated.

One problem in this is the following. Let us say, that the AI itself is an instance of the AI structure (IG-MP-RP) called S_{self} , and when the opponent's next move is to be calculated, a new instance of the AI $S_{opponent}$ is created, so the same AI structure is used. But $S_{opponent}$ also contains an instance of the "Estimate opponent's next move" IG which then again creates a new instance of the AI in its calculation, which repeats the whole procedure again. To avoid this problem, $S_{opponent}$ needs to nullify the "Estimate opponent's next move" IG making it output nothing. This means that the AI instance $S_{opponent}$ becomes less precise, but the loss of precision must be accepted in order to incorporate

this behavior. This is also why other AI techniques needs to be explored.

AI techniques that involves learning will not be considered as an alternative to this. The reason for this is that for the learning AI to fully learn the opponent's next move, it will have to learn from a lot of games against the opponent AI. This will adjust the AI so that it can predict moves from exactly that type of opponent. But what happens if the AI suddenly plays against another type of opponent with a different playing style? Then all the learned moves cannot be used for anything and so does not predict the opponent's next move well. A solution to this could be if the AI could recognize the type of opponent by analyzing its playing style and comparing it to playing styles from a number of previously played games where the opponents have been classified. This would be a big task because even if the opponent was recognized it would need to play a great number of turns in the game before it would happen. And to recognize the type of opponent very late in the game will be too late.

Another way to make a learning AI good at estimating the opponent's next move would be if it could learn how the opponent plays *during* the game instead of learning from a large number of training examples. This is hard because the AI will not have much information to base its estimates on. Finding a method capable of doing that would be a complex task and falls outside that scope of this project, so that kind of AI will not be investigated any further.

To sum up, here is a list on whether each AI technique could be useful to solve the problem of determining the opponents' next moves:

Decision trees:

This is a method based on learning and will not be considered as an option.

Neural network:

This is a method based on learning and will not be considered as an option.

Bayesian network:

It is very difficult to model this task in a BN. The problem mainly lies in the output node which should contain all possible moves. Also it is not possible to output more than one move from this node. What we are interested in here is a list of all moves the opponent would do, not just one.

Naive Bayes classifier:

This is a method based on learning and will not be considered as an option.

How Close an Opponent Is to Owning a Continent

This IG should deliver an estimate on how close each opponent is to owning each continent. This can be delivered as a list of probabilities with a percentage on how close each player is to do so.

How Close An Opponent Is To Owning A Continent

Input: The board, RISK cards owned by a player.

Output: A percentage for each player on how close they are to owning each continent.

Decision trees:

The major problem of using a decision tree for this task is that the output basically is a list of the players stating how close each is to occupying each continent. However this may, in this case, be possible to solve by building more than one tree and using the each output in combination to get the final result.

Neural network:

An ANN for this IG would take the input as stated above, which has been discussed in previous modules. The output would be one output for each of the players, and for each player an output for each continent and for each continent there would be a finite number of outputs. Each output would state the probability for the player owning a continent in one round from now, two rounds from now and so on. The network can only be trained with training examples. When it is time to train the network further, some method of determining whether the ANN was right or wrong is needed. For this, the script for this IG could be used, but this is what the training examples are based on, and we would not get any further. Another way is to say that if the AI won the game, then everything the IG produced was correct throughout that game.

An example of such a network can be seen on Figure 2.36.

Whether or not such a network would learn, is difficult to say. There could be for instance $5 \times 6 \times 5 = 150$ output neurodes, which is really many. It would probably require many more training examples than other networks in the framework, which is a significant drawback. Perhaps the learning rate for such a network should be increased, such that it would more quickly fit the training data. But this could quite easily end in overfitting, which is bad too.

Bayesian network:

It is possible to model this in a BN. However, there is a problem that must be solved with the board. Having an input node for each territory will make the size of the BN too cumbersome to work with. Therefore some abstraction over the board must be done for the BN to work.

Naive Bayes classifier:

This task can also be solved by naive Bayes classification. All inputs are independent and parents to the output node.

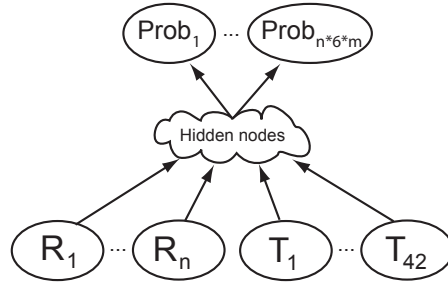


Figure 2.36: Neural network for the task “How Close an Opponent Is to Owning a Continent”. Input nodes $T_1 \dots T_{42}$ represents the configuration of each of the 42 territories on the board. $R_1 \dots R_n$ represents the number of RISK cards owned by each of the n players. The output node $Prob_1 \dots Prob_{n*6*m}$ gives a probability of each of the n players is going to own each of the 6 continents in the next m rounds.

2.10.3 Master Prioritizer

The MP should take each of the Information Givers as input and produce a prioritized list of goals.

Master Prioritizer

Input: Each of the Information Givers

Output: A prioritized goal distribution.

Decision trees:

Building a decision tree to handle how to prioritize the goals would be inefficient. The reason for this is, that there are 37 goals (including sub goals) that each needs a priority between 0 and 100. However, the states are mutually exclusive meaning that if one point is given to a goal, it is not given to any other goal. This gives far too many possible combinations in the target attribute for this to be expedient to solve with a decision tree. It might be possible to group the priorities into larger intervals this might be unfortunate since a dividing it into too many intervals does not fix the problem, and dividing it into too few will simply make the MP too imprecise to use as a basis for the AI.

Neural network:

An ANN could take the information from each of the IGs. But a unique numeric encoding of “opponent’s next move” is then needed, since this is a list of actions of arbitrary length. If such an encoding can be found, then that IG can be input to an ANN. The output of such an ANN is of course a priority of each of the goals. But the output has to be normalized, because the importance of the goals all sum to 100%.

Evaluating whether or not the goal distribution produced by the ANN is a good distribution, cannot be done by evaluating each goal distribution locally at the end of each round, because a successful outcome of the game might be dependent on a goal distribution that might locally be judged as being a poor distribution. So it is the set of all goal distributions through a game, that should be evaluated. This is easily done by

evaluating all goal distributions as good in a won game.

Again, such an ANN is dependent on finding a good representation of some input. If a good representation can be found, then this ANN should be able to learn to output a correct goal distribution.

Bayesian network:

We do not find it possible to model this in other ways than a naive Bayes classifier.

Naive Bayes classifier:

Even though some of the IGs use each other internally, when their estimates are delivered to the MP they can be considered independent. This means that the network can be considered as a naive Bayes network, meaning that a naive Bayes classifier can solve the problem with the distribution as target attribute. However, this presents the same problem as with decision trees with the possible values of this target being too many. Therefore we do not consider naive Bayes classifier as an option for the round planner.

2.10.4 Round Planner

This section will describe which techniques can be used in which of the RP's modules.

Make attack plan

This module outputs a sequence of territories consisting of a path going from the starting territory (occupied by the AI) through a number of opponent territories, ending at the target territory (occupied by an opponent).

Input: Occupied territory, target territory, MP goals, board.

Output: Attack plan.

Decision trees:

Building a decision tree for this task would be inefficient since using a path-finding approach as the A-star algorithm is more beneficial.

Neural network:

A path-finding approach would also be more beneficial than constructing a neural network to do path-finding.

Bayesian network:

A path-finding approach would also be more beneficial than constructing a BN to do path-finding.

Naive Bayes classifier:

A path-finding approach is more beneficial than using naive Bayes classifier for this problem.

Calculate attack plan cost

This module calculates the minimum army cost and estimated army cost for following a given attack plan.

Input: Attack plan, board, MP goals.

Output: Minimum army cost, estimated army cost.

Decision trees:

It would not be efficient to build a decision tree for this task, since it can be done through simple calculations.

Neural network:

An ANN would also be impractical compared to simple calculations.

Bayesian network:

It is more useful to calculate the cost of an attack plan directly. Also the it is very difficult to model both an attack plan and the board in the same model without the size of it exploding. There needs to be a node for each territory stating whether or not it is in an attack plan and also a node for each territory with information on owner and number of armies placed on it. This size problem would make the BN unusable since it would require too much space to work.

Prioritize attack plan

This module calculates a priority for a given attack plan based on the MP goals it helps satisfy.

Input: Attack plan, MP goals, board.

Output: Priority.

Decision trees:

The problem with building a decision tree for this task is that part of the input is an attack plan. The value of this attribute is not a discrete and finite value since there theoretically are unlimited combinations of territories that makes up an attack plan. However, if this problem can be solved there would not be any problem in giving an attack plan a priority.

Neural network:

Having an attack plan as input to a ANN is difficult. There could be made some numeric abstraction of the given attack plan or perhaps a hash value that can cover all configurations of attack plans. So constructing a general ANN taking an attack plan as input would have a number of inputs corresponding to the maximum length of attack plans. If such a representation can be constructed and used as input along with the goal distribution from the MP, then a ANN could be used to prioritize an attack plan. Evaluation could be done locally by assessing whether or not

the priority of the attack plans resulted in a better position for the AI, but it has been argued in Section 2.8.10 that a scoring function for how well a player is positioned on the board can not be found. It therefore needs to be done globally at the end of the game.

Since such an ANN only output a single value, then it seems to be very useful for this module. The only difficulty is finding a good representation of an attack plan, and as argued before, finding a good representation of the board.

Bayesian network:

The problem with having both the board and an attack plan in the same network also persists in this task.

Bayesian networks:

It may be possible to make a BN model that solves this problem. However, it will require that we can find a way to model how well the attack plan satisfies the different goals and also how it dissatisfies other since this should actually be used to decide whether or not an attack plan is good. At this moment we do not have any knowledge on how large the BN will be, and will leave it unsaid for now if it is possible or not to create a working model that is not naive.

naive Bayes classifier:

It is possible to model this in a naive Bayes network where all inputs are represented as parent nodes to a node representing the output. The independence assumption holds in this case, since all the inputs are independent nodes.

Discard Attack Plan

Given an attack plan and the priority and costs associated with it, this module discards attack plans which should not be taken into consideration by the Attack AI.

Input: Priority, minimum army cost, estimated army cost, number of reinforcements without cashing Risk cards, Risk cards.

Output: Yes/No.

Decision trees:

The inputs to this task are relatively few and easy to represent, it should therefore be possible to build the tree that will output “No” if an attack plan is reasonable and “Yes” if not.

Neural network:

An ANN for this module could have 5 input nodes - one for each of the inputs to the module. The output would be binary 1/-1 for yes/no.

After training based on training examples, the evaluating in neuroevolution could be done locally, but it is difficult to say locally if some plan should not have been discarded compared to another plan. The evaluation could instead be done when the game has ended - if a game was won, the discarding was good, if not, it was bad.

Such an ANN has a limited amount of input and a single output, so it seems very likely that such an ANN is easily trainable. It is not dependent on finding some representation of values, all values can be input directly.

Bayesian network:

This problem seems inefficient to solve with a BN, it seems more appropriate to solve with a naive Bayes classifier since all inputs are independent and it does not seem there is a obvious way to model this in other way.

Naive Bayes classifier:

A naive Bayes classifier seems useful to solve this task..

Calculate defense cost

This module gives an estimate on how many extra armies are needed to defend a given territory from being conquered by an enemy.

Input: Territory, board.

Output: Estimated army cost.

Decision trees:

This task seems reasonable to solve with a decision tree. The only problems that may be is due to the number of attributes and also the number of variables within these attributes.

Neural network:

Such an ANN could have the input as stated above. The board has been discussed previously and a representing a territory in a value can be done easily by converting each of the 42 territories to unique values between 0 and 1. The output could be a single numeric value. Training will first be done by training examples. Further training can be done locally by seeing if the territory were actually attacked, and if the defense held. But there is no way of knowing what the actual placement module will do with the cost given by this module. Perhaps only 45%, 50% or 95% of the cost will be used, and then it can not be evaluated whether this cost was correct or not. So evaluation should be done at the end of the game, saying that the output of this module was correct if the AI won.

Bayesian network:

It is possible to model this task in a BN. An obvious thing to benefit from modeling is an abstraction over the board. By making such a model,

having all territories as input may be avoided. If such a model cannot be found, the only way to model this is through a naive BN.

naive Bayes classifier:

Since all inputs are independent, it is possible to make a naive BN for this task. This may be solved using a naive Bayes classifier.

Prioritize territory needing defense

This module will prioritize how important it is to give a territory reinforcements compared to other territories.

Input: Territory, MP goals, opponents' next moves, number of cards owned by opponents.

Output: Priority.

Decision trees:

It is possible to use the ID3 algorithm to build a decision tree solving this problem from the given input. However, it requires that there is an input node for each territory stating whether or not it is part of an opponent's next move.

Neural network:

Again, some numeric representation of "the opponents' next moves" is needed for it to be used as input to an ANN along with the other input. The output would be a floating point number. After the training with training examples, the evaluation could be done locally in the following way: if the AI's position has been weakened the following round, but since a scoring function of the AI's position on the board has been argued against, then the evaluation can only be performed globally at the end of the game.

Such an ANN is again dependent on finding a good representation of some input, which may result in the ANN having trouble learning. In this case it might not be important to have the opponents' next moves as input. Whether or not the ANN will learn to produce a good priority based mainly on the MP goals will become apparent in further investigation.

Bayesian network:

This problem cannot be solved without having a node for each territory stating whether or not it is in an opponent's next moves (or some other representation of this) and a node for each goal stating the importance of it. Since these nodes will all be parents of the priority node, this task is more suitable for a naive Bayes classifier.

Naive Bayes Classifier:

As stated above a naive Bayes Classifier is better suited for this task, than a Bayesian Network.

Cash cards

This module determines whether the Risk cards on hand should be cashed this turn.

Input: List of priorities, list of minimum and estimated number of armies needed to satisfy each priority, number of armies gained from occupying territories and continents, Risk cards on hand.

Output: Yes/no.

Decision trees:

It is not possible to represent any of the three lists in one node. Therefore, decision trees are not considered an option for this task.

Neural network:

The list of priorities has a maximum size of 42, as does the list of number of armies needed to satisfy each priority. This fact makes it possible to have the lists as input to ANN. This could be done by having each element in the list be an input node. The third type of input could be a single numeric value representing the number of armies gained, and the last input, the RISK cards on the hand only needs some unique numeric encoding for it to be used as an input in an ANN. So it is possible to create an ANN with the inputs stated above. The output would be a binary 1/-1 for yes/no. After the training with training examples, the evaluation of the output of the module can be done locally by seeing if the AI had too few armies to execute all attack plans, meaning that the module should have outputted *yes* instead of a possible *no*. But that will only teach to ANN to output *yes* - it will never be taught not to output *yes*. It might be better to evaluate this module globally.

Such an ANN would have over 42×3 nodes, possibly making it hard to teach. With this fact in mind, ANNs might not be suitable for this module. But this might be empirically proven not to be the case, so the design of such an ANN is continued. It is highly dependent on the amount of training data created.

Bayesian network:

It might be possible to model this task in a BN. However, to do so it is needed to separate some of the input nodes from the output nodes to avoid the BN being a naive Bayes.

This problem seems reasonable to model as a naive Bayes classifier.

Bayesian networks:

It does not seem possible to build a BN for this task that would not be naive. Therefore a naive Bayes classifier is a better choice of technique to solve this problem. This also makes sense since there are no dependencies between the input nodes.

Place armies

This module takes care of placing the armies received as reinforcement one by one in the occupied territories on the board.

Input: List of territories needing reinforcements, list of priorities, list of minimum and estimated costs, number of reinforcements left.

Output: Territory.

Decision trees:

A problem with this task concerning decision trees is, that part of the input are three connected lists: The list of territories where each territory has a priority and both a minimum and estimated cost attached. This may pose a minor problem since then basically there are two attributes depending on one. This problem must be solved in order to build the tree.

Neural network:

The list of territories has a maximum size of 42, so an ANN with 42 inputs plus an input with the number of reinforcements is needed. The output would then be a numeric encoding of a territory. After learning from training examples, the ANN be done locally, but there is no way of knowing whether this placement was good compared to some other placement, so the evaluation is done globally at the end of the game. If a game was lost, all placings were poor and if it won, then all were good.

Such an ANN would be very similar to that for the cash card module, and therefore have the same problems.

Bayesian network:

It does not seem that there is any other way of modeling this than as a naive Bayes.

Naive Bayes classifier:

Since the only possible BN is a naive Bayes and none of the parent nodes causally influences each other, the problem can be solved using a naive Bayes classifier.

Transfer armies

In the end of the turn, this modules responsibility is to determine if a transfer should be made and if so, between which territories the army transfer should be made and the number of armies to include in the transfer.

Input: MP goals, opponent's next move, board, number of cards owned by opponents.

Output: Source territory, destination territory, number of armies to transfer (0 = no transfer).

Decision trees:

Because of the number of outputs a decision trees cannot be used for this task.

Neural network:

Again, a numeric encoding for “opponents’ next move” is needed. With that along with the other input an ANN can be constructed. There would be 3 outputs, two with numeric encodings for territories and a value for the number of armies to transfer. After the learning from training examples, learning is done through evaluating the performance of the ANN. This evaluation does not make sense to be done locally, since the consequences of a poor transfer could become apparent many rounds after the transfer has been made. So a global evaluation at the end of the game is made.

Bayesian network:

It is not possible to model this as a BN because of the three outputs that are closely connected.

Naive Bayes classifier:

Because of the output, NBCs cannot be used either for this task.

2.10.5 Initial Army Placement

At the very beginning of the game this module is responsible for placing armies on the board.

Place army

Input: Board, Mission.

Output: Territory to place in.

Decision trees:

It is possible to use already played games as a basis for how to initially place armies. If the input is represented in a useful way a decision tree can be used for this task.

Neural network:

An ANN for the initial army placement is straight-forward. The board and the AI’s own mission as input, and a single output is needed. The output would be a numeric encoding of the territory to place an army in. At first, the learning of the ANN would be based on already played games as training examples. This should result in a fairly good placement AI. Secondly, the learning could thereafter be based on the outcome of games played, where it is the performance of the placement AI that is evaluated. But the initial army placement has little influence on the outcome of the game, and all that is needed from a initial placement AI is a good starting point for the main AI to pick up. So intuitively the learning of such an ANN would suffice with training examples. Otherwise, if it turns out that such an ANN would not suffice, and further training is

needed, the evaluation is done globally at the end of the game.

If an good representation of the board can be found, then such an ANN should be well suited for this module.

Bayesian network:

Since problem needs to deal with the whole map on a territory basis, it seems there is no way of representing the map than with one node for each territory. Also we need both information on who occupies the territory as well as how many armies are placed in it. These will all be parents to the same observed node, which will make the task more suitable for a naive Bayes classifier.

Naive Bayes classifier:

Since the inputs are independent, this task seems suited for a naive Bayes classifier solution.

2.10.6 The Past

It is also important to analyze if there are some AI techniques to be used, when the past information is collected. But since the past is simply a matter of collecting information from the board and track which actions are done by players throughout the game there is no need for some AI technique to reason about this. This information could just be put into some list as the game is played. For the activity maps, there will most likely be used influence maps in conjunction with other tools to make up, for instance, a script.

2.11 Replacing the Whole Framework

It is possible to replace the presented IG-MP-RP framework with other frameworks. A way of doing this is having a set of inputs, a “black box”, and a set of outputs as seen in Figure 2.37. Here the input could be the board and the outputs would be an action or a list of actions that alters the board. The black box could for example be a neural network, a decision tree or even a script.



Figure 2.37: The black box approach where, used in the RISK game, the input would be observations of the board and the output would be movements on the board. The black box could be any suitable technique.

This approach could without doubt present a well-playing AI, but it would not suffice for this project. Since the focus is to test different combinations of different AI techniques that combined gives a well-playing AI we need the more modular approach. Exchanging our framework with the “black box” framework would simply make it impossible for us to replace smaller parts of the AI with other techniques and we would thereby not have any ground for

testing. What we would have would be the basis for comparing different AIs playing a game of RISK which is not what is wanted directly.

Chapter 3

Design

Now that the AI framework has been defined and AI techniques explored, it is time to go into details on the design of the framework. This chapter will start out with a section designing the data structures in the framework and how each module communicate. This will be followed by a section on different prerequisites which are needed, before going into detail on the design of each module in the framework. In the Analysis, each module was combined with a set of AI techniques. Each AI technique for each module is designed.

In some cases, where the design of a given module is straightforward and an elongated design discussion would be tedious, a summary will be made, and the reader will then be referred to the Appendices, which contain a more explicit description.

3.1 Overall Framework Design

Generally speaking, the design of the framework will follow the object-oriented paradigm, inspired by [Lar00].

To make the behavior of each module easily replaceable, each of the modules are classes. Each module has a public *run()* method, which has the input as the module. The method then outputs what ever the module should output.

The modules in the RP are divided into a Planner, Reinforce, Attack and Fortify part. The following sections will discuss how these are transformed to classes.

3.1.1 Planner Class

The Planner class contains the “Make Attack Plan”, “Calculate Attack Plan Cost”, “Prioritize Attack Plan”, “Discard Attack Plan”, “Calculate Defense Cost” and “Prioritize Territory Needing Defense” classes. The Planner itself also contains a *run()* method that takes the input as described in the Analysis. It then calls the *run()* methods on each of the associated classes in the right order and produce in return a set of attack plans and a defense list as output. The Planner

class can be seen as a module in itself, since it performs no actions in the game, it only calculates attack plans and a defense list.

3.1.2 CashCards Class

On the other hand, “Cash Cards” potentially changes the state of the game, since the AI’s number of RISK cards are reduced and the number of reinforcements increase dramatically. A class containing this behavior is therefore needed, and a Reinforce class is therefore not needed.

The CashCards class also has a *run()* method, taking the input as the “Cash Cards” module described in the Analysis and produce the corresponding output.

3.1.3 PlaceArmies Class

Such a class is also needed for the same reasons as above. It also has a *run()* method.

3.1.4 Attack Class

This is actually not a module in the framework, as argued in the Analysis. It merely performs the attack plans produced by the Reinforce class. But this behavior is still needed and a class is therefore also needed. This also has a *run()* method. This method takes a prioritized list of attack plans as input. This method has two different kinds of output: “Revise Attack Plan” or “Done”, depending on the outcome of the attacks.

3.1.5 TransferArmies Class

The fortify part of the RP only contains the “Transfer Armies” module. So there is no need for a Fortify class. The *run()* method in the TransferArmies class takes the input as described in the Analysis and produce no output. Instead it performs a transfer army action on the board.

3.1.6 InitialArmyPlacement Class

This class is the “Initial Army Placement” module. This also contains a *run()* method that places a single army in a territory. It takes the input as described in the Analysis.

3.1.7 Framework Class

The Framework class is the overall class that contains all of the above classes. This class is also responsible for performing the right action depending on which phase the game is in.

The *private* methods for this behavior is the following:

initializeRound(): In the beginning of each round, this method is called. It calls the *run()* method on each IG and calls the *run()* method on the MP.

It then calls the *run()* method on the Planner which produce attack plans and a defense list. This is also output from this method.

reviseAttackPlan(): Every time the situation on the board deviates from the plans made by the Planner, the attack plans are revised. As written in the Analysis, this could either be done by calling all of the IGs, the MP and the Planner again or just the Planner. A solution for this can only be found through implementation and testing the two approaches. The output of this method is also attack plans and a defense list.

The *public* methods are:

initialArmyPlacement(): The method merely calls the *run()* method on the InitialArmyPlacement class, and this method therefore takes the same input.

cashCards(): This is the first decision in each round, so *initializeRounds()* is called first. This means that this method has the following input: The board, the past and the opponents' number of RISK cards. When the initialization is done, the *run()* method on the CashCards class is called.

placeArmies(): This method calls the *run()* method on the PlaceArmies class. It therefore takes the same input.

attack(): This method calls the *run()* method on the Attack class and have the same input. If the Attack class outputs "Revise Attack Plan", then *attack()* calls *reviseAttackPlan()* and then calls *run()* on the Attack class again, until it outputs "Done".

fortify(): This method calls the *run()* method on the TransferArmies class and have the same input.

None of the above methods have any output - they all perform their corresponding action in the game.

3.2 Dataflow in Framework

The design of the data structures and data flow in the framework is mostly straight forward. However, there are a few points that will be presented in this section for clarity. The full design of the data flow is found in Appendix C.1.

One point that is made in the design is that the past is divided into two parts: an *action past* and a *board past*. This is shown in Figure C.1 on page 158.

Another important point is how the attack and defense lists output from the planner AI are merged before entering the reinforce AI as seen in Figure 2.12. The attack plan is reduced to be just the starting territory with the attached cost and priority. When merging the list of start territories with the list of territories needing defense, the same territory could appear more than once because it could both need defense and be the starting territory in an attack plan. In that case the territory with the highest priority should be kept, the other should be deleted from the list.

3.3 Training Examples

This section will cover how the training examples for the AI are designed. As written throughout the Analysis, training examples are a big part of learning AIs. Decision trees, artificial neural networks and bayesian networks use training examples in their training. Especially for ANNs, there has to be a great detail in the training examples. All decisions made in each module by the AIs in the training examples needs to be stored for the ANNs to learn from. So a training example for a single RISK game will look as in Figure C.2 on page 162 in the Appendix.

The training example starts out by stating the number of players in the game, their name and color and their initial ownership of territories. Remember that each territory starts out with a single army. All territories are divided equally between the players. Then the initial army placement is recorded. Each player places in turn a single army on a territory that he occupies. The game then begins with the first round and the output of each module that delivers useful information is recorded in turn for each player. When the output of all modules has been recorded for a single player, the RISK card earned (if any) is also recorded. The recording continues with round 2 and so on, until some player has won.

The state of the board throughout a game is not recorded. The state of the board can be derived from the output of the modules in the training example. If the board is updated everytime a player does an action on the board, then there is no need to also record the board state.

3.4 Prerequisites

Before going into detail with the design of each of the modules in the framework, some prerequisites needs to be discussed.

3.4.1 Estimated Army Cost Function

In many of the scripts in the design of the modules in the framework, it is necessary to have an estimate on how many armies you would lose if you attacked a territory occupied by n enemy armies. An *Estimated Army Cost Function* that calculates the amount of armies an attacking army on average will lose when attacking a territory is needed. A general formula has been developed to calculate this cost [ris], $\Gamma(n)$ calculates the cost given n armies:

$$\Gamma(n) = 0.8534144n - 0.2213413(1 - (-0.525359)^n) \quad (3.1)$$

where n is the amount of armies in the defending territory.

There could not found any form of proof for the validity of Equation 3.1, so the formula had to be proven empirically. A program that simulated the dice rolls of an attack on a territory with different amounts of defending armies was constructed. The program produced with the average of 100000 attacks the same results as the formula within a range of ± 0.01 with $n = 20$ and ± 0.06 with $n = 200$ which is acceptable.

3.4.2 Probable Outcome of a Battle

Another estimate is also needed: the probable outcome of a battle between neighboring territories when the attacker has x armies and the defender has y armies. A general formula for this could not be found nor derived. So a brute force algorithm simulating 100000 battles were constructed. It is assumed that the attacker always attacks with 3 dice when possible and the defender always defends with 2 dice when possible. Table 3.1 shows result of the algorithm - the probability of winning an attack.

X\Y	1	2	3	4	5	6	7	8	9	10
1	41.601	10.862	2.667	0.639	0.166	0.004	0.001	<0.001	<0.001	<0.001
2	75.533	36.295	20.475	9.080	4.921	2.109	1.234	0.495	0.252	0.113
3	91.707	65.767	46.952	31.530	20.702	13.265	8.305	5.299	3.166	2.060
4	97.105	78.494	64.120	47.554	35.983	25.393	17.950	12.124	8.649	5.645
5	99.029	89.158	76.996	63.864	51.023	39.743	29.778	22.397	16.067	11.836
6	99.656	93.505	85.613	74.596	63.978	52.087	42.459	32.732	25.845	19.278
7	99.888	96.666	90.892	83.257	73.615	64.032	53.875	44.716	35.894	28.502
8	99.955	98.033	94.735	88.826	81.907	73.001	64.613	54.799	46.544	38.024
9	99.990	99.012	96.756	93.009	87.477	80.907	72.431	64.696	55.671	48.228
10	99.996	99.462	98.098	95.414	91.632	86.134	79.905	72.608	65.090	57.065

Table 3.1: The probability in percent of winning an attack with x armies against y defending armies.

3.4.3 Conquering a Continent

Another prerequisite is the design of an algorithm that makes the AI able to conquer whole continents in an efficient manner.

An algorithm must be found that is able to find a path through every enemy territory on continent given an occupied start territory and a destination territory. This problem corresponds to finding a hamiltonian path from node s to node t in a graph G as explained in Section 2.8.7 on Page 58. An example of such a path can be seen on Figure 3.1 where a path is wanted from s to t in order to conquer Africa. The white nodes represents occupied territories while the dark nodes are enemy territories. The hamiltonian path is shown with thick edges.

A way to find out if the graph contains a hamiltonian path between s and t would be to simply try all different ways of constructing the path. If unsuccessful then the path cannot be made. Doing it this way will be sufficient since the size of the graph will be limited by the largest continent size, in this case Asia. The Asia graph contains 12 nodes with an average number of edges to follow per node at 3.5. This will result in a fairly limited number of possible hamiltonian paths.

So although the problem is NP complete it is easily calculated in the Asia graph. A simple recursive depth-first search algorithm will be sufficient.

Before the algorithm is run the graph is constructed, corresponding to the board graph pruned from territories outside the continent in question and occupied territories, except from s .

Figure C.3 on page 163 in the Appendix shows an example of an algorithm finding a hamiltonian path between s and t . This algorithm will from now on be referred to as the HamPath algorithm.

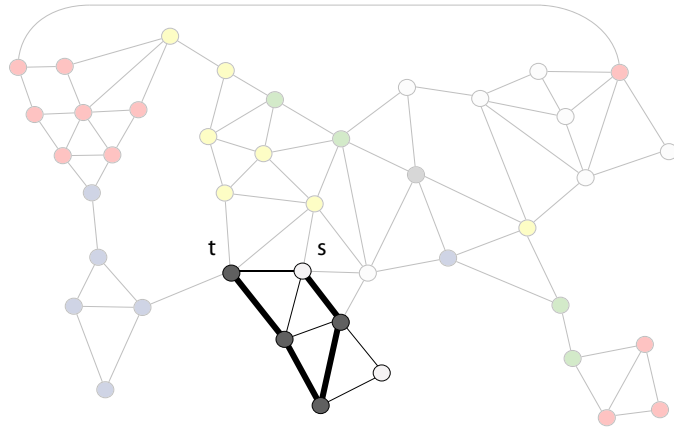


Figure 3.1: The continent of Africa as an undirected graph with white nodes represents occupied territories while the dark nodes are enemy territories. A hamiltonian path from s to t is shown with thick edges.

3.5 Information Givers

This section will deal with the design of the Information Givers (IG). Each IG has, in the Analysis, been associated with a number of AI techniques. Each associated technique will be designed for each IG.

3.5.1 Estimating Opponents' Missions

This section will cover the design of the IG that provides an estimation on each opponents' mission.

Input: The Past.

Output: List of missions with an attached certainty on each.

Scripting

The following is performed for each player in the game:

When estimating an opponent's mission, there are different indicators to what mission he is trying to fulfill. These indicators are different from mission to mission, so each mission type will be dealt with separately.

Conquer a set of continents: When estimating if an opponent has this type of mission, an influence map I_C is constructed for each opponent. This influence map is initialized to all zero values. The action past is then iterated through and for each territory the opponent has attacked, the corresponding value in the influence map is increased by 1. If a player has conquered a whole continent and is defending it successfully, then there will not be any actions in that area, but it is still an indicator that this continent is in the player's mission. It is therefore needed, that if a

player occupies an entire continent, each territory in that continent is increased by 1 on the influence map. If however, a player's continent is lost and not recaptured by that player, then it is an indicator that the continent is of little interest to him - then all territories in that continent should be decreased by 0.5 on the influence map.

Overall this will give a picture of where a given player's interest lies, and also an indicator of his mission in the game.

Kill X player: For each opponent the following is done: create an array of integers where each element in the array is reserved for each of the players. Each element in the list holds a counter for how many times the opponent has attacked each of the other players. If one of the players has been attacked more than the others, then it is an indicator of the opponent having that player as his mission. If the counters for each player is more or less equal, it is an indicator that the opponent does not have this type of mission.

Occupy a number of territories: These two types of missions are very hard to estimate. "Occupy 24 territories" is almost impossible to estimate since the opponent most likely would go for any of the continents, in order to build up his army - just as any of the other players would. "Occupy 18 territories with at least 2 armies on each" is a bit easier: if any of the territories occupied by the opponent does not have any enemy neighbors, but still have more than a single army, then it is an indicator that the opponent has this mission. This is not a very strong indicator and very hard to spot until it is too late.

If at any point in the game, the opponent fulfills one of these missions, but does not win the game, then it is clear that the opponent can not have this mission, and it should not be considered any more. Also, if an opponent is estimated of having the mission "Kill Red Player" and the red player is killed by another player, then the estimate on "Kill Red Player" should be transferred to "Occupy 24 territories" since this is what would have happened, if the opponent actually had the mission "Kill Red Player".

Finding an estimate on each mission for each opponent is done in the following way:

For each value in a territory on the influence map I_C , the value $C_{continent}$ for the continent in which the territory lies, is increased. The total number of points given to all continents is called a . When all these points have been counted, the result is a value for C_{Asia} , $C_{NorthAmerica}$, $C_{SouthAmerica}$, C_{Africa} , C_{Europe} and $C_{Australia}$, which summed together results in a . The same is done each time the opponent attacks a specific player. The values C_{red} , C_{blue} etc. are used for that. When summed, they sum up to b . There is also a value for each mission in the game, $M_{Asia_SouthAmerica}$, M_{Asia_Africa} , $M_{NorthAmerica_Africa}$, $M_{NorthAmerica_Australia}$, $M_{Europe_SouthAmerica_3rd}$, $M_{Europe_Australia_3rd}$, one M_{color} for each color and lastly M_{24} and M_{18} . These are the values in interest - the estimates on each mission.

Each of the "set of continents" are calculated as follows:

$$\begin{aligned}
M_{Asia_SouthAmerica} &= \frac{1}{4} \left(\frac{C_{Asia}}{a} + \frac{C_{SouthAmerica}}{a} \right) \\
M_{Asia_Africa} &= \frac{1}{4} \left(\frac{C_{Asia}}{a} + \frac{C_{Africa}}{a} \right) \\
M_{NorthAmerica_Africa} &= \frac{1}{4} \left(\frac{C_{NorthAmerica}}{a} + \frac{C_{Africa}}{a} \right) \\
M_{NorthAmerica_Australia} &= \frac{1}{4} \left(\frac{C_{NorthAmerica}}{a} + \frac{C_{Australia}}{a} \right) \\
M_{Europe_SouthAmerica_3rd} &= \frac{1}{4} \left(\frac{C_{Europe}}{a} + \frac{C_{SouthAmerica}}{a} \right) \\
M_{Europe_Australia_3rd} &= \frac{1}{4} \left(\frac{C_{Europe}}{a} + \frac{C_{Australia}}{a} \right)
\end{aligned}$$

The $\frac{1}{4}$ is used because each of the two terms in parenthesis needs to be multiplied by $\frac{1}{2}$ to divide evenly between two, and to divide evenly between the “continent” and “kill color” group, another $\frac{1}{2}$ needs to be multiplied to the whole expression resulting in the $\frac{1}{4}$.

The fact that there is a 3rd continent in the two last missions is ignored. It could theoretically be any of the other continents that is the 3rd continent.

The M_{color} is then calculated

$$M_{color} = \frac{|Players|}{6} \left(\frac{C_{color}}{2b} \right)$$

where $|Players|$ is the number of players in the game. If this first term was not included, the M_{color} missions would get a higher estimate than the “set of continents” missions: If there was 3 players, they would split $\frac{0.5}{3}$ on average each and the “set of continents” mission would get $\frac{0.5}{6}$ on average each. This however, means that the remaining $\frac{6-|Players|}{12}$ share will then be evenly distributed to each “set of continents” missions estimate.

Now all “set of continents” and “kill color” missions all sum to 1. And their estimate will be based on how many times the opponent has killed a given color and attacked in a given continent.

Appendix C.4 on page 163 contains an example on how M_{color} , $M_{Asia_SouthAmerica}$, M_{Asia_Africa} etc. should be calculated.

But M_{24} and M_{18} both have zero. So the shares given to these two will need to be taken from all the others, so they all still will sum to 1. As argued, these are difficult to estimate. One could argue that, the closer the opponent is to occupying 24 territories, the more likely it is, that he has mission M_{24} . Perhaps it could just mean that he has difficulty fulfilling some other mission, but it seems like the only way of estimating anything on M_{24} . Therefore

$$M_{24} = \frac{1}{10} \left(\frac{|T_O|}{24} \right)$$

where T_O is the number of territories occupied by the opponent. The term $\frac{1}{10}$ is to give M_{24} the same share as the others - further tests will show if this term should be altered. The value of M_{24} should now be evenly taken from the other mission estimates, making them all still sum to 1.

M_{18} could be calculated in the same manner:

$$M_{18} = \frac{1}{10} \left(\frac{|T_o|}{18} \right)$$

where T_o is the number of territories occupied by the opponent, that has *more than 1 army*. The value of M_{18} should also be evenly taken from each of the other mission estimates (not including M_{24}).

This should hopefully all give some estimate on which mission a given opponent has. It is of course difficult to estimate on the first few rounds of a game, but the estimates should become more precise as the game progresses.

Decision Trees

Even though the ID3 algorithm should be able to build the decision tree automatically, there are still some things that needs to be specified. These are, the target attribute, and considerations over what data to use from the past. There must be one tree build for each player. The target attribute for the tree contains each of the missions with a specific certainty on each of the missions. The attributes used to build the tree are *how often has the player attacked each of the other players* for each player, an attribute for *how often has a player attacked the different continents* for each continent on the board. Two more attributes are needed to catch the "Occupy X territories"-missions. These must state how close each player is to fulfilling these missions, only in the sense of occupied territories on the board and not how close they are to actually fulfilling it.

For each round in the training data games, this information is derived from the past. The first two attributes are specified as probabilities. How often a certain player is attacked is calculated by $\frac{\text{AttacksOnSpecificPlayer}}{\text{TotalNumberOfAttacks}}$. In the same way the players attacking continents can be derived: $\frac{\text{AttacksOnSpecificContinent}}{\text{TotalNumberOfAttacks}}$. The last two attributes are calculated directly from the board information stored in the past. The "Occupy 24 territories" is calculated by $\frac{T_O}{24}$, where T_O is the number of territories occupied by the given player. The "Occupy 18 Territories with at least 2 armies" is solved by $\frac{T_o}{18}$, where T_o is the number of territories occupied by the player with at least two armies.

Both the attributes and the target attributes have a large range of values as they are floats. This means that there may be need of grouping them. The precise way of grouping them may require some testing, but initially they are grouped such that for each probability X is rounded downwards such that $(X * 100) \bmod 5 = 0$ resulting in $[0-0.05, 0.05-0.1, 0.1-0.15, \dots, 0.95-1]$.

Neural Network

The ANN will be a standard feed-forward network with a number of inputs, a single hidden layer and a number of neurodes in the output layer.

The same ANN will be used for all players. The relevant information in the past for a given opponent is then given to the ANN and the ANN is reused for another opponent.

Creating an ANN for this IG, a representation of the past is needed, since this can not be directly plugged into an ANN. The same representation of the past as in the above script is used.

This results in the following inputs for an ANN for a given opponent:

- First 6 input nodes $I_{continent}$ representing the opponent's interest in each the continents.
- Then 5 input nodes I_{color} representing the opponent's interest in each of the other players in the game (if a game only has 3 players, then only the information on the 2 opposing players are used, and then the 3 last inputs will just be zero).
- Lastly 2 input nodes I_{24} and I_{18} representing how close the opponents is to occupying 24 territories and how close the opponent is to occupying 18 territories with at least 2 armies on each.

And the output:

- 14 outputs $O_{mission.n}$ - an estimate on each of the 14 missions. n is the index number of each of the missions ("Conquer Asia and South America" would have $n = 0$, "Conquer Asia and Africa" $n = 1$ and so on).

In fact, a representation on the board can be found in the same manner as the estimates were found in the above script. An influence map is created and the values $C_{continent}$ and C_{color} is calculated. The sum of all $C_{continent}$ s is a and the sum of all C_{color} s is b .

For each continent, the input value $I_{continent}$ is calculated:

$$I_{continent} = \frac{C_{continent}}{a} \quad (3.2)$$

and for each player I_{color} :

$$I_{color} = \frac{C_{color}}{b} \quad (3.3)$$

and I_{24} and I_{18} :

$$I_{24} = \frac{|T_O|}{24} \quad (3.4)$$

$$I_{18} = \frac{|T_o|}{18} \quad (3.5)$$

where $|T_O|$ is the amount of territories occupied by the opponent and $|T_o|$ is the amount of territories occupied by the opponent, with more than 1 army on.

The number of hidden neurodes is decided as described in the analysis: let "the number of hidden neurodes be the square root of the input nodes plus output neurodes, and then add a few.". This gives us $\sqrt{13 + 14} + 3 \approx 8$ hidden neurodes.

When training the network from some training example, the output of this ANN should be normalized to 1, since the output of any given ANN would most likely not sum to 1 as required by the IG., so the output needs to be normalized:

$$O_{mission_n} = \frac{O_{mission_n}}{S_{mission_n}}$$

where $S_{mission_n}$ is the sum of all the values of $O_{mission_n}$:

$$S_{mission_n} = \sum_{n=0}^{13} O_{mission_n}$$

The error in the output is calculated and then multiplied by $S_{mission_n}$ before it can be used in the network.

When the training is done with neuroevolution and an evaluation function is used, the output is evaluated against the real mission of the opponent. This could be considered as cheating, but the performance of the ANN is only updated when the game has ended, where the mission is free for all to see anyway.

The fitness of a genome representing the weights in the ANN is calculated as follows - each time a ANN has produced an output, the fitness is increased by f , where f is the output of the ANN corresponding to the actual mission. If $O_{Asia_Africa} = 0.45$ and the mission was actually "Asia-Africa" then the fitness is increased by 0.45. When the game has finished, the fitness is divided with the number of times the ANN has been run, to get an average output "correctness". This should leave the genomes that generally produce a good estimate with a higher fitness than other genomes that does not produce good estimates. This is evaluative feedback, but this is actually one of the only modules where instructive feedback also can be used for the ANN (even when no training example is available, or all training examples has been used). This is because the correct output of the ANN is directly dictated by the actual mission of the opponent. So in fact, the actual mission could be viewed as a training example, which then could be used as supervised training for the ANN.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees. The coherent BN will be, as all other naive BNs in this section, that which has the output of the task represented as child node to all nodes representing the input.

3.5.2 How Close The Opponent Is To Winning

This section will cover the design of the IG that provides an estimation on how close each player is to winning the game.

Input: Opponent's missions, the board, RISK cards owned by the opponents.

Output: A list of probabilities, stating whether each opponent is going to win in the next round, the next round again and so on up to 5 rounds from the current.

Scripting

The following is performed for each player in the game:

When estimating how close an opponent is to winning, each mission type is dealt with separately.

Conquer set of continents: Estimating on how easily an opponent will conquer a set of continents in done by examining the opponent's attacking ability against the opponent's enemies' defending power in the set of continents. The set of all the territories in the set of continents is T_C . The set of territories in T_C that is occupied by the opponent is T_O , and the set of territories in T_C occupied by the enemies of the opponent is T_E . What is needed is the probability that the opponent will conquer all of T_E , n rounds from now, where $0 \leq n \leq 4$. Finding such a probability is done in two steps.

First step is only looking on the territories in T_C , and not on the territories outside T_C . The probability P_I of the opponent conquering T_E only using armies available in T_C is done as follows

$$P_{I0} = \Phi(\alpha(T_O) - |T_E|, \alpha(T_E)) \quad (3.6)$$

where $\Phi(X, Y)$ is the probability of the opponent winning the attack with X armies on a territory with Y defending armies. This can be found as written in Section 3.4.2 on page 91. $\alpha(T)$ gives the number of armies in the set of territories T . P_I calculates the estimated probability of the opponent winning against all the armies in T_E , accounting that the opponent will place 1 army on each territory in T_E .

Second step is then finding how many armies outside T_C can help the opponent in conquering T_E . This is found by estimating how many armies it will cost for an amount of armies in a given territory to reach the continents. For this, the set of border territories T_B is needed. A border territory is a territory in a continent that has a neighbor that belongs to a different continent. "Greenland" and "Alaska" are examples of border territories. If any of the territories in T_B is occupied by the opponent, then they are removed - these are of no interest, since the opponent can not make an attack path through a territory he occupies himself. T_N is the set of territories occupied by the opponent on the board not in T_C . For each territory in T_N that has more than 1 army, the cheapest path (measured in loss of armies) to a territory in T_B is found and the amount of remaining armies is calculated. This amount is accumulated for all best paths from all territories in T_N to a territory in T_B . This accumulated amount of armies α_{acc} , is the maximum amount of armies the opponent could muster when conquering T_E . An algorithm for finding the amount of accumulated armies can be seen in Figure C.5 on page 164 in the Appendix.

After finding the amount of armies the opponent could potentially muster, this is added in the Equation 3.6, resulting in the following:

$$P_0 = \Phi(\alpha(T_O) + \alpha_{acc} - |T_E|, \alpha(T_E)) \quad (3.7)$$

This is the probability that the opponent will conquer all territories in T_E in this round ($n = 0$).

But an estimate for $n = 1 \dots 4$ is also needed. This can be done by using the amounts of reinforcements the opponent potentially would get, ρ_n , n rounds from now. ρ_n can be found by looking at the amount of territories the opponent has, $|T_O|$, and the amount of reinforcements received by occupying a whole continent γ , ρ_n will then be

$$\rho_n = \max(3, \frac{|T_O|}{3}) + \gamma \quad (3.8)$$

In RISK, one reinforcement is given for each 3 territories you occupy, and if $|T_O| < 9$ then the reinforcements received is always 3, which explains the *max* argument.

The amount of reinforcements received by RISK cards is calculated as $\theta(\delta + n)$ where δ is how many RISK cards he has (figuring in, that he will earn a RISK extra per round), and $\theta(x)$ finds the best combination of x number of unknown RISK cards (the one that gives the greatest amount of reinforcements).

A general expression for P_n can then be found:

$$P_n = \Phi(\alpha(T_O) + \alpha_{acc} + n\rho_n + \theta(\delta + n) - |T_E|, \alpha(T_E)) \quad (3.9)$$

where ρ_n is added for each round from now.

Kill X Player: This is done in an entirely different way. Finding an estimate on how close an opponent is to kill a given player P , is in fact a matter of finding how powerful the opponent is compared to player P . But in a situation where P is in trouble, the other players are likely to help P , because they of course are not interested in the opponent fulfilling his mission.

In addition to finding the strength ratio between the opponent and P , it is also important to estimate how easily the opponent could conquer all of P 's territories. But this is matter of finding all potential attack plans for each of P territories and that seems intuitively too extensive.

So the only estimate made here, is the probability that the opponent will conquer all of P 's territories, but factoring in, that the other players possible will assist P .

First the probability that the opponent will conquer all of P 's territories

$$P_0 = \Phi(\alpha(T_O), \alpha(T_P)) \quad (3.10)$$

where $\alpha(T_O)$ is the amount of armies in the opponent's territories and $\alpha(T_P)$ is the amount of armies in player P 's territories. However this does not factor in the help from the other players.

But it is unlikely that all of the other players will assist P with all their armies, since they still will try to keep their own continents and territories. So the neighboring territories to P 's territories are only factored in. T_{NE} is the set of territories which is neighbor to a territory in T_P and not occupied by the opponent. But the other players can not assist P when it is not their turn, so a constant of 0.2 is multiplied to the aid from the territories in T_{NE} , which is a very hard coded estimate, but it seems to make sense, and can be tweaked after further tests. This is then added to the above equation

$$P_0 = \Phi(\alpha(T_O), \alpha(T_P) + 0.2\alpha(T_{NE})) \quad (3.11)$$

Again, a measure for n rounds from now is needed, and the estimated amount of reinforcements received by the opponent is factored in

$$P_n = \Phi(\alpha(T_O) + n\rho_n, \alpha(T_P) + 0.2\alpha(T_{NE})) \quad (3.12)$$

Occupy a set of territories: This is a matter first finding the amount of territories the opponent needs to conquer s and then estimating how likely he is to conquer that amount. The size s is easily calculated as either $s = 24 - |T_O|$ or $s = 18 - |T_O|$, where T_O is the set of territories occupied by the opponent, depending on the mission.

Finding how likely the opponent will conquer s number of territories is first a matter of estimating on which territories he will attack. This is done by simulating an attack phase done by the opponent. It is assumed that the opponent always will attack the territories where he is most likely to win - the territories with the highest Φ values. This can be done with the algorithm depicted in Figure C.6 on page 165 in the Appendix.

Since the algorithm always make the opponent attack where the probability of success is highest, it might run into a local maximum, where the probability seems best, but in fact attacking an other neighbor is in the long run a better choice. This is a risk we need to take, since finding the perfect series of attacks is too extensive.

Now, finding the joint probability for n rounds from now, is a matter of distributing the potential reinforcements received by the opponent n rounds from now to territory needing it. But those reinforcements needs to be placed before the algorithm in Figure C.6 has begun. Estimating where the reinforcements are best placed, is a matter of finding the optimal defend phase, which is too extensive. Placing all reinforcements in the first territory found by the above algorithm will be just as good (assuming the algorithm avoids the local minima and trapping itself).

Decision Trees

To build a decision tree for this task, the board needs to be represented in some other way to suit the task. To represent the board with all its 42 territories with a variable number of armies on each will simply be too large an input set to build a usable tree. Instead we represent the board as being 6 continents with a power distribution on each as input attributes. This distribution is considering the opponent X's armies in relation to all his opponents' in that continent: $\text{Percentage of armies} = \frac{X's \text{ Armies}}{\text{All Armies In The Continent}}$, which will be in the ranges [0-0.05, 0.05-0.1, 0.1-0.15, ..., 0.95-1]. The opponent's mission is an integer between 1 and 14 (the number of missions) also in ranges. The number of RISK cards is an integer between 0 and 5 (the maximum number of RISK cards). The target attribute is an estimation on how close each player is to winning in the following 5 rounds. The estimate will again be rounded down such that $X \bmod 5 = 0$, where X is the probability in question. There are 5 of these ranges in the target attribute - one for each of the 5 rounds. A tree is built for each player.

But there is a flaw in this way of building the tree. It will most likely not estimate how close opponents' with the mission "Destroy player P" are to completing their missions very well. The reason for this is, that the board representation used here does not solely compare the opponents' armies vs. X's armies. A second set of trees is therefore built for this mission type. The board will here be represented in the relation between X's armies and P's armies: $\text{Relation} = \frac{X's \text{ Armies}}{P's \text{ Armies}}$. For the "24 territories" and "18 territories" missions, the first type of tree is descriptive enough to handle those mission types.

Whatever mission given as the most likely mission, states which type of tree is selected for the estimation of how close each player is to winning.

Neural Networks

Making an ANN for this IG, the board is a vital input, since it gives the most information on the current situation. There are two different scenarios when representing the board: 1) Each territory is represented as its own input node and the ANN will be trained just fine, 2) the 42 input nodes to the network is too many and the ANN will take forever to learn, and some other representation of the board is needed, with a minimum amount of loss of information. But this will only become apparent when the ANN is tested, and both scenarios will be designed.

The Board as Direct Input

Using the board as direct input to an ANN, 42 pairs of nodes are needed. Each pair represents a territory. The first node represents the amount of armies in the territories. The amount of armies are subdivided into ranges: [0, 1, 2, 3, 4, 5, 6-7, 8-10, 11-14, 15-19, 20-25, ..., >100] which correspond to [0, 0.05, 0.1, 0.15, 0.2, ..., 1].

The second node will be a representation on which player owns the territory. This node has 6 values between 0 and 1 - one for each player in the game.

This will result in a pretty accurate representation of the board using 48 input nodes.

Representing the Board with Loss

The representation of the board should somehow indicate how well the opponent is going to conquer the territories in the continents which is in his mission. But with a minimum number of input nodes.

This can be achieved by having an input for how powerful the opponent is in each of the continents compared to his enemies. There will of course be some information loss here, since the information on how easily the opponent is going to capture those territories is not available, but how powerful he is, is still a good measure of this.

The first 6 input nodes $I_{continent}$ is calculated as done in Equation 3.6 on page 98 in the above script:

$$I_c = \Phi(\alpha(T_O) - |T_E|, \alpha(T_E))$$

where $\Phi(X, Y)$ is the probability of the opponent winning the attack with X armies on a territory with Y defending armies. $\alpha(T)$ gives the number of armies in the set territories T . T_O is the set of territories occupied by the opponent in continent c . T_E is the set of territories occupied by the enemies to the opponent in that continent.

The Other Input

Now that two different board representations have been explored, the other input nodes now needs to be discussed. These are different depending on what mission the opponent is estimated on having

The missions are divided into three types and for each, a different ANN is created:

Conquer set of continents: The representation of the board is an input depending on what type of representation are found to be the best.

Then an input node for the estimated mission is also needed. This will have 6 values between 0 and 1 for each of the mission types in this category. An input node for the amount of RISK cards is also needed. This will have 6 values between 0 and 1, since an opponent can have minimum 0 cards and a maximum of 5 cards.

Kill player: In this type of missions, some information on how powerful the player P in the opponent's missions is. This will result in a ANN like the one above for "set of continents" missions, but if the lossy board representation is selected, then 6 more input nodes are needed, which give information on how powerful P - found in using Equation 3.13 for P instead of the opponent. This will give an ANN which has information on how powerful both P and the opponent is in each continent, and it should then be able to produce a useful output.

Occupy amount of territories: The ANN for this type of missions is the same as the “set of continents” missions, but with one more input node. This node gives information on how close the opponent is to fulfilling his mission in terms of the amount of territories he occupies. The value of this is calculated as in Equation 3.4 and 3.5 on page 96. The other input nodes should then provide information on how easily he will conquer the remaining territories. The input node for the estimated mission will in this case be binary valued: 0 for “24 territories” and 1 for “18 territories...”.

There is 5 output neurodes for all of the 3 ANNs. One for each of the 5 rounds from now, the estimate should consider.

Training such a network is trivial when it is done with training examples. And training with neuroevolution has been argued in the analysis to be unusable for this IG.

Bayesian Networks

Since the task is finding the probability for the opponent winning within the next five rounds, a time sliced Bayesian network is used for this task. We obtain the structure by having a Bayesian network for how close the opponent is to occupying each continent, one for how close he is to killing another player, how close he is to occupying 24 territories, and how close he is to occupying 18 territories with 2 armies at each. These networks are connected to a “Close to Win” node which also takes the opponent’s estimated mission as input.

Close to Continent

How we model how close an opponent X is to occupying specific continent is seen in Figure 3.2. Instead of representing the board directly in the this model, we consider only the continents in question and its nearest neighbors. These variables are have the number of armies, represented as integers, as states. As the figure shows, both X and the X’s opponents’ reinforcements affect how many armies there are possibly placed in both the continent and in the vicinity of the continent. The “Armies_left” nodes are used to gather X’s armies and X’s opponent’s armies into one node and thereby divorce the “Number of armies”-nodes within a continent from those outside. The node states how many armies X would have left after an attack. The states are “X has much more than his opponents”, “X has a little more than his opponents”, “Almost tie battle”, and “Opponents have more than X”.

In the next time slice, “ArmiesXContinent” and “ArmiesXOppCont” and “ArmiesXClose” and “ArmiesXOppClose” are influenced by the same nodes from the previous time slice. This is because whether or not the armies are spend in the previous time slice influences the number of armies available the following round.

Close to Kill Player

For each player X can destroy, a model looking as in Figure 3.3 is used. The armies on the board are split in two: X’s armies (“Armies_X_Board”) and the remaining armies (“Armies_Opp_Board”). The “Armies_Opp_Board” can be used because of an assumption that the remaining players will either prevent X from

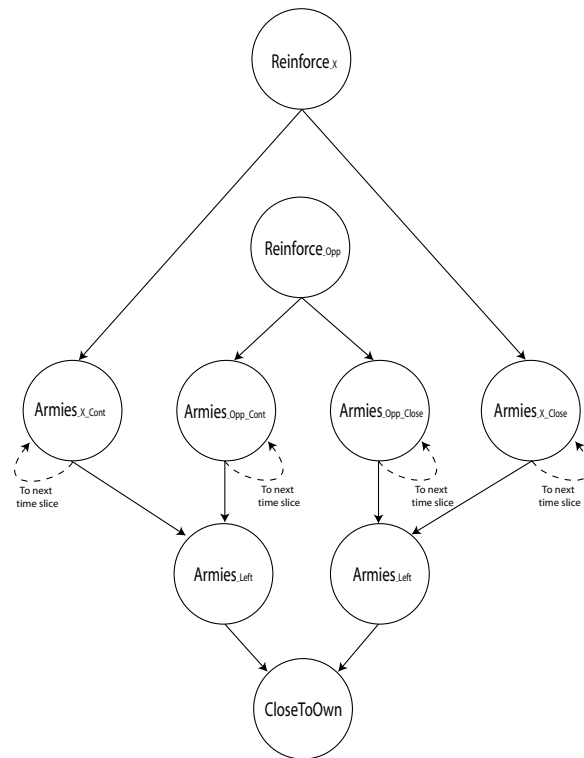


Figure 3.2: A BN for the task: “How Close an Opponent Is to Owning a Continent”. The BN uses armies within the continent and armies close to the continent to decide how close a player is to occupying an entire continent. Since this calculation is done for 5 rounds, this BN is part of a time slice model. Only the armies in and close to a continent has influence on the next time slice.

killing the player if they believe this is X’s mission, or they will kill the player themselves if the player has little power on the board. Therefore they are part of the node with the player. Evidence is entered into this node by taking all of the players armies and summing them with one fifth of the remaining of X’s armies. The tenth is because you cannot rely on all other players to prevent X from killing a player. It is merely used to take into consideration that X might have to go through territories occupied by other players. The nodes sent to the next time slices are the ones containing the number of armies X and the remaining players have on the board. These naturally influences how the board looks the following round.

Close to Occupy 24 or 18 Territories

We did not find is possible to model this task in a BN. Therefore we will represent these missions with just one node stating how close X is to complete it. This information is estimated as done in the script.

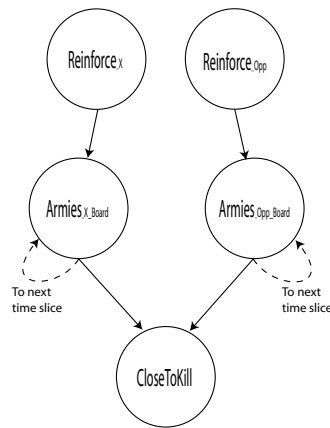


Figure 3.3: A BN for deciding how close a player is to destroying another player. The scheme is similar of that concerning how close a player is to occupying a continent. The difference being that in this case, armies on the entire board is used. Since this calculation is done for 5 rounds, this BN is part of a time slice model. Only the friendly and hostile armies have influence on the next time slice.

Estimating the mission

All the nodes estimating how close a player is to complete a given task (conquering a continent, killing a player, etc.) are parents to a “CloseToWin” node. This is seen in Figure 3.4. The dashed nodes are those which denotes each of the networks for how close the player is to performing a given task. There are one of these for each continent there is on the board and for each player in the game. Also there is a node for each of the missions “24” and “18”. There is one BN of this type for each player in the game. The mission node which is also parent to the “CloseToWin” node is instantiated to the mission the player in question is estimated to have.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

3.5.3 The Opponents’ Next Moves

This section will cover the design of the IG that provides an estimation on each opponents’ next moves.

Input: The output from the other IGs, the board, opponents’ RISK cards.

Output: A list for each player of actions made by that player.

Scripting

As written in the Analysis, estimating an opponent’s next moves is done by running the AI’s own IG, MP and RP on the information available to the opponent. This will be done by creating new instances called IG_O , MP_O and RP_O

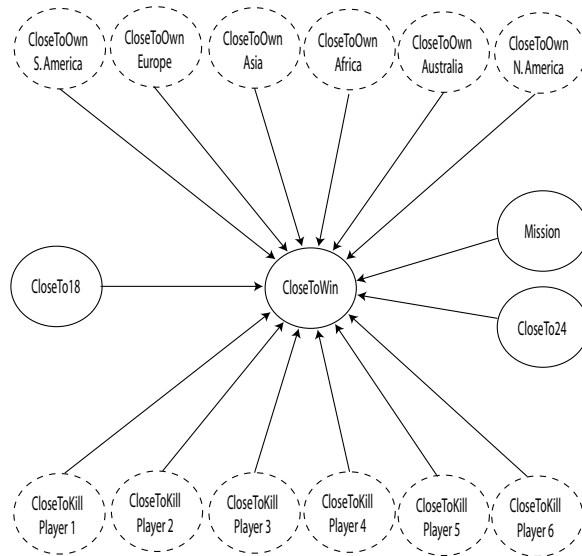


Figure 3.4: The BN used to decide on how close a player is to winning. The dashed nodes are each parts of other BNs which have been removed for clarity.

which does the same as IG , MP and RP but viewing everything from the opponent O 's point of view.

It is actually not necessary to run the IG_O , the information available in the IG can just be used. The module “How Close an Opponent is to Winning” would just return an estimate on how close it thinks the AI itself is to winning. The module “Opponents’ Next Moves” should just return an empty list of actions as discussed in the Analysis (the module would create a new instance of the IG , which in return also creates a new instance and so on). So the information already available in the other IG s is sent to the MP_O . The MP_O calculates a goal distribution and sends it to RP_O . The RP_O should then perform some actions, but instead, it keeps the actions in a list, which is the output of this IG . Figure 3.5 depicts this procedure.

3.5.4 How Close The Opponent Is To Owning a Continent

This section will cover the design of the IG that provides an estimation on how close each player is to owning each continent.

Input: The board, RISK cards owned by the opponents.

Output: A list of probabilities, stating whether each opponent is going to conquer each continent in the next round, the next round again and so on.

Scripting

Finding how close a given opponent is to owning a given continent is quite similar to the script for estimating “How close an opponent is to winning”:

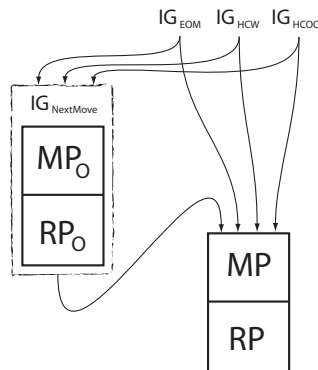


Figure 3.5: How the Information Giver “Opponent’s Next Move” creates a new instance of the Master Prioritizer and Round Planner to estimate an opponent’s next move.

Equation 3.9 on page 99 is used for each of the 6 continents in the game.

Decision Trees

The input/output format for this decision tree is the same as with that of “How close the opponents’ are to winning”. The difference being that there are less input attributes.

Neural Networks

The ANN for this IG is very similar to the ANN for the missions “set of continents” in the IG “How Close the Opponent is to winning”.

Input: This could even be the accurate representation of the board or a lossy one. The lossy one would have an input node stating which continent is in question, and 6 input nodes stating how powerful the opponent is in each of the continents - it is not only the continent in question the ANN need information on, because the opponent could use armies outside that continent also.

Output: A single output being the estimate on how close the opponent is of occupying the given continent.

This same ANN can then be queried for each opponent and each continent in the game. This ANN is only trained through training examples, and the evaluation in neuroevolution is not discussed.

Bayesian Networks

The BN used in “How close is an opponent to winning” to determine how close an opponent is to occupying a continent is reused here.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

3.6 Master Prioritizer

In this section the MP is designed. The input for the prioritizer are:

Input: The estimates received from the information givers.

Output: A prioritized goal distribution.

3.6.1 Goal Weighting

Scripting

This section only contains a summary of the design of the MP. A more detailed description on all the considerations and decisions made, can be found in Appendix C.6.

At first, two terms are defined when talking about which type of play the AI employ. These are *mission-specific* and *preventive*. The AI plays mission-specific when none of the opponents are close to winning the game. If the AI estimates an opponent of being close to winning, then the AI starts playing preventive, and giving high priorities to goals such as “Obstruct and Defend Obstruction” on the opponent’s continents.

Prioritizing the goals is done by distributing 100 *importance points* to the goals. The points are at first distributed evenly, resulting in each goal getting $\frac{100}{\text{NumberOfGoals}}$ points.

Each goal then makes a *claim* on how many importance points it needs. Goals with the highest claim are considered most important. The claims made are based on whether the AI plays mission-specific or preventive. Goals that does not have any claims release their importance points to a pool. Points are then given to goals from this pools, starting with the most important goal first. If the pool is emptied and some goals did not get their claims met, then the least important goals release importance points and decrease their claims. This results in a list of prioritized goals which can be passed to the RP.

An important point to make, is the fact that a big part of the MP is giving the AI a way of concealing the mission it is trying to fulfill. This could have been a goal given to the RP, but it is instead done by producing a different importance distribution on the goals instead.

This is done by checking what the opponents’ would think the AI’s mission is. This is done by running the IG “Estimating an Opponent’s mission” on itself. If the estimate is correct and above 75% certainty the AI should do some random actions to mislead the other players. However, the random moves should not include leaving mission-specific continents. Rather it should give a distribution on the goals with an even distribution on “Attack Player” which will leave it up to the RP whom to attack

Neural Network

An ANN for producing a goal distribution would have the information given by each IG. So the information from each IG needs to be represented in the input to the network.

Estimated Mission: This is 5 pairs of input nodes. Each pair of nodes is first a value stating the mission type a given opponent is estimated of having and the second is a value for how sure the IG is on that mission. There are 5 of these pairs - one for each player in the game (not including one self), resulting in 10 nodes.

How Close to Winning: This is 6 input nodes. One for each estimate on how close a given player is on winning the game (this includes one self, which is useful information when the AI is close to winning the game).

How Close to Occupying a Continent: This is 36 input nodes. One for each estimate on how close a given player is on occupying each of the 6 continents (also including one self for the same reason as above).

Next Move: This is a bit more tricky. A good representation of the next move of a player is needed. It will take a lot of input nodes to represent all players' next moves. And as with the board representation for ANNs for the IGs, it is not certain whether so many input nodes will make the ANN unable to learn anything valuable - the number of training examples needed is too great. But it should turn out that a great amount of input nodes is not a problem, then a design of such an ANN should be available. If it on the other hand, turns out that the ANNs are unable to train, then the information from the IG "Next Move" will be omitted in the ANN for the MP.

But even when representing an opponent's next move as an input to an ANN, there still needs to be some abstraction over the actual input, since the data structure in the opponent's next move can not directly be used as input.

One way of making an abstract representation of an opponent's next move is by having 42 binary input nodes. Each node stating whether the opponent is going to make an attack in the corresponding territory or not. This will give the ANN information on where an opponent's interest lie. This will of course mean that 5 times 42 input nodes is needed - a set of 42 nodes for each opponent player.

The output of the ANN for the MP is a set of node for each goal, since most of the goals involve each of the opponents or each of the continents. This result is the following output for each goal:

Conquer and defend continent: 6 neurodes, one for each continent.

Conquer continent: 6 neurodes, one for each continent.

Defend Continent: 6 neurodes, one for each continent.

Obstruct continent: 6 neurodes, one for each continent.

Obstruct continent and defend obstruction: 6 neurodes, one for each continent.

Attack player: 5 neurodes, one for each opponent player.

18 territories with at least 2 armies: 1 neurode for this goal.

24 territories: 1 neurode for this goal.

This result in 37 output neurodes. The sum of the values of the output should be 1, which mean that the output from the ANN should be normalized like the output from the IG “Estimating Opponents Mission”.

As argued in the Analysis, when using neuroevolution, the goal distribution can not be evaluated locally, but needs to be evaluated globally at the end of a game. All the genomes used in the MP in a winning game has their fitness increased. Over the course a series RISK games, where the same genomes are used again and again their fitness is increased, and the best genomes can be selected for the next generation.

3.7 Round Planner

This section will cover the design of each module in the RP.

3.7.1 Make Attack Plan

The only AI technique suitable for the “Make attack plan” module was found to be scripting. This will therefore be the only technique described throughout this section. The inputs and outputs of the module are as follows:

Input: Occupied territory, target territory, MP goals, board.

Output: Attack plan (list of territories).

Scripting

Depending on whether the occupied territory and the target territory is on the same continent it must be determined when to use the A* algorithm or the HamPath algorithm when computing the path. If the two territories are in different continents then A* is chosen, because using HamPath would not make sense. If the two territories are in the same continent then both algorithms can be of use. When attacking internally in a continent it can either be for the purpose of simply conquering the other territory or it can be because one wants to conquer the entire continent. To cover both cases the module should be called twice, once where it makes use of the A* algorithm and once where it uses HamPath.

Most of the work performed by the module is taken care of by A* and HamPath. In order to make the attack plan with MP goals taken into account, A* must be modified to do so. In the example while making a path where it can

be chosen to move through either an enemy territory whose owner's death is a priority in the MP goals and one where the owner is not, A^* should choose the first territory if they have around the same amount of armies. This behavior can be produced by giving territories owned by players who should be killed according to the MP goals a slightly lower step-cost in the A^* algorithm.

The path consisting of the occupied territory and a series of enemy territories to attack, ending with the target territory, is then returned by either of the path finding algorithms. This will then be output by the module.

3.7.2 Calculate Attack Plan Cost

The only AI technique found to fit this module was scripting so this will be the technique used for the design of this module. The inputs and outputs are:

Input: Attack plan, board, MP goals.

Output: Minimum army cost, estimated army cost.

Scripting

The minimum army cost for following an attack plan can be calculated quite simply. It is the cost in armies if all territories in the attack plan are conquered without losing a single army. The only army that will be lost when conquering a territory will be the one that are required to be left in the source territory when moving into the conquered territory. The minimum army cost is then the same as the number of times it will be needed to move from one territory to an other. If the goal "Conquer 18 territories with 2 armies on each" has high priority this number should be doubled:

$$Cost_{min} = |T_{plan}| \times A_{min} \quad (3.13)$$

where T_{plan} is the set of territories to conquer in the attack plan and A_{min} is the minimum number of armies to leave behind. A_{min} is 2 if the "Conquer 18 with 2 on each" has high priority, 1 otherwise.

The estimated army cost is the number of armies it would cost to follow the attack plan by using the most probable outcome of each attack. This can be calculated using the estimated army cost function mentioned in Section 3.4.1. By using this function for each territory to conquer and add the results, an estimated cost for the attack plan is found. If the MP goal "Conquer 18 continents with 2 armies on each" has high priority, then the cost for leaving one extra army behind should be added to the cost. The estimated cost is calculated this way

$$Cost_{estimated} = \sum_{t \in T_{plan}} \Gamma(A_t) + Cost_{min} \times A_{min} \quad (3.14)$$

where t is a territory in the set T_{plan} of territories to conquer, Γ is the estimated cost function, A_t is the number of armies in t and A_{min} is the minimum army cost.

3.7.3 Prioritize Attack Plan

This module calculates a priority for a given attack plan based on the MP goals it helps satisfy.

Input: Attack plan, MP goals, board.

Output: Priority.

Scripting

The priority of an attack plan should be related to the priority of the MP goal it helps satisfy. The MP goals can be divided into four categories:

1. Conquer continent & obstruct continent
2. Kill player
3. Conquer territories
4. Defend continent

To determine if an attack plan helps satisfying a goal of the “Conquer continents” type, it can be checked how many territories in the plan belongs to a continent included in the goal. The more territories covered by the attack plan, the higher priority the attack plan should have according to the goal. At most the plan can have a priority equal to that of the goal. If the plan conquers the territories needed to occupy all continents belonging to the goal, the plan’s priority should be equal to that of the plan. This gives this formula to calculate an attack plan’s priority:

$$P_{plan,continent} = \frac{|T_{goal} \cap T_{plan}|}{|T_{goal}|} \times P_{goal} \quad (3.15)$$

$P_{plan,continent}$ is the priority of the attack plan for this type of MP goal, T_{goal} is the set of unoccupied territories covered by the MP goal, T_{plan} is set of territories to conquer in the attack plan, and P_{goal} is the priority of the MP goal. If the plan contains all remaining territories covered by the goal, its priority will be equal to that of the goal.

Giving priority to an attack plan on whether it helps satisfy an “Obstruct continent” MP goal can be done exactly the same way.

If the attack plan helps to satisfy a goal of the type “Kill player”, the plan’s priority should take two things into consideration: How many of the target player’s armies will be killed and how many of the target player’s territories are conquered. The first thing because the less the amount of armies owned by the target player the closer he is to getting killed. The second because the less territories the target player owns, the less reinforcements he will receive each turn and the easier he will be to kill.

$$P_{plan,kill} = \frac{A_{target} - A_{plan}}{A_{target}} \times \frac{|T_{target} \cap T_{plan}|}{|T_{target}|} \times P_{goal} \quad (3.16)$$

$P_{plan,kill}$ is the priority of the attack plan for this type of MP goal, T_{target} is the set of target player's territories, T_{plan} is set of territories to conquer in the attack plan, A_{target} is the total number of armies belonging to the target player, A_{plan} is the number of armies belonging to the target player in territories in the plan, and P_{goal} is the priority of the MP goal. If the plan contains all of the target player's territories and following the plan will kill all his armies, its priority will be equal to that of the goal.

The "Conquer territories" class of MP goals covers two MP goals:

1. Conquer 24 territories
2. Conquer 18 territories with 2 armies on each

It is difficult to prioritize an attack plan with respect to the first goal. In fact all attack plans will help satisfy this goal in some sense since they all will conquer territories. Attack plans containing many territories should get a higher priority because following them will bring the computer player closer to the MP goal, like this:

$$P_{plan,24territories} = \frac{|T \cap T_{plan}|}{|T|} \times P_{goal} \quad (3.17)$$

where P_{plan} is the priority of the attack plan for this type of MP goal, T is the set of enemy territories on the board, T_{plan} is set of territories to conquer in the attack plan, and P_{goal} is the priority of the MP goal.

Not all attack plans will help satisfying the second goal. The only plans satisfying the goal are those which leave two armies behind when moving armies into the conquered territory. These attack plans can be recognized by having a minimum cost of two pr territory to be conquered. Plans conquering many territories with two armies on each should receive a higher priority compared to plan conquering a few or none with two on each. The priority can again only be as high as the MP goal it helps satisfy:

$$P_{plan,18territories} = \frac{|T \cap T_{plan}|}{|T|} \times Cost_{min} \times (A_{min} - 1) \times P_{goal} \quad (3.18)$$

P_{plan} is the priority of the attack plan for this type of MP goal, T is the set of enemy territories on the board, T_{plan} is set of territories to conquer in the attack plan, $Cost_{min}$ is the minimum cost of following the plan, A_{min} is the minimum number of armies to leave behind when moving armies to a conquered territory, and P_{goal} is the priority of the MP goal. If the plan contains all enemy territories on the board and following the plan will leave two armies on each territory when conquered, its priority will be equal to that of the MP goal.

An attack plan can either help satisfy or dissatisfy MP goals of the type "Defend continent". An plan will help satisfy the goal if it ends in a territory in the continent covered by the goal because then the plan would essentially have transferred armies from its starting territory to a territory needing defense. In that case the plan's priority should receive the priority of the MP goal. If the

starting territory is located in the goal's continent, following the plan will mean that armies are taken away from the territory needing defense. In this case the plan's priority should be penalized. For example by subtracting the goal's priority from the plan's priority. The priority will then possibly get negative.

Now that a way has been found to prioritize an attack plan according to each kind of MP goal all that is left is to find a way to calculate the total priority for the plan. Since the priority received by each MP goal can at most be as high as the priority of the goal itself, and since the sum of all MP goals' priorities is 1, the priority of an attack plan can simply be computed as the sum of the priority values computed for each of the MP goals.

$$P_{plan} = \max(\sum_{g \in G} P_{plan,g}, 0) \quad (3.19)$$

For each goal g in the set of goals G , the priority $P_{plan,g}$ of the attack plan according to g is found. The maximal priority of 1 is achieved if an attack plan satisfies all MP goals. If the priority is negative then it is set to the minimum priority value of 0.

Decision Trees

The attack plan is input by having an attribute for each territory on the board with the states "True/False" indicating whether a territory is part of any opponent's attack plan or not. In the sense of the decision decided by this decision tree, no valuable information is lost. The reason for this is that the only thing interesting is what territories that are attacked. An input is used for each goal where the states are integers ranging from 0 to 100. The board is input as the representation used earlier where the power distribution on each continent is found. The target attribute is the priority, which is an integer between 0 and 100. The decision tree should learn to rate attack plans from the given input.

Neural Networks

An ANN for prioritizing attack plans needs to somehow find an abstraction of an attack plan before using it as input. This abstraction could be similar to the representation of "opponent's next move" in the ANN for the MP: 42 binary input nodes stating whether the corresponding territory is in the attack plan or not. This should be a sufficient representation of the given attack plan, when it is a matter of prioritizing it.

The board can be represented as discussed in the ANN section of the IG "How Close The Opponent Is To Winning": either by having the board as direct input by 42 pair of input nodes, or a more lossy representation with 6 input nodes representing how powerful the AI is in each of the 6 continents.

Each of the 37 values in the goal distribution need also be input nodes in the ANN.

This results in either 127 or 85 input nodes, depending on which board representation is chosen.

The ANN only has a single output neurode stating the priority of the attack plan by a value between 0 and 1.

When training with neuroevolution, the evaluation of a goal distribution is done in the same manner as for the ANN for the MP.

Bayesian Networks

The Bayesian network for this task can be seen in Figure 3.6. The figure is not complete, but merely an example to display the size and complexity of the BN. The nodes denoted with an “AP” are the territory nodes for attack plans. The nodes “Satisfies *GoalName*”, are used to separate all their parents from each other and thereby avoiding modeling an naive BN. These nodes have nodes of relevance for each specific goal as parents. For a “Satisfies Conquer & Defend Continent” goal, this is AP territories, and the number of friendly and opponent armies occupying the continent as well as territories involved in the attack plan.

This approach is repeated for every different type of mission. The “AP: Starting Continent” node is used to specify which continent the attack plan emerges from. This is because attacking from a continent means *not* defending it. So this needs to be known to prevent the BN from giving an attack plan high priority without considering whether or not defensive goals are jeopardized.

The “Prioritize” node is the one that will be observed when using the BN for this task. It has all the “Satisfies *GoalName*” as parents as well as the goals’ priorities given from the MP.

The model in the figure is, as mentioned, incomplete. The complete model would naturally contain a node for each territory in the attack plan (which is theoretically unlimited). The actual number of nodes needed is not known at this point since it is depending on which plans the RP creates in an actual gaming situation, but the absolute maximum is 42 meaning that we need to have 42 nodes which has two states “Yes” and “No” denoting whether they are in an attack plan or not. The actual number of nodes used for “Number of territories in AP occ by player X” is the number of players possible in a game, which is 6. For the missions “24”-and “18 territories” a node is used that directly tells how many territories there would be obtained if the plan is carried out. This node and a node specifying how many territories are needed to fulfill the mission are parents to the “Satisfies 18” and “Satisfies 24” nodes. Specifying the CPTs on the nodes is done based on training data. However, the “Satisfies *GoalName*” nodes are not directly available from the training data. This must be learning by a Bayesian learning algorithm.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

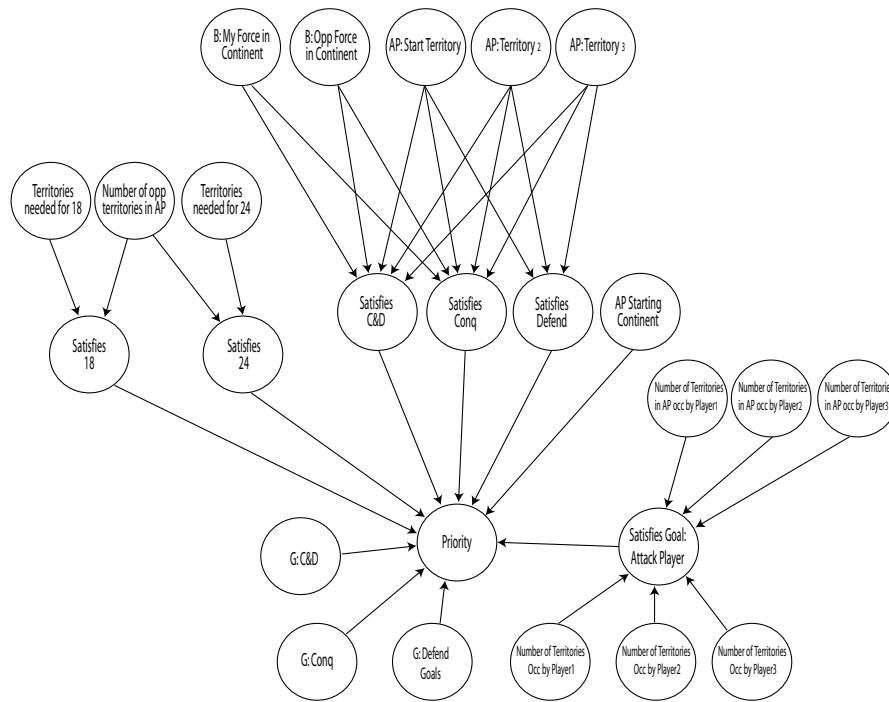


Figure 3.6: The BN used to assign a priority to an attack plan.

3.7.4 Discard Attack Plan

Given the priority for an attack plan and cost, this module discards plans which have not a high priority and cannot be completed this round.

Input: Attack plan priority, minimum army cost, estimated army cost, number of possible reinforcements without cashing Risk cards, Risk cards.

Output: Yes/No.

Scripting

Plans which have an estimated army cost lower than the maximum number of reinforcements received this turn should *not* be discarded. But even if a cost is greater than the reinforcements in this turn should it not be discarded. If the plan's minimum cost can be payed this turn then there is still a chance of completing it, although it will require some luck. So if the estimated cost is close to how much can be spent this round, like 10% higher, the plan should be kept.

If a plan has a very high priority (e.g. 90%) and its estimated cost is a great deal higher than the reinforcements in this turn, it should not be discarded right away, but be carried out over more rounds. By estimating how many reinforcements the computer player will receive the next rounds, an estimate can be found on how many rounds it will take to carry out the attack plan. If the number of rounds is greater than 3, then spending armies on this plan for

so many rounds will prevent all other attack plans from being completed. The number of future reinforcements can then be calculated by

$$\rho(C, n) = R \times n + \theta(C, n) \quad (3.20)$$

where the R is the number of reinforcements received this round, n is the number of rounds for which the number of future reinforcements should be estimated. $\theta(C, n)$ finds the amount of armies received by combining the Risk cards on hand C with the Risk cards received during the next n rounds. It is assumed one Risk card is received each round and the board configuration does not change the next n rounds.

A script for this module could for example look like Figure 3.7. The formula in Equation 3.20 is used in the script as the function *CalcFutureReinf*.

```

0  if ESTIMATED_ARMY_COST <= NUMBER_OF_REINFORCEMENTS then
1      return NO
2  else if MINIMUM_ARMY_COST <= NUMBER_OF_REINFORCEMENTS and
3      ESTIMATED_ARMY_COST * 110% <= NUMBER_OF_REINFORCEMENTS then
4      return NO
5  else if PRIORITY >= 90% and
6      ESTIMATED_ARMY_COST <= CalcFutureReinf(MyRiskCards, 3) then
7      return NO
8  else
9      return YES;

```

Figure 3.7: Algorithm for determining whether an attack plan should be discarded.

Decision Trees

The input is used as follows: the priority is an integer between 1 and 100. Minimum army cost, estimated army cost needs to be represented in intervals since they are theoretically infinite, which makes them unsuitable for decision trees. They are represented in the following intervals [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11-15, 16-20, 21-30, 31-40, 41-60, 61-80, 81-100, >100]. The maximum number of reinforcements is an integer between 1 and 38. The target attribute has the states “Yes/No”.

Neural Networks

The input for this ANN is the following:

Priority: The priority is already a value between 0 and 1 and can directly be represented in a single input node.

Minimum Army Cost: This value can theoretically be infinitely, or at least very large. When representing this as a value between 0 and 1 it needs to be subdivided into ranges, where the lower values are finely divided and

the larger values (e.g. +50) are more roughly divided. This is because there will most likely be many examples of a minimum army cost where the value is low, and not so many large, so dividing the larger values roughly will have less loss over time. The ranges are: [0, 1, 2, 3, 4, 5, 6-7, 8-10, 11-14, 15-19, 20-25, ..., >100] which correspond to [0, 0.05, 0.1, 0.15, 0.2, ..., 1].

Estimated Army Cost: Done in the same manner as above.

Max Number of Reinforcements: This value has a maximum of 38 (24 if the AI occupy each continent plus 14 if he occupy all territories). So the input will be: $\frac{NumReinforcements}{38}$.

Number of RISK cards: This has a maximum value of 5, so the input value will be $\frac{NumberOfRISKCards}{5}$.

This results in 5 input nodes. The output is a simple binary 0/1 output.

The evaluation for neuroevolution is done as with the ANN for the MP.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

3.7.5 Calculate Defense Cost

This module gives an estimate on how many extra armies are needed to defend a given territory from being conquered by an enemy.

Input: Territory, board.

Output: Estimated army cost.

Scripting

Looking locally at the board and the neighboring territories surrounding the territory in question, the defense cost is quite simple to calculate. Assuming that the enemies will try to conquer the territory at all costs, the cost of defending the territory is the enemy's attack force minus the amount of armies the enemy will lose, which is

$$Cost = \sum_{t \in T_{enemy}} A_t - |T_{enemy}| - \Gamma(A_o) \quad (3.21)$$

where t is a territory in the set of enemy territories T_{enemy} , A_t is the number of armies in t , and $\Gamma(A_o)$ is the estimated cost for the enemy to conquer the occupied territory o defended by A_o armies.

To also take enemy territories farther away into account. T_{enemy} could be expanded to also include the neighbors' neighbors and so on, and then use Equation 3.21 to calculate the cost. But this would be the same as assuming that all enemy territories can attack the territory directly, which is not correct.

The enemy may lose armies on the way to the territory and may not even be able to move into attack position because one of his own territories could be blocking midways. Also, simply the presence of other armies in neighboring territories belonging to a player other than the enemy, will reduce an enemy's urge to attack. All these issues are difficult to incorporate in a formula. A better way would be to use an influence map which would not suffer from these issues.

Calculating Influence

Using an Estimated Influence Map (described in Section 2.8.8, Page 59) will be useful in this situation. The influence map can be used as a way of describing the influence a player's armies in a territory has on all the other territories on the board. In case of the Estimated Influence Map, the influence an opponent's territory has on a neighboring territory is calculated in two ways, depending on whether the territory belongs to himself or one of his opponents. In the case of the territory belonging to someone else than the opponent (e.g. the AI player), the influence is calculated by subtracting the estimated cost of conquering that territory from the number of armies in the opponent's territory minus the one that would need to be left behind. If the neighboring territory belongs to the opponent himself, the influence is the same as the number of armies in the opponent's territory minus one. This is to simulate that the other players' urge to conquer the neighboring territory decreases if he can look forward to lose it next turn, because the opponent would use his armies to revenge the attack. In other words, the influence an opponent's territory has on a territory belonging to someone else than the opponent can be calculated as

$$I_{o,t} = \max(A_o - 1 - \Gamma(A_t), 0) \quad (3.22)$$

where $I_{o,t}$ is the influence the opponent territory o has on the territory t belonging to someone else than him, A_o is the number of armies in o , A_t is the number of armies in t , and $\Gamma(A_t)$ is the estimated cost of conquering t . If the cost is greater than there is armies in the opponent's territory, the opponent's territory's influence on the other territory is zero.

The influence an opponent's territory has on a territory occupied by himself, is

$$I_{o,p} = A_o - 1 \quad (3.23)$$

where $I_{o,p}$ is the influence o has on the territory p occupied by himself and A_o is the number of armies in o .

The influence of armies in the opponent's territory should be able to spread through neighbor territories occupied by someone else behind and affect territories behind them, because the opponent could perform an attack through a series of territories. This is illustrated on Figure 3.8 where $I_{A,C}$ is calculated using the value of $I_{A,B}$ as the number of armies C can be attacked with.

If there was yet another territory belonging to someone else behind C , the influence on that territory would be calculated using the $I_{A,C}$ and so on, until the influence is zero. As mentioned before, the influence cannot spread

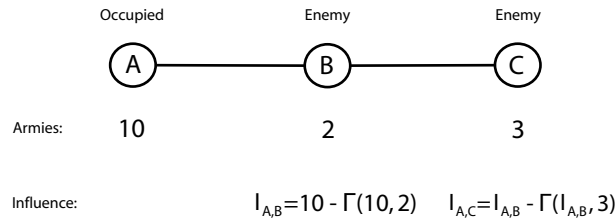


Figure 3.8: The influence the opponent territory *A* has on the territory *C* belonging to someone else is calculated using the influence on *B*.

through territories occupied by the opponent himself, so when a territory occupied by himself is met, the spreading of influence stops there.

In the situation of Figure 3.9 where more than one opponent territory are connected to the same territory occupied by someone else, the influence of the opponent territories is simply added. This makes sense because if the opponent wanted to conquer *C* he would be able to use the armies in each of his territories in the attempt. If *C* had also been an opponent territory, the combined influence of *A* and *B* is also calculated as the addition of these.

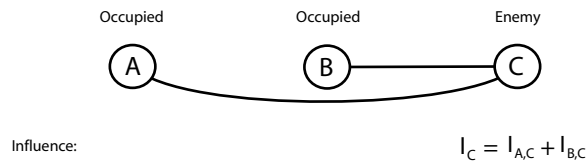


Figure 3.9: The combined influence of opponent territories *A* and *B* on the territory *C* belonging to someone else than him.

Using the Influence Maps

An influence map covering all the territories on the board can then be calculated for each player. Each node in the influence map corresponding to an enemy territory will then have a value representing an estimate on how many armies the player would have left if he tried to conquer the territory at all costs. Each node corresponding to a territory occupied by the player will have a value representing the amount of armies he could use from his neighboring occupied territories in attacking it, should it be conquered by an enemy.

To find out how threatened an occupied territory is, only the enemy player with the most influence is considered. The other enemies need not to be considered because the worst that can happen is that the enemy with the most armies attacks. The influence of the enemy with the biggest influence on the occupied territory corresponds to the number of armies he can attack it with. This value will be called A_e .

To find the number of armies that defends the occupied territory, the actual amount of armies in the territory as well as the influence value I_o from the computer player's own influence map should be used. The influence value should not count fully as defense force, since the enemy may attack anyway

regardless of how many armies the computer player has in the neighboring territories. The influence value could for example count as 20% defense force, but will probably need to be tweaked when the game can be tested.

The estimated amount of armies that would be left after the enemy has attacked is

$$a = A_o - A_e - \Gamma(A_o + I_o \times 0.2)$$

where A_o is the number of armies in the occupied territory o , A_e is the number of armies in the enemy's attack force, and Γ is the estimated cost function.

If a is positive then it will cost no extra armies to defend the territory. If a is negative then the cost of defending the territory is a negated. The cost of defending the occupied territory is therefore

$$C_o = -a \quad (3.24)$$

so a negative C_o means C_o armies are in surplus on this territory. Negative values will only be of use in the Transfer Armies module which will be describe as the last in this section. For simplicity, C_o should just be set to zero if negative when C_o is given to other modules, since that is all the informaion they need.

Decision Trees

The representation of the board formerly used in the decision trees does not work in this case, since what we are interested in here is the threat to a specific territory and not the power balance within a continent. A way to represent this information is to calculate the threat to the territory through the use of an influence map locally for that territory. The data is presented as input to the decision tree through a series of attributes *threats*. There are three of these where the first one is the combined threat from territories neighboring the territory, the second threat is for territories two territories away, and the third is three steps away. The threat is calculated by, for the territory in question, calculating the number of opponent armies in the neighboring territories and subtracting the number of armies in the territory. The range of this result is infinite since a random number of armies can be placed in the territories. Therefore we, again, restrain the input to lie within ranges. These ranges are [< -100 , -80 – -91 , -90 – -61 , -60 – -41 , -40 – -31 , -30 – -21 , -20 – -11 , -10 , -9 , -8 , -7 , -6 , -5 , -4 , -3 , -2 , -1 , 0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 , 10 , 11 – 15 , 16 – 20 , 21 – 29 , 30 – 39 , 40 – 59 , 60 – 79 , 80 – 100 , >100]. The target attribute is the cost of defending the territory. The range of this is integers: $[0, 1, 2, \dots, 100, >100]$.

Neural Networks

An ANN for this module has the following input:

The Board: This has one of two different representations: 42 pair of nodes or the representation presented in the above decision tree: three inputs indicating the threat on the given territory.

Territory: A numeric representation of the territory in question is needed. This could be as follows: $\frac{\text{TerritoryIndex}}{42}$ resulting in a single node.

The output node is a single node stating the estimated army cost. This needs to be in ranges as the input “Estimated Army Cost” for the “Discard Attack Plan” such that the output: [0, 0.05, 0.1, 0.15, 0.2, ..., 1] corresponding to [0, 1, 2, 3, 4, 5, 6-7, 8-10, 11-14, 15-19, 20-25, ..., 100].

In neuroevolution, the evaluation of the cost is correct or not, is argued in the Analysis only to be done globally at the end of the game. If the game was won, then all cost calculations were good.

Bayesian Networks

Instead of representing the full board which without doubt would give the most precise result, it makes sense to have some abstraction over the board as it looks close to the territory to avoid the BN getting too large. This BN is seen in Figure 3.10. Here the “MyArmies_{-T}” and “OppArmies₋₁” are the number of armies in the territory and the opponents neighboring the territory. “MyArmies₋₁” are the AI’s armies in territories neighboring this territory and “OppArmies₋₂” are the territories two territories away and so forth. We make the limitation that we only consider continents three steps away. The reason for this structure is that to get to the target territory, the opponent may need to attack through the territories neighboring the target territory, these are naturally not always occupied by the AI, but to keep the model somewhat simple, this is an assumption that we make. Otherwise we would have to have a node for each player instead of the “OppArmies” nodes making the structure more complex than necessary. Since it is not possible to attack with all armies (one is always left behind), the opponent territories at the first step from the territory are subtracted by one army. The opponent territories two steps away are subtracted two each, since they need to go through another territory to get to the target territory, and so forth. These army nodes are all parents to their respective “Armies_{Left}” node, which is the same as that used when estimation how close an opponent is to occupying an entire continent on Page 103. This approach may not give a very precise result, however this approach lowers the number of states in this node which is beneficial for computational reasons. If this in fact turns out to be too imprecise, alternatively the node could have a numeric range denoting the precise number of armies left instead. The “Armies_{Left}” nodes are all parents to a node which states the number of armies needed to defend the territory.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

3.7.6 Prioritize Territory Needing Defense

This module will prioritize how important it is to give a territory reinforcements compared to other territories.

Input: Territory, MP goals, opponent’s next move, board, number of cards owned by opponents.

Output: Priority.

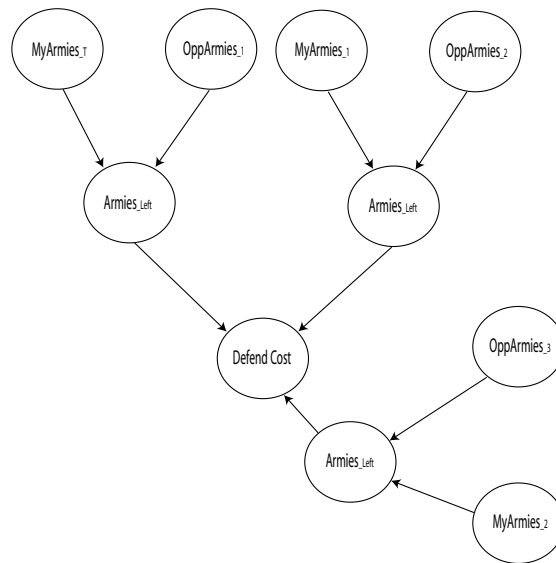


Figure 3.10: The BN used to calculate the cost of defending a territory. Only neighboring opponents up to three steps away from the territory are considered threats.

Scripting

The most important things that should give the defense of a territory high priority are:

1. If occupying the territory will help satisfy an MP goal
2. If it is a border territory in an occupied continent
3. If the territory will be attacked by an opponent this round

For each item in the list above, the occupied territories on the board can be given points depending on how well they fulfill them. The territory with the most points will then be given the highest priority while the territory with the fewest points will get the lowest priority.

To give points according to MP goals, the territory should receive a number of points equal to the priority of the goal it helps satisfy. This gives four different cases, one for each MP goal type:

- Conquer continents:
If the MP goal involves the continent to which the territory belongs it should be given points.
- Kill player:
If the territory is neighbor to one of the target player's territories it should be given points.
- Conquer territories:

If the goal is “Conquer 24 territories”:

The territories neighboring weak enemy territories should be given points since having a big number of armies in these will make it possible to conquer more territories.

If the goal is “Conquer 18 territories with 2 armies on each”:

Territories with exactly one army should be given points. Territories neighboring weak enemy territories should also be given points, but not as big a percentage of the MP goal’s priority as a territory with one army. The percentage could for example be 80%/20% in favor of territories with one army.

- Defend continent:

If the MP goal involves the continent to which the territory belongs it should be given points.

In the case of giving points to border territories to an occupied continent, it would make sense to not just blindly place the points on a territory inside the continent if it is neighbor to a territory on another continent. It might be so that the territory on the other continent is also occupied and that territory has enemy territories on its other side. This is the situation on Figure 3.11. *B* (and *C*) actually has the role of a border territory since it separates *A* from the enemy territory *X*. The points should therefore be given to *B* and *C* instead of *A*.

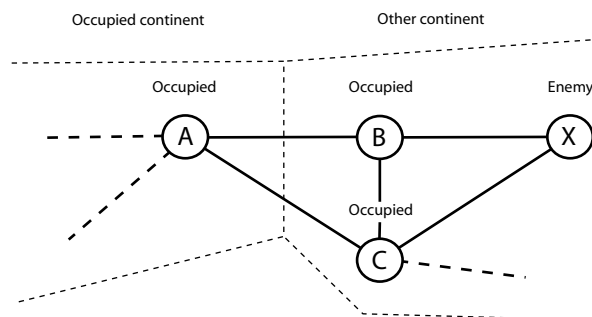


Figure 3.11: *B* and *C* act as border territories for the occupied continent and should be given points instead of *A*.

The number of points each given *B* and *C* should be equal to number of points *A* would have received if *A* had a territory as neighbor. If the border territories are found for each continent it may be the case that a territory serves as border territory for two continents and the points should then just be added. The theoretically maximum number of points a territory can be given is then 1, if it served as a border territory for all continents and all continents on the board were occupied.

If the opponent’s next moves shows that a territory will be attacked, the territory should be more important to defend. This could be expressed as a factor the territory’s priority should be multiplied by. This factor could be 0.5 for territories that are not attacked and 1 for territories that will be attacked.

The minimum value of 0.5 is to give territories that are important to occupy a chance of receive reinforcements. The minimum value may need some tweaking in the game test.

The priority output by this module is found by combining the values for each of the ways of giving points. The first two gives each a value between 0 and 1 and the last is a factor between 0.5 and 1 by which the priority should be multiplied. If the output value is negative it should be set to 0 since 0 is the lowest priority a territory can get. The module's output will then be a value between 0 and 1, given by

$$P = \frac{p_1 + p_2}{2} \times f \quad (3.25)$$

where p_1 and p_2 are the points given for satisfying an MP goal and being a border territory and f is the factor on whether the territory will be attacked.

Decision Trees

"Opponents' next moves" is input through a new attribute "IsInOpponentsNextMove" with the states "Yes/No". This can be done since all we are interested in is whether or not the territory is attacked the following round. The target attribute "Priority" is a percentage scale. Each MP goal and the number of RISK cards are integers ranging $[1, \dots, 100]$ and $[1, 2, 3, 4, 5]$ respectively.

Neural Networks

The ANN for this module would have the following input node:

Territory: The numeric representation for a territory is done in the same way as for the ANN for the "Calculate Defense Cost" module.

MP goals: Each of the 37 values in the goal distribution need also be input nodes in the ANN.

Next move: The only information needed from a given opponent's next move, is whether or not the territory in question is attacked in his next move. Then only a single binary 0/1 node for each of the opponents is needed.

Opponents' RISK cards: This is done as in the ANN for the module "Discard Attack Plan": a single node with 6 different ranges between 0 and 1.

This result in an ANN with 44 input nodes.

The output is a single neurode stating the priority of the territory needing defense.

Evaluation can again not be done locally, since there is no way of knowing how the placement of armies use the information given by the module, so it is hard to say if it was a good priority. But it can be done globally at the end of the game.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

3.7.7 Cash Cards

This module determines whether the Risk cards on hand should be cashed this turn.

Input: List of priorities, list of minimum and estimated number of armies needed to satisfy each priority, number of armies gained from occupying territories and continents, Risk cards on hand.

Output: Yes/No.

Scripting

If the combination of Risk cards yields 10 armies they should be cashed right away since it is the maximum number of reinforcements that can be gained. If the computer player has less than three Risk cards on hand or five or more cards on hand, this module needs not to consider anything because the cards then either cannot be traded or have to be traded. If this is not the case (the computer player has three or four Risk cards) it must be decided whether to trade the cards or not.

Given the number of armies gained from occupying territories and continents and the number of armies that would be gained from cashing the Risk cards on hand, two values, a minimum and a maximum number of reinforcements r_{min} & r_{max} , can be calculated. r_{min} is simply the number of armies gained from occupying territories and continent and r_{max} is r_{min} plus the armies that would be gained from cashing Risk cards. If $r_{min} = r_{max}$ then the Risk cards cannot be cashed because the cards on hand does not yield a valid combination. If this is not the case then it must be decided if the Risk cards should be cashed or not.

Two reasons exist when deciding whether to cash cards or not:

1. Cash cards if they make it possible to pay at least the minimum cost for a plan with very high priority (e.g. 90%).
2. Do not cash cards if the next card the AI draws from the deck will give a much better combination of cards, yielding a substantially bigger amount of reinforcements compared to the combination possible now.

Only if there is no plan with a priority satisfying reason number 1, reason number 2 should be taken into account. If the cards' value is only 4 armies or 6 armies and if receiving one more card could improve the value to 10 armies, postponing the card cashing to a later round would be a good idea.

Neural Networks

The ANN for this module has the following input:

List of priorities: This is a list of maximum 42 values. So there will be 42 input nodes representing this, where the unused inputs will be zero.

List of minimum cost: This is also a list of maximum 42 values. So there will be 42 input nodes representing this, where the unused inputs will be zero. The value of the input will be subdivided into ranges as with the ANN for the “Discard Attack Plan”.

List of estimated cost: And this is also a list of maximum 42 values. So there will be 42 input nodes representing this, where the unused inputs will be zero. The value of the input will also be divided into ranges.

Number of reinforcements: This will be done in the same manner as the same input to the ANN for “Discards Attack Plan”: $\frac{NumReinforcements}{38}$.

RISK cards: This will be done in the same manner as the above decision tree: 5 input nodes each stating what kind of card it is.

This will result in 132 input nodes.

The output is binary 0/1.

When training with neuroevolution, it would be best to evaluate globally at the end of the game, as written in the Analysis.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

3.7.8 Place Armies

This module takes care of placing the armies received as reinforcement one by one in the occupied territories on the board.

Input: List of territories needing reinforcements, list of priorities, list of minimum and estimated costs, number of reinforcements left.

Output: Territory.

Scripting

Basically, choosing which territory’s cost to pay can be done in two ways: Either by choosing the territory with the highest priority or by choosing the territory with the lowest cost. The first way will make sure important territories will have their cost payed, the last will pay the cost for as many territories as possible. The problem with the last way is that it ignores the priority totally and so high priority territories may not receive any armies just because their cost is a little higher than some low-priority territories with low cost.

Basing the decision on the priority does seems like a good idea since it will make sure that important territories get what they need. If two territories have equal priority, the one with the lowest cost should be payed for. This will way there is a better chance a high priority mission will be completed.

Decision Trees

The problem with the three dependent lists is solved by splitting “List of territories” into two attributes: “Territory_Priority” with the states being integers between 0 and 100, and “Territory_Cost” with the states [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11-15, 16-20, 21-29, 30-39, 40-59, 60-79, 80-100, >100]. Each territory on the board is represented by these two attributes. The cost is calculated by taking the average of the minimum cost and the estimated cost for each territory. The number of reinforcements left is an integer between 3 and 48. The target attribute is a territory. However, since the target attribute is a territory, eventually the tree might learn to output territories not in the input list. This is countered by, before entering the data, converting the territory to a float between 0 and 1 that maps it onto the input list of territories. This means that the output of tree will be a float instead of a territory. This float is then remapped to a territory.

Neural Networks

Such an ANN would have the following input:

List of territories and priorities: The two lists will be combined to a single list, where each entry in the list contains a priority. The territories that are not in the list of territories would then have priority 0. Each element in the list will then always corresponds to the same territory on the board. For example: the priority of Ural will always be in entry 0 and the priority of Alaska will always be in entry 34. This list will always have a size of 42 and the ANN will therefore have 42 input nodes for this list.

List of minimum cost: This is also a list of maximum 42 values. So there will be 42 input nodes representing this, where the unused inputs will be zero. The value of the input will be subdivided into ranges as with the ANN for the “Discard Attack Plan”.

List of estimated cost: And this is also a list of maximum 42 values. So there will be 42 input nodes representing this, where the unused inputs will be zero. The value of the input will also be divided into ranges.

Number of reinforcements: This will be done in the same manner as the same input to the ANN for “Discard Attack Plan”: $\frac{\text{NumReinforcements}}{48}$ (this is including possible reinforcements from cashing RISK cards).

The relation between priority and estimated/minimum cost could have been dictated to the ANN and represented in 42 relation-values, instead of 42 nodes for each priorities list, estimated cost and minimum cost lists. But that would take away the ANN’s chance of figuring out a better relation. So these are all fed to the ANN and hopefully it will make some sense out of them, of course based on the training data given.

The output is a bit tricky. There is a constraint on the output, that the territory outputted has to be in the list of territories given as input. This can be

done by mapping the output value, which is a value between 0 and 1, to an index in the list of territories. The output value 0 could map to the first territory in the list and 1 to the last. This will result in a single output neurode.

When training with neuroevolution, it would be best to evaluate globally at the end of the game, as written in the Analysis.

Naive Bayes Classifier

The input and target attribute for the naive Bayes classifier are represented as that in decision trees.

3.7.9 Transfer Armies

In the end of the turn, this module's responsibility is to determine if a transfer should be made and if so, between which territories the army transfer should be made and the number of armies to include in the transfer.

Input: List of territories needing reinforcements, list of priorities, list of minimum and estimated costs, list of all occupied territories.

Output: Source territory, destination territory, number of armies to transfer (0 = no transfer).

Scripting

It is difficult to script a behavior that covers all aspects in an optimal way, but the important subjects to take into consideration will be described here.

Each territory in the list of the territories needing reinforcements has been marked as originally belonging to a defense list or begin the starting territory in an attack plan. The difference need to be made because placing armies according to the priority of an attack plan territory will be made on a basis more inaccurate compared to a defense list territory. The board configuration will have changed the next time it is the computer player's turn, so the starting territories in attack plans generated next round will not be made on exactly the same board state as now. The defense of territories on the other hand will be based on exactly the board configuration as now. The knowledge of which territories will be long to an attack plan next round and the priority and costs of them can therefore only be considered as an estimate. It would therefore make sense to reflect this in the priorities of attack plan territories and multiply them with factor corresponding to the certainty on how the board configuration will be similar to the one now. This could for example be 75% but this will probably need some tweaking in the test phase.

After the whole list of territories has been updated this way, the search for the transfer's destination territory can begin. The destination territory can simply be picked as the highest priority territory in the list. To find the source territory, its occupied neighbors should then be tested one by one to see how many armies they can spare without making themselves vulnerable or unfit for starting an attack from. In any case it will not be a good idea to use a territory as source if its priority is high, and it should definitely not have a priority higher than the destination territory.

The amount of surplus armies in a source territory can be read from its cost - if the cost C_s is below zero then the number of surplus armies is $a = -c$. If a is enough armies to pay the destination territory's cost C_d then the transfer should be made between these. If, for a given source territory, a is close to C_d then it would make sense to perform the transfer anyway.

Neural Networks

All inputs in the ANN are the same as for the ANN to the "Place Armies" module, except the "Number of reinforcements left", which does not apply to this module. The list of occupied territories is represented in the same way is the list of territories needing defense.

The outputs have the following constraints:

- Both the source and destination territories needs to be in the list of occupied territories given as input. The output value is then mapped to an index in this list.
- The number of armies output must also somehow map to how many armies there are available in the source territory. So if the output value is 1, then 100% of armies available in the source territory is transferred to the destination territory.

This results in 168 input nodes and 3 output nodes.

When training with neuroevolution, it would be best to evaluate globally at the end of the game, as written in the Analysis.

3.8 Initial Army Placement

This module takes care of placing the armies in the very beginning of the game.

Input: Board, Mission.

Output: Territory.

Scripting

Getting hold of a continent right from the beginning of the game will always be a big advantage. Therefore, trying to get a continent yourself and keep other players from getting one is important. Of course the mission should also have an influence on where the armies are placed, but trying to occupy any continent or get in position for it, is most important since this will give a good basis for the rest of game. If the AI from the start occupies a good deal of territories in a continent compared to the opponents, armies should be placed in the most threatened territories in the continent. If the situation is the opposite and an opponent occupies a good deal of a continent, the AI can try and obstruct the continent by placing some armies in one of the occupied territories there, if any.

Taking the mission into account gives a different strategy, depending on the mission. The three types of missions are:

1. Kill player
2. Conquer continents
3. Conquer a number of territories

If the AI's mission goal is to kill an opponent player, it should look at where the opponent places his armies and place armies somewhere near them. Also, it should prevent especially that opponent from easily gaining continents by placing armies, if possible, in continents where the opponent has the majority of the territories from the start.

If the AI has a mission where it has to conquer continents, it should of course see if it can become the largest force on any of these continents, preferably the ones that are easiest to defend after taking them. If an opponent starts placing armies in one of the continents the AI needs, it should also place armies in that continent if possible.

If the AI has a mission where it needs to occupy 18 or 24 territories, the IAP cannot do a specific thing to get closer to completing the mission since the number of occupied territories in the beginning of the game is fixed. However, it could try to place armies near weak opponent territories, but most important is to try to get control over any of the continents, preferably one of those that gives a lot of reinforcements.

To decide where to place the armies, each territory could be given points according to how important it is to place an army there. This can be done using a combination of the methods mentioned in "Prioritize Attack Plan" and "Prioritize Territory Needing Defense", taking into account the importance of occupying or getting close to occupy a continent. The territories with the highest score are then the most interesting to place armies in.

Decision Trees

The input of the board is straight forward: one node for each territory. As with the "Place Armies" task, the tree learning to output a territory not occupied by the AI is possible. This is solved in the same way as in "Place Armies".

Neural Networks

The ANN for this task takes a representation of the board as input. This representation is 6 nodes each representing the power distribution in each continent. This has been described previously. The reason for using this lossy representation, is that, as argued in the script for this module, the most important information the placement should base its decisions on, is the other players' influence in the continents.

The AI's mission is represented as a single input node representing one of the 14 missions.

The output is a mapping to one of the AI's occupied territories, as done in the output for the ANN for the "Place Armies" module.

Evaluation is done globally at the end of a game.

Naive Bayes Classifier

The input attributes and target attributes are represented as those of decision trees.

3.9 Finding the Best Combination of AI Modules

When all learning algorithms have been trained to a point where their performance is satisfactory, or where no further training is possible, then it is time to find the best combination of AI techniques in the modules. This section will contain a discussion on how that combination can be found.

Finding such a combination is in fact a search for the optimal playing AI, which is ideal for a genetic algorithm.

A population of AIs with a random AI techniques for each module will be made. Those AIs will play each other and based on the outcome of those games, the AIs will have their fitness adjusted accordingly.

But for a genetic algorithm to be used, some way of representing a specific AI as a binary string is needed. This can be done in the following way:

Each of the 6 AI technique get its own binary representation and collecting those binary representation with the 16 different modules will make a binary representation of a specific AI - a genome. The binary representation of AI techniques could be as shown in Table 3.2.

Random	000
Scripting	001
Decision Tree	010
Neural Network	011
Bayesian Network	100
Naive Bayes Classifier	101

Table 3.2: *The binary representation of each of the AI techniques.*

The binary representation *111* is not used. If such a representation is found in genome, then that genome is stillborn and will be thrown away.

Having these binary representations concatenated 16 times with a genome representing an AI. Such a genome can be seen in Table 3.3.

The genome *000001000001011011001001010011010001000000011001* in Table 3.3 use a *random* initial army placement, *scripting* when estimating opponents' mission, *neural network* when estimating how close an opponent is to winning and so on. In this way, any AI can be represented as a genome for a genetic algorithm. In the same manner as for the *111* binary representation, a genome stating that for instance a decision tree has to be used in a module where it was found that decisions trees can not be used, then such a genome is stillborn - it can not function and is thrown away.

Initial army placement	000
Estimating opponents' mission	001
How close an opponent is to winning	011
The opponents' next moves	001
How close an opponent is to owning a continent	011
Master Prioritizer, goal weighting	011
Make attack plan	001
Calculate attack plan cost	001
Prioritize attack plan	010
Discard attack plan	011
Pick territory needing defense	010
Calculate defense cost	001
Prioritize territory needing defense	000
Cash cards	000
Place armies	011
Transfer armies	001

Table 3.3: A genome representing a specific AI. Each of the IGs, MP, the modules in the RP and the Initial army placement module have an binary representation of which AI technique is used in the given module.

As written before, the algorithm starts out with a population of random genomes. These genomes play against each other and their fitness is tweaked according to the outcome of the game. The best performing genomes are combined and mutated and the playing is repeated. In this manner the best overall performing AI can be found, and it can be determined whether a scripted AI or a learning AI is best - or perhaps a combination.

Chapter 4

Conclusion

The goal of this project was to compare AIs for computer games using different learning and non-learning techniques. To do this we first found an already existing computer game called JRisk that seemed suitable for implementing and testing the different AIs. We then analyzed the different things a player needs to take into account when playing the game and then discussed a possible structure of the AI. We found that a layered structure with three parts (Information Givers, a Master Prioritizer, and a Round Planner) would be suitable. For each of the parts, the different tasks the AI should perform were identified and put into a module each. This enabled us to test different AI techniques for each separate task in the framework. Next, we chose some commonly used AI techniques and discussed different approaches to make learning AIs able to learn playing RISK. We then combined each AI technique with each module representing a task and analyzed which techniques would be suitable for each module.

Then we designed the overall framework class structure and decided how the training data should be organized. For each of the modules in the IG, MP, and RP parts of the framework, we designed how each suitable AI technique could be used to solve that module's task. This resulted in a complete design of how the AI techniques should be implemented in the game. Finally, we presented a way to test all the possible different combinations that results in using different AI techniques for different modules.

Reflection

Now that the project has come to an end, reflections can be made upon whether the decisions made could have been different.

Module Design

The designed framework is very modular and allows for testing of most of the AI techniques for each task. Each module has a well-defined interface through

which all communication flows, making the module independent of the technique working inside it. This makes the design very suitable for our purpose.

However, constructing a framework with fewer modules each taking care of broader tasks could result in improved performance of some of the AI techniques, compared to if tasks are split up and the techniques applied to each of these sub-tasks, as we have done. Also, in identifying tasks and modules in our framework, each module could have been designed directly for a given AI technique, possibly improving its performance. For instance, perhaps the planner part of the AI could perhaps have been covered by one decision tree, but as also discussed in the Analysis, making the framework very modular gives us the opportunity to compare many different AI techniques for each small module. If the goal of this project were to develop a perfectly playing RISK AI, then such an approach might have been employed.

Shortcomings in the Designed AI

As already argued, the goal of this project is not to develop a perfectly playing AI, but however in order to compare the performance of different techniques, the AI needs to at least play a decent game of RISK. This should not be a problem, since over time, the learning algorithms should learn to play better and better RISK. But this last statement only holds if our developed framework does not hinder the AIs in learning all the subtleties of playing RISK. For instance, when a player has conquered the continent of Australia, then it is actually best defended from the territory of Siam (in the continent of Asia), and not Indonesia, which is the only border territory of Australia. From a “defend Australia” point of view, it does not really matter whether Australia is defended from Siam or Indonesia, but in Siam, one also obstructs anyone from conquering Asia, making this a better choice. It can be argued whether these little subtleties are utilized by the AI, or that the framework in its use of the attack plan scheme hinders the AI in using these small ways of playing. But we believe our framework will accommodate an AI that should learn to play a decent game of RISK, making a comparison between the different techniques possible.

Another matter that needs a bit of reflection, is whether our framework makes the AI too reactive and not so much a planning AI. If the plans of the round planner that potentially would take more than 1 turn to complete were removed, the AI would be reactive, only reacting to the state of the board in this round. But plans covering more than one turn are not removed. If they are of high enough priority they are kept, and carried out as far as possible. In the next round, the RP should recreate the rest of the plan (provided the MP produce a similar goal distribution) resulting in an AI planning over more than one rounds.

Future Work

Based upon our work in this project a number of interesting topics for future work can be proposed.

1. Implementation of the framework. This is both implementation of the structure of the framework and implementation of the design for each AI technique in each module. After the implementation of the framework, implementation of methods for developing training examples is needed. Lastly implementing methods for structuring the training examples such that they easily can be fed to each learning algorithm can be done.
2. Testing the different learning and non-learning algorithms against each other, finding the best combined AI. This requires, however, that at least all scripts are implemented along with some subset of the learning algorithms. Preferably the entire design is implemented to accommodate a complete test. When such a test has been made, the matter of determining why a given AI technique performs better than another could be an interesting topic. Also how one would define “the best playing AI” could be an interesting topic. It could be based directly on how successful the AI is in winning against other AIs. But these other AIs could be the set of either the second best AIs or all the other AIs. Or perhaps the best AI is the one that is the most fun playing against, where a human player sometimes wins and sometimes loses.
3. Lastly, one could study the outcome of such an test, reflect upon how the different AI techniques would perform in a different game environment and thereby determine the generality of the techniques. This could perhaps be done by developing a similar framework for another game and determine the best AI combination, and compare it to the findings of the above test. Are the best AI in Risk also the best AI in other games?

Bibliography

- [Dav89] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1989.
- [Gar93] Gary Chartrand and Ortud R. Oellerman. *Applied and Algorithmic Graph Theory*. McGraw-Hill, Inc., 1993.
- [hug] Hugin Experts A/S - Hugin Researcher. <http://www.hugin.com>.
- [J. 92] J. D. Schaffer and D. Whitley and L. J. Eshelman. Combinations of Genetic Algorithms and Neural Networks: A Survey of the State of the Art. In L. Darrell Whitley and J. David Schaffer, editors, *International Workshop on Combinations of Genetic Algorithms and Neural Networks*. IEEE Computer Society Press, 1992.
- [Jen02] Finn V. Jensen. *Bayesians Networks and Decision Graphs*. Springer, 2002.
- [Joh02] John Manslow. Learning and Adaptation. In Steve Rabin, editor, *AI Game Programming Wisdom*, chapter 11.1. Charles River Media, inc., first edition, 2002.
- [jri] JRisk by Yura Mamyrin. <http://jrisk.sourceforge.net/>.
- [Lar00] Lars Mathiassen and Andreas Munk-Madsen and Peter Axel Nielsen and Jan Stage. *Object Oriented Analysis and Design*. Marko Publishing ApS, 2000.
- [P. 68] P. Hart and N. Nilsson and B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on System Science and Cybernetics*, SSC4(2):100–107, 1968.
- [Ric98] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction (Adaptive Computation and Machine Learning)*. MIT Press, 1998.
- [ris] FAQ for the Board Game RISK. <http://www.kent.ac.uk/IMS/personal/odl/riskfaq.htm>.
- [Rus90] Russel C. Eberhart and Roy W. Dobbins, editor. *Neural Network PC Tools - A Practical Guide*. Academic Press, Inc, 1990.

- [sta] Amit J. Patel's Page on Heuristics at Stanford University.
<http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html>.
- [Tho] Thomas D. Nielsen. Notes on EM Algorithm.
- [Tom97] Tom M. Mitchell. *Machine Learning*. McGraw-Hill Companies, Inc., 1997.
- [Toz02] Paul Tozour. The Perils of AI Scripting. In Steve Rabin, editor, *AI Game Programming Wisdom*, chapter 10.6. Charles River Media, inc., first edition, 2002.
- [Woo02] Steven Woodcock. Recognizing Strategic Dispositions. In Steve Rabin, editor, *AI Game Programming Wisdom*, chapter 5.2. Charles River Media, inc., first edition, 2002.

Appendix A

The Rules of RISK

RISK is a board game with three to six players. The board consists of 42 territories. Each territory belongs to one of 6 continents. In Figure A.1 the board of a RISK game is depicted.



Figure A.1: The board in a RISK game.

At the beginning of the game, territories are distributed equally and randomly between each player. Also, each player is assigned a secret mission that he must solve in order to win the game.

The possible missions are:

- Conquer Asia and South America

- Conquer Asia and Africa
- Conquer North America and Africa
- Conquer North America and Australia
- Conquer Europe and South America and a 3rd continent of your choice
- Conquer Europe and Australasia and a 3rd continent of your choice
- Occupy 18 territories with at least 2 armies in each territory
- Occupy 24 territories
- Kill black player
- Kill purple player
- Kill red player
- Kill yellow player
- Kill blue player
- Kill green player

If your own color is the color you need to kill, or if someone else kills that color, your mission changes to "Occupy 24 territories".

Initially each territory is occupied by a single army. Each player then gets a number of armies to place in the territories they occupy. This number is dependent on how many players there are in the game:

- 3 players: 35 armies each
- 4 players: 30 armies each
- 5 players: 25 armies each
- 6 players: 20 armies each

Then in turn the players place an army in one of their occupied territories until no more armies are available. This is called the *Initial army placement*.

When this is done, the game is ready to start.

A player's turn consists of four phases:

1. Cashing in RISK cards
2. Defend
3. Attack
4. Fortify

For a better overview, Figure A.2 on page 145 shows the activities in the four phases.

A.1 Cashing in RISK Cards Phase

In the first phase, the player may trade in RISK cards for extra armies. RISK cards are earned from successfully conquering an opponent's territory. It is only possible to earn one RISK card in a turn. There are three types of RISK cards: a cannon cards, an infantry card and a calvary card. There exist one RISK card for each territory in the game. Each RISK card has the corresponding territory depicted on it.

It is possible to cash RISK cards when three or more cards are on hand. The traded sets have the following values:

- 3 Cannon: results in 4 extra armies.
- 3 Infantry: results in 6 extra armies.
- 3 Calvary: results in 8 extra armies.
- 3 Different: results in 10 extra armies.

A player must have a maximum of five RISK cards at a time, meaning he must trade cards before receiving the sixth.

If you own one of the territories depicted on the traded RISK cards, you may place two armies in that territory. This can only be done for one of the three RISK cards traded.

A.2 Defend Phase

In the defend phase, players also receive reinforcement armies for occupied territories. The number of armies are given the following way:

- The total amount of your territories divided by 3, with a minimum of 3 and rounded down (e.g. 14 territories gives 4 armies).
- If you own South America an extra 2 armies are earned.
- If you own North America an extra 5 armies are earned.
- If you own Europe an extra 5 armies are earned.
- If you own Africa an extra 3 armies are earned.
- If you own Asia an extra 7 armies are earned.
- If you own Australia an extra 2 armies are earned.

All armies are then placed in the territories of your choice before entering the next phase. This also include the armies gained from cashing RISK cards.

A.3 Attack Phase

The next phase is the attack phase. In this phase, which is optional, the player may try to conquer opponent players' territories. This is done by attacking from an occupied territory into a neighboring opponent territory. The attacker decides whether to attack with one, two, and three armies. The attacker use the amount of dice according to the number of armies he attack with (1 army = 1 die, 2 armies = 2 dice and 3 armies = 3 dice). The defender has the same choice, but can only defend with 2 armies.

An attack is done as follows:

1. The attacker decide where to attack (source and target territory).
2. The attacker decide whether to attack with 1, 2 or 3 armies.
3. The defender decide whether to defend with 1 or 2 armies.
4. The attacker and defender both roll their dice.
5. The dice of both attacker and defender are ranked according to value and are compared for each rank. If, in one rank comparison, the dice are the same, the defender wins in this rank.
6. The defender and attacker lose the amount of armies equal to the rank comparisons they lost.
7. The attacker then decide whether to attack again or to stop (retreat). If the attacker decides to attack, step 2-7 are repeated. If the attacker has no more armies to attack with, he has lost and must continue to attack somewhere else or move on to next phase.
8. If the attacker has won he has to move a minimum of X armies to the captured territory, where X is the amount dice he used in the winning roll.

When conquering a territory, the attacker must leave at least one army in the territory he attacks from.

A.4 Fortify Phase

In the final phase, which is also optional, the player may fortify a territory by moving armies from exactly one territory to a neighboring territory. He must leave at least one army at the territory he moves armies from.

When a player kills another player, he gets all the killed player's RISK cards.

A.5 RISK Activity Diagram

These four phases results in Figure A.2, which is an activity diagram for a player in a game of RISK.

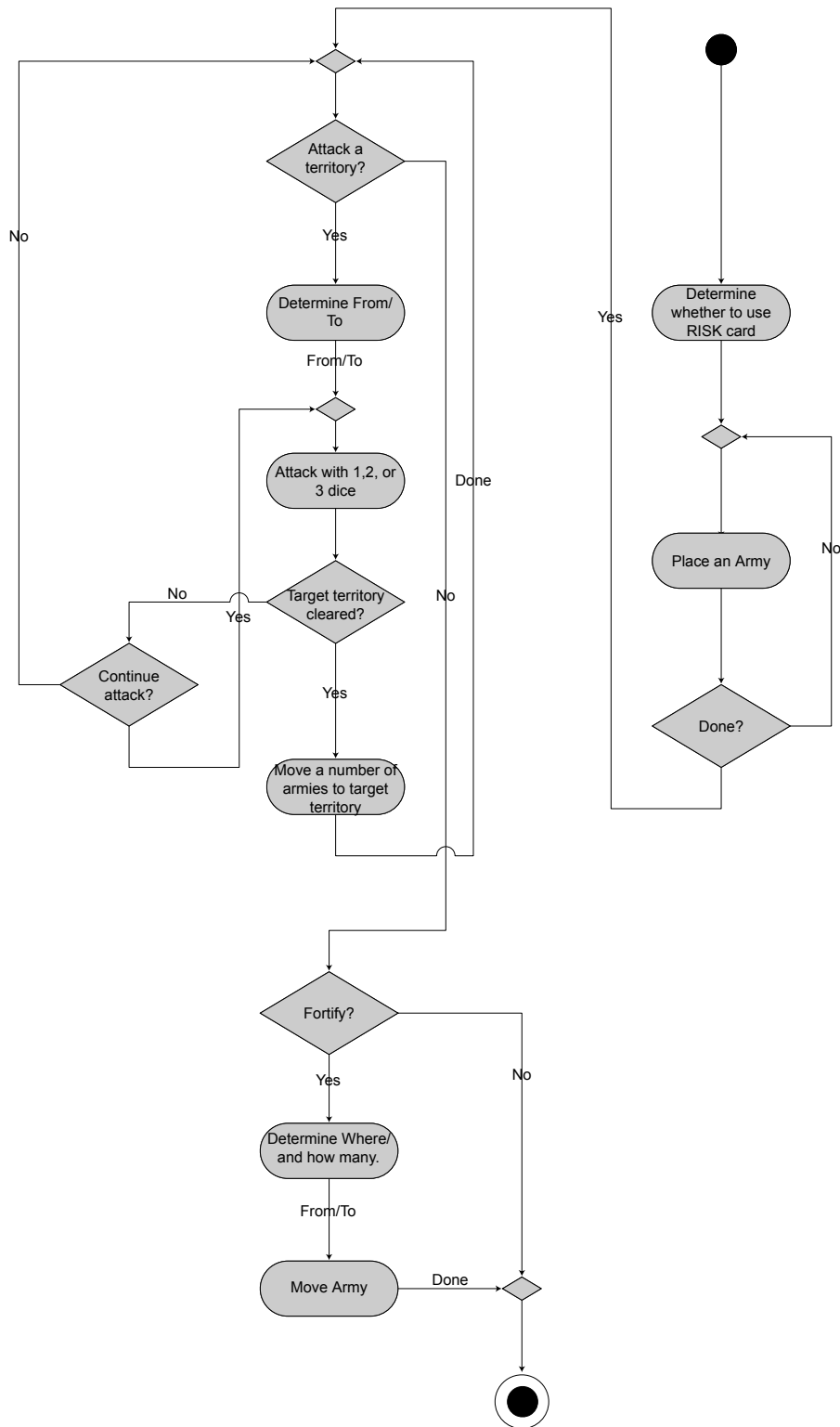


Figure A.2: The activities of a player's turn in RISK. Diamond shapes are conditions and rounded boxes are actions. The starting point of the figure is the black dot. Phase 1 is the first action "Cash RISK cards" following the starting point. Phase 2 is the next action and following condition on whether the placement is done. Phase 3 is the next six actions and conditions which represent the "Attack" phase. The last condition and two actions are phase 4, "Fortify". The turn ends in the circle at the bottom.

Appendix B

Analysis Appendix

This chapter of the report contains the analysis appendix. The chapter will contain analysis material that seemed to extensive to have in the actual report.

The reader should be aware that there is not any continuity between sections in this chapter. This is meant as a lookup when a greater detail in some part of the Analysis. This chapter however, does not contain text, that has not been referenced to in the actual report.

B.1 Scripting Example

To illustrate how scripting is used in practice, imagine a simple two-player game where the purpose of the game is surviving and killing the opponent. Both players have a certain amount of *Health* which ranges from 0 to 100. They have some *Ammunition* in their weapons which ranges from 0 to 10. The environment is a square map with randomly placed *Ammunition Packs* and concrete walls. The players can both carry 2 *Ammunition Packs* at a time which enables him to reload his weapon.

The players have the following abilities:

Move:

Moving is done in every direction (N, S, E, W) at a constant speed.

Sprint:

Sprint gives a 5 seconds speed boost, where movement speed is increased by 15%.

Shoot:

Shooting is done at a specific target. The accuracy of which the shot is fired is based on the *Health* attribute and the distance to the target. A successful hit results in the opponent loosing 10 health points.

Heal:

Healing is done by standing still and using the *Heal* command. It takes 20 seconds to heal from 0 to 100. If interrupted while healing, healing restarts unless another action is used.

Reload:

Reloads the weapon. This takes 1 second.

The basic scripts needed for this small game are:

Find opponent:

This script is run when the AI is ready to search for the opponent. This means that he believes that he can actually survive meeting the opponent.

Kill opponent:

This is run when the AI meets the opponent and still believes he can win the battle.

Flee:

This is run when the AI no longer believes he will survive the battle.

Heal:

Heal should be run when the AI has successfully escaped the attacking opponent.

Reload:

Reloading should be done when in combat the last shot have been fired. When not in combat, reloading should be done if the weapon is not fully loaded.

Get ammunition packs:

This should be run if the AI runs out of ammunition while in battle. If he has not got 2 packs of ammunition while searching for the opponent, he must pick some up.

Sample Scripts

In Figure B.1 there is a script which will make the AI search for the opponent on the map. The conditions on this script is the ones located between *if* and *then*. If these conditions are not met, the script is not activated and the AI will have to do something else that search for an opponent. If the script is entered, the actions are performed one of a time. In the case of finding the opponent, the AI moves in each direction possible for a random amount of time. Making the amount of time moving in each direction random has two benefits. First, it is easy to implement compared to more advanced approaches. Secondly, it makes the AI more unpredictable. There is no way to know where precisely it is located after some time. This overcomes one of the cons of using non-random scripted AIs: Namely the predictability. This is also argued in [Toz02].

The script for killing the opponent is seen in Figure B.2. After an opponent has been seen, the AI will start shooting at him until the opponents dies or the AI gets too hurt to continue the battle or runs out of ammunition. While attacking, the AI will move closer to the target to improve accuracy. If the

```

Script: Find Opponent::
if
  Health >= 75 && NumberOfAmmoPacks = 2 && AmmoInWeapon = 10
then
  While OpponentNotFound
    Move North for RANDOM seconds;
    Move East for RANDOM seconds;
    Move South for RANDOM seconds;
    Move West for RANDOM seconds;
else
  DoSomethingElse

```

Figure B.1: Script for finding the opponent

AI has not found an opponent or loses him of sight, it will go back to *Find opponent* mode. Of course the *Find opponent* is not the most appropriate for this task since it does not consider where the opponents was last seen and could therefore start wandering in a totally different direction if the opponent disappears behind a wall, but this is merely a small example.

```

Script: Kill Opponent::
if
  OpponentSpotted
then
  While Health > 50 && AmmoInWeapon > 0 && OpponentNotDead
    ShootAtOpponent;
    MoveTowardsOpponent;
else
  FindOpponent;

```

Figure B.2: Script for killing the opponent in the game

If the agent ends up losing the battle, he will try to escape his attacker. He will, in this case believe he is losing if his health goes below 50 or runs out of ammunition. Escaping is, as seen in Figure B.3, done by using the *sprint* ability moving away from the attacker for a random amount of time. This is followed by moving randomly in various directions which hopefully will make the AI move behind a wall. Again, this is no way near perfect, but seems reasonable for the example.

Until now, all conditions have been stated directly in the scripts. However, there is another way of doing it. It is possible to take all the conditions from the different scripts and incorporate them in a “Master Script” which decides what to be done when meeting the different conditions in the game. An example of such a script is seen in Figure B.4. The advantage of doing this is that the

```

Script: Flee::
if
    Health < 50 OR AmmoInWeapon = 0 && OpponentNotDead
then
    if
        OpponentSpotted
    then
        Sprint;
        MoveOppositeDirectionOfOpponent for RANDOM seconds;
        Move North for RANDOM seconds;
        Move East for RANDOM seconds;
        Move South for RANDOM seconds;
        Move West for RANDOM seconds;
    else
        Heal;
else
    FindOpponent;

```

Figure B.3: *Script for fleeing from the opponent.*

designer gets a better overview of what is going on when certain situations emerges. However, it is easy to see that if the AI has many different attributes, scripting can quickly become a difficult task. Therefore getting this structured in a meaningful way is very important for the AI designer in order to ease e.g. debugging. Since every situation in a game needs to be handled by a script, designing the AI may become very difficult as the number of attributes on the AI and the working environment increases.

B.2 Artificial Neural Network Example

This section contains an example of an ANN. The ANN can be seen in Figure B.5. This network is a character recognition network. The network inputs each pixel intensity and outputs a value for each A, B etc. up to Z. The largest of the output values is the ANN's estimate on which character is given as input.

B.3 The A* Algorithm

This section will describe the main steps in the path finding algorithm A* and illustrate the pseudo code for the algorithm.

Before the algorithm can begin, a heuristic has to be defined. In the analysis it was explained how a heuristic table was set up to be used in the A* algorithm. Pseudo code for constructing such a table can be seen in Figure B.6.

```

Script: Master::
  if
    Health >= 75 && NumberOfAmmoPacks = 2 && AmmoInWeapon = 10 &&
    OpponentSpotted = false;
  then
    FindOpponent();
  else
    if
      Health >= 75 && NumberOfAmmoPacks = 2 && AmmoInWeapon < 10
      && OpponentSpotted = false
    then
      Reload();
    else
      if
        Health < 75 && NumberOfAmmoPacks : 2 && AmmoInWeapon = 10
        && OpponentSpotted = false
      then
        Heal();
        .
        .
        .

```

Figure B.4: Master script for the example game.

When the heuristic table has been set up, the main A* algorithm can begin. The algorithm can be seen in Figure B.7.

The following data structures are used in the algorithm:

- $g[n]$ - the cost of going from the starting node to the node n .
- $h[n]$ - the estimated cost of going from this node n to the target node.
- $f[n]$ - the sum of $g[n]$ and $h[n]$. The value of f represents our best guess for the path going through n . The lower the f , the better we think the path is.
- $parent[n]$ - this structure holds the best step backwards from a node n .
- **OPEN list** - a list of nodes that all potentially could be on the best path, but have not been explored yet.
- **CLOSED list** - a list of nodes that have been explored. All nodes connecting to a node in the CLOSED list have all been look at.
- **step_cost(n)** - this is a simple function that outputs the cost for entering node n , which is simply the number of armies located on n .

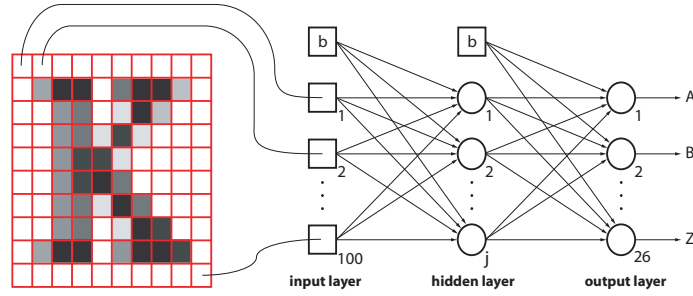


Figure B.5: A feedforward example of character recognition. The network inputs each pixel intensity and outputs a value for each A, B etc. up to Z. The largest of the output values is the ANNs estimate on which character is given as input.

```

Construct a table T of size (N,N), where N is the number of nodes in graph G.
Initialize all values in T to a value larger than N.
for each node n in the G do
    update T(n,n) with zero.
    call recursive-heuristic-update(n,n).
end for.

function recursive-heuristic-update(node s, node t)
    for each neighbor m to s do
        if T(s,t)+1 < T(s,m) do
            update T(s,m) with the value in T(s,t)+1.
            continue with the neighbors to m by calling recursive-heuristic-update(s,m)
        end.
    end.

```

Figure B.6: The algorithm for constructing a heuristic table for a RISK graph.

The algorithm starts out by initializing OPEN list and CLOSED list, and the start node is put into the OPEN list as a starting point. The main algorithm starts in line 5. The node n with the lowest (best) f value is selected from the OPEN list - this is as the best node as of now. All the neighbors to the best node is then looked at. If any given node m of those neighbors is not in the OPEN and not in the CLOSED list, we put it in the OPEN list for further exploring. The g , h , f , and parent information on m is recorded. If m is already in OPEN or CLOSED list, we check if the path to m through n is better than the other path recorded for m . If it is, we update the g , h , f , and the parent information for m . When the target node is found to be the best node in the OPEN list, we end the while loop, and construct the resulting path by following the parent information on the target node back to our starting node. This will result in the shortest path.

```

0 function getShortestPath(node s, node t, heuristic_table T, current_player P)
1   First check if the two nodes are valid (t not owned by P and s !=t).
2   Initialize an OPEN and a CLOSED list of dynamic size.
3   add s to the OPEN list.
4   update g,h,f for s: g[s]=0, h[s]=T(s,t), f[s]=g[s]+h[s].
5   while the OPEN list is not empty do
6     n := the node with the lowest f from OPEN list.
7     if n = t do
8       the target has been found, and we break out of the while.
9     remove n from OPEN list.
10    add n to the CLOSED list.
11    for each neighbor m to n do
12      if the owner of m is not P do
13        if m is not in OPEN list and not in CLOSED list do
14          add m to OPEN list.
15          update g,h,f for m: g[m]=g[n]+step_cost(m), h[m]=T(m,t),
16            f[m]=g[m]+h[m].
17          update the parent for m: parent[m] = n.
18        else
19          if g[n]+armies_on[m] < g[m] do
20            update g,h,f for m: g[m]=g[n]+step_cost(m), h[m]=T(m,t),
21              f[m]=g[m]+h[m].
22            update the parent for m: parent[m] = n.
23          end if.
24    end while.
25  If we then follow the parents starting with parent[t],
26  we should end in s - this is the shortest path.
27 end.

```

Figure B.7: *The A* algorithm.*

B.4 EM Learning Example

This section will go through an example of use of the EM algorithm. It is based on the network shown in Figure B.8.

The unobserved node is C and training data for the BN is shown in Table B.1.

Initially probability distributions for each node in the BN is assigned. $P(A) = (0.7, 0.3)$, $P(B) = (0.4, 0.6)$, $P(C|A, B)$ is specified as shown in Table B.2 and $P(D)$ is shown in Table B.3.

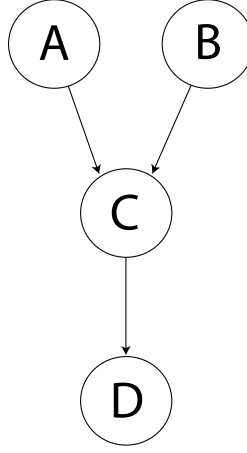


Figure B.8: The nodes A , B , and D are always observable and the node C is never observed in the training data.

	A	B	C	D
1	a_1	b_2	?	d_2
2	a_2	b_1	?	d_2
3	a_1	b_2	?	d_1
4	a_2	b_2	?	d_2
5	a_2	b_1	?	d_1
6	a_1	b_1	?	d_1
7	a_1	b_2	?	d_2

Table B.1:

B.4.1 Step 1: Expected Number of Occurrences

The first step done is finding the expected number of occurrences of $P(C|A, B)$. This is done by counting the number of occurrences for each possible configuration of $P(C|A, B)$ in the training data. The following will calculate this value for $P_0(C|A = a_1, B = b_1, D = d_1)$:

$$\begin{aligned}
 \mathbb{E}[\mathcal{N}(C = c_1|A = a_1, B = b_2)] &= 2 \times P_0(C = c_1|A = a_1, B = b_2, D = d_2) \\
 &\quad + P_0(C = c_1|A = a_1, B = b_2, D = d_1) \\
 &= 2 \times 0.4615 + 0.1818 = 1.1048
 \end{aligned}$$

	a_1		a_2	
	b_1	b_2	b_1	b_2
c_1	0.3	0.1	0.4	0.8
c_2	0.7	0.9	0.6	0.2

Table B.2: $P(C|A, B)$

	c_1	c_2
d_1	0.3	0.1
d_2	0.7	0.9

Table B.3: $P(D|C)$

$$\begin{aligned}
\mathbb{E}[\mathcal{N}(C = c_2|A = a_1, B = b_2)] &= 2 \times P_0(C = c_2|A = a_1, B = b_2, D = d_2) \\
&\quad + P_0(C = c_2|A = a_1, B = b_2, D = d_1) \\
&= 2 \times 0.5385 + 0.8182 = 1.8952
\end{aligned}$$

There are three cases in the training data where $(A = a_1, B = b_2)$, and the expected probability that $C = c_1$ is $\sum_{i=1}^N P(C = c_1|\mathbf{d}_i)$, where N is 3 and $\mathbf{d}_i$ is those three cases. The expected number of occurrences is found for all probabilities in node C .

B.4.2 Step 2: Maximum Likelihood Estimate

The maximum likelihood estimate for $P_1(C|A = a_1, B = b_2)$ can now be calculated for each state of C using the values found in step 1. For instance:

$$\begin{aligned}
P_1(C = c_1|A = a_1, B = b_2) &= \frac{\mathcal{N}(C = c_1|A = a_1, B = b_2)}{\mathcal{N}(A = a_1, B = b_1)} \\
&= \frac{1.1048}{3} = 0.3683
\end{aligned}$$

$$\begin{aligned}
P_1(C = c_2|A = a_1, B = b_2) &= \frac{\mathcal{N}(C = c_2|A = a_1, B = b_2)}{\mathcal{N}(A = a_1, B = b_1)} \\
&= \frac{1.8952}{3} = 0.6317
\end{aligned}$$

This is done for all probabilities of C . The rest of the variables in the network will be calculated the same way. The values found in step 2 will then be used in step 1 in the next iteration of the algorithm. The algorithm will continue until the difference between the maximum likelihood estimate calculated in iteration n and the maximum likelihood estimate in iteration $n - 1$ is smaller than some predefined threshold value.

Design Appendix

This chapter of the report contains the design appendix. The chapter will contain design material that seemed to extensive to have in the actual report.

The reader should be aware that there is not any continuity between sections in this chapter. This is meant as a lookup when a greater detail in some part of the Design. This chapter however, does not contain text, that has not been referenced to in the actual report.

C.1 Full Design of Dataflow in Framework

This section gives the full design of the dataflow in the framework.

C.1.1 Data Structures

This section covers the data structures used in the framework. This include how the board, the past and opponents' number of RISK cards is structured.

The Board

The board is used as input for many different tasks in the AI. The board can be represented as a list of territories. A territory has the following attributes:

- **Name:** The name of the territory.
- **Owner:** The player who occupies the territory.
- **Armies:** The number of armies placed in the territory.
- **Continent:** What continent the territory belongs to.
- **Neighbors:** The set of territories bordering the given territory.

The Past

The Past is a list of actions performed by each player from the game begins to the current round. The past is a kind of log over what has happened in the game until now. The past is divided into two different logs: an *action past* and a *board past*. The action past contains the actions performed by a player. The same entry in the board past contains the board state *after* that action has been performed.

All entries in the action past begin with a player color - the performer of the given action. Following that, the past can have the following entries:

- **CASH_CARDS card1 card2 card3:** A player has decided to cash in his RISK cards.
- **PLACE_ARMY territory amount:** A player has placed an amount of armies in a territory.
- **ATTACK territory territory amount:** A player attacks from the first territory to the neighboring second with some amount of armies.
- **MOVE amount:** After an attack, a player is forced to move some amount of armies to the conquered territory (but not if he lost or retreated the attack).
- **FORTIFY territory territory amount:** This is always the last action a player can do (if he chose to). An amount of armies were moved from the first territory to the second.

The board and action past can best be illustrated in an example. Figure C.1 is an example of an action past log and the corresponding board past log.

Action Past

```
red PLACE_ARMY ural 4
red ATTACK ural siberia 3
red ATTACK ural siberia 3
red MOVE 2
red FORTIFY china siberia 4
blue CASH_CARDS infantry cavalry cannon
blue PLACE_ARMY yakutsk 13
blue ATTACK yakutsk siberia 3
blue ATTACK yakutsk siberia 3
blue MOVE 8
blue ATTACK siberia china 3
blue MOVE 6
...
```

Board Past

```
7,1;2,2;1,2;5,1;
4,1;1,2;1,2;5,1;
3,1;0,1;1,2;5,1;
1,1;2,1;1,2;5,1;
1,1;6,1;1,2;1,1;
1,1;6,1;1,2;1,1;
1,1;2,1;14,2;1,1;
1,1;1,1;11,2;1,1;
1,1;0,2;11,2;1,1;
1,1;8,2;3,2;1,1;
1,1;7,2;3,2;0,2;
1,1;1,2;3,2;6,2;
...
```

Figure C.1: An example of an action past and the corresponding board past. The board past only contains information on the 4 involved territories, but a real board past would contain all 42 territories. The first line in the board past is read: 7 armies in ural occupied by player 1 (red), 2 armies in siberia occupied by player 2 (blue), 1 army in yakutsk and 5 armies in china. The action “Cash Cards” does not change the board, and the corresponding line in the board past is just left unchanged.

Opponents' number of RISK cards

The opponents' number of RISK cards is also an information used in the framework. This is straightforward: this data structure holds, for each player, an integer stating how many RISK cards the given player is holding.

C.1.2 Communication Between Modules

This section covers the how each module communicate using input and output. To assist in the explanation of the communication in the AI Figure 2.12 on page 33 is used.

Information Givers

This section covers the input and output structures for each IG.

Estimating Opponents' missions

As seen in Figure 2.12 this information giver takes the past as input. The output of this IG is a two dimensional table of certainties:

- **Player:** First dimension contains each player in the game.
- **Mission:** Second dimension contains each of the 14 missions in the game.
- **Certainty:** Each element in the table is a float, which is the estimate on which mission a given player has.

How Close is an Opponent to Winning

This IG takes both the board and the estimate of opponents' missions as input. The output of this function is a two dimensional table of probabilities:

- **Player:** First dimension contains each player in the game.
- **Number of Rounds:** Second dimension contains the number of rounds from 1 to 5.
- **Probability:** Each element is a probability of the player finishing the game within the given round.

Opponents' Next Moves

This function takes the board, number of opponent's RISK cards and the out from all the other IGs as input. The output is the same as the action past - a list of actions to be made by each player. A structure similar to the board past is not needed, since actions in "opponents' next moves" are based on the current state of the board, which is directly readable.

How Close an Opponent is to Owning a Continent

This IG takes the board and the opponents' number of RISK cards as input. The output is a two-dimensional array with the probability for each player owning each continent in the following five rounds.

Master Prioritizer

The Master Prioritizer takes all outputs from the information givers as input. It outputs a prioritized list of goals. The output is represented as an array of floats representing the priority of each goal.

Round Planner

As seen in the Figure 2.12 on Page 33 the RP is divided into three separate parts: Planner, Reinforce, and Fortify. Before going into data communication within these parts, we will first look at what is transmitted between the planner and the reinforce AI.

Planner → Reinforce

Data output from the planner is attack plans and defense lists. An attack plan is a list of opponent territories forming a path going from an occupied territory to a target territory. These plans have a cost and a priority attached. The defense list is a list of territories that needs reinforcements in order to withstand an attack. The territories in this list also have a cost and priority attached. Before these two types of data enter the reinforce part of the AI, they are merged into one list. The attack plan is reduced to be just the starting territory with the attached cost and priority. When merging the list of start territories with the list of territories needing defense, the same territory could appear more than once because it could both need defense and be the starting territory in an attack plan. In that case the territory with the highest priority should be kept, the other should be deleted from the list.

Finally, this combined list with territories, priorities and costs is input to the Reinforcement AI. After the Reinforcement AI has finished reinforcing, new attack plans are made from the new board state.

The Remaining Round Planner Modules

In the following, the input and output data structures, of the RP modules are described.

Make Attack Plan

This planner module takes two references to territories as input - an occupied starting territory and a target territory. It also takes the goals output from the MP, which is an array of floats, and the board as input. The data structure for the board is as written Section 3.2. The output is an array of references to territories beginning with the starting territory ending at the target territory.

Calculate Attack Plan Cost

Takes the input: attack plan (an array of territories), the board and MP goal distribution. The output is an integer representing the estimated cost to fulfill the plan.

Prioritize Attack Plan

Takes the same input as the module above. The output is a float representing the priority for that attack plan.

Discard Attack Plans

Takes the input: a priority (float), minimum and estimated army cost (integers), number of reinforcements (integer) and number of ISK cards (also an integer). It outputs “Yes” or “No” depending on how important the plan is versus its cost. If the output of the function is “Yes”, the plan will be discarded, otherwise the plan, along with its cost and priority, is sent out of the planner module.

Calculate Defense Cost

This planner module takes a reference to each occupied territory one at a time as input along with the board as input. The output is an estimated cost for defending the territory.

Prioritize Territory Needing Defense

This planner module takes the following input: a reference to each occupied territory one at a time, the opponents’ next moves (an array of strings for each player) and the number of RISK cards owned by the opponent (integer). The output is a float representing the priority for defending the territory.

Cash Cards

This module takes the following input: a list of priorities (array of floats), list of minimum and estimated cost (each an array of integers), number of armies gained (integer) and the number of RISK cards (integer). The output is a straightforward boolean on whether to cash cards or not.

Place Armies

The data structure for the input to this module is the same as the data structures for the above module. The output is simply a reference to a territory to place an army in.

Transfer Armies

The data structures for this module have all been described before and are straightforward. The module has three outputs: a reference to the source territory, a reference to the destination territory, and an integer for the number of armies to transfer.

C.2 Training Examples

This section shows an example of a training example. The example can be seen in Figure C.2.

C.3 HamPath Algorithm

This section shows the brute force algorithm for finding a Hamiltonian path in a small graph. The algorithm can be seen in Figure C.3.

```

<player name="red_player" color="red">
  <mission value="Conquer North America and Africa">
  </mission>
  <ownership>
    Congo
    Ural
    ...
  </ownership>
</player>
<player name="mrBlue" color="blue">
  ...
</player>
...
<initial_placement>
  <player_turn color="red">
    <place_one_army>
      Congo
    </place_one_army>
  </player_turn>
  <player_turn color="blue">
    ...
  </player_turn>
  ...
</initial_placement>
<round value="1">
  <player_turn color="red">
    <module_output name="IG_EOM">
      red:con_asia_sam=10%;
      red:con_asia_afr=20%;
      ...
      blue:con_asia_sam=11%;
      blue:con_asia_afr=43%;
      ...
    </module_output>
    ...
    <module_output name="transfer">
      source=ural;
      dest=ukraine;
      num=4;
    </module_output>
    <risk_card_given>
      Iceland;
    </risk_card_given>
  </player_turn>
  ...
</round>
<round value="2">
  ...
</round>
...

```

Figure C.2: An example of a training example for a single game of RISK.

```

Path HamPath(INT s, INT t, Graph g) {
    Path r = NULL;
    IF (!g.get(s).visited) THEN
        g.get(s).visited = TRUE;
        IF (s == t)
            IF AllVisited(g) THEN
                r.push(t);
            ELSE
                INT i = 0;
                REPEAT
                    r = HamPath(g.get(s).getNeighbor(i), t, g);
                    i++;
                UNTIL (i == g.get(s).neighborCount) OR (r != null);
                r.push(s);
        RETURN r;
}

```

Figure C.3: Algorithm for finding a hamiltonian path between two specified nodes in an undirected graph.

C.4 Scripting for the IG “Estimating Opponents’ Missions”

This is an example of how the estimation of a player’s mission (not including “24 territories” and “18 territories”) can be calculated.

C_{Asia}	= 4	C_{Red}	= 12
$C_{NorthAmerica}$	= 16	C_{Blue}	= 6
$C_{SouthAmerica}$	= 16	C_{Green}	= 6
C_{Africa}	= 8		
C_{Europe}	= 0	$b = 24$	
$C_{Australia}$	= 4		
$a = 48$			

Figure C.4: This information is taken from the Past and is depicts a player’s interest in each of the continents and each of the other players.

Figure C.4 is an example of the information takes from the Past: C_{color} , C_{Asia} , $C_{NorthAmerica}$, $C_{SouthAmerica}$ etc., which depicts the player’s interest in each continent and each of the other players. The corresponding M_{color} , $M_{Asia_SouthAmerica}$, M_{Asia_Africa} etc. which are the estimations on the respective missions, are then calculated as follows:

$$M_{Asia_SouthAmerica} = \frac{1}{4} \left(\frac{4}{48} + \frac{16}{48} \right) = \frac{5}{48}$$

$$M_{Asia_Africa} = \frac{1}{4} \left(\frac{4}{48} + \frac{8}{48} \right) = \frac{3}{48}$$

$$M_{NorthAmerica_Africa} = \frac{1}{4} \left(\frac{16}{48} + \frac{8}{48} \right) = \frac{6}{48}$$

$$M_{NorthAmerica_Australia} = \frac{1}{4} \left(\frac{16}{48} + \frac{4}{48} \right) = \frac{5}{48}$$

$$M_{Europe_SouthAmerica_3rd} = \frac{1}{4} \left(\frac{0}{48} + \frac{16}{48} \right) = \frac{4}{48}$$

$$M_{Europe_Australia_3rd} = \frac{1}{4} \left(\frac{0}{48} + \frac{4}{48} \right) = \frac{1}{48}$$

$$M_{red} = \frac{4}{6} \left(\frac{12}{48} \right) = \frac{8}{48}$$

$$M_{blue} = \frac{4}{6} \left(\frac{6}{48} \right) = \frac{4}{48}$$

$$M_{green} = \frac{4}{6} \left(\frac{6}{48} \right) = \frac{4}{48}$$

Which sum to $\frac{40}{48}$. The last $\frac{6-4}{12}$ is then evenly distributed to each “set of continents” mission estimate and the whole thing will sum to 1.

C.5 Scripting for the IG “How Close the Opponent is to Winning”

This section contains algorithms used in the scripting of this Information Giver. Figure C.5 shows the algorithm for finding the amount of armies a player potentially could muster from the rest of the board when conquering a continent.

```

0 for each territory tn in TN do.
1   for each territory tb in TB do.
2     find shortest path P from tn to tb.
3     add P to Pnb.
4   Pc = the cheapest path in Pnb.
5   a = get the amount of armies left after doing Pc.
6   Aacc = Aacc + a.
```

Figure C.5: The algorithm finds first the cheapest path from each territory t_{nb} in T_N to a territory in T_B . P_{nb} holds the shortest paths from t_{nb} to all territories in T_B . P_c is the cheapest of those. a is the amount of armies left when doing P_c . α_{acc} is the accumulated armies we wish to find.

The algorithm in Figure C.6 finds how likely it is, that a given player will conquer s number of territories

```

0 repeat until the opponent has conquered s territories {
1   for each territory t in T0 do
2     for each neighbor n to t not occupied by the opponent do
3       calculate the probability P of winning an attack from t to n.
4       add P to Plist and [t, n] to Attacklist
5   Find the highest P in Plist, Ph, and the corresponding [tA,nA]
      in Attacklist.
6   Pjoint = Pjoint * Ph
7   Assume that the opponent won with no loss (worst case) and move
      all armies in At to nA minus 1 or 2 (depending on mission).
8 }

```

Figure C.6: The algorithm simulates a series of attacks made by the opponent, where he attacks where the probability of winning is most highest. This result in P_{joint} - the joint probability of the opponent winning all attacks.

C.6 Scripting the Master Prioritizer

This section contains a more detailed description on how the importance points on each goal is distributed in the script. The design of the script for the MP in the actual report only contains a summary of the important parts of the script, this section contains the full description.

Remember the MP takes the following input and output:

Input: The estimates received from the information givers.

Output: A prioritized goal distribution.

When scripting the MP, it will be necessary to hard code a lot of the scripts to the specific game of RISK. This is unavoidable since the goals are based on mission that are very specific for the game. This means that if the missions were changed, the MP would not necessarily perform as intended.

The script for the MP gives a distribution over the goals given to the RP. This distribution is limited within a percentage scale between 1 and 100%. The distribution states the importance of each goal. The closer a goal gets to 100%, the more important it is. The unit of importance is an *importance point*. There are 100 of these available for the distribution over all goals.

There are two ways for the AI to play the game. The first way is pursue its mission by either conquering mission-specific territories and continents or by destroying opponents. This is from now on denoted as doing *mission-specific* actions. The other way the AI can play the game is by keeping the opponents from fulfilling their missions. This is said as being *preventive*, and that the AI is doing *preventive* actions. To aid in the task of deciding on whether or not the AI should play mission-specific or preventive, an array over players winning the game is maintained as seen in Figure C.7.

```

FOR_EACH PLAYER
  IF
    IG_CloseToWinningRounds[1] > 0.75 OR
    IG_CloseToWinningRounds[2] > 0.8 OR
    IG_CloseToWinningRounds[3] > 0.85 OR
    IG_CloseToWinningRounds[4] > 0.9 OR
    IG_CloseToWinningRounds[5] > 1.0;
  THEN
    PlayersWinningArray.add( playerName );

```

Figure C.7: Any player close winning within the next five rounds due to some threshold is added to an array (*PlayersWinningArray*) to keep track of which players are winning.

Initial Distribution

Initially the importance distribution over the goals is equal. This means that as a basis, all goals are instantiated to $InitialImportanceOfGoals = \frac{100}{Numberofgoals}$. Looking at the description of the goals we can see that some of them are mission-specific. These are “18 territories with at least 2 armies” and “24 territories”. Therefore it is not necessary to include these in the initial distribution if neither of them is the AI’s mission. Therefore the initial distribution script for the RISK game will behave as the pseudo code in Figure C.8.

```

IF
  Mission NOT_EQUAL_TO 18_Territories OR 24_Territories
THEN
  FOR_EACH Goal
    Importance = 100/6;
ELSE
  Importance = 100/7;

```

Figure C.8: Initial distribution script.

Internally within the goals there needs to be an equal distribution over the number of continents or players in the game. This is obtained by giving each continent within each goal the importance $InitialImportanceOfContinents = \frac{Initialimportanceofgoal}{Numberofcontinents}$. After this distribution, points are removed from continents that are not part of the AI’s mission and ones where the AI has a probability lower than 5% for conquering it the next round. This is, if possible, to free up some importance points from continents where the AI has too small a chance to do anything anyway. The same procedure is followed for the “Attack Player” goal, where “Number of continents” is substituted by “Number of Players”.

Distributing Importance

The goal distribution should be done in such a way that the AI mainly plays mission-specific when other players are not close to winning. Of course this should not necessarily always be the case. If for example the AI has recently conquered a continent it may be forced to play defensive for a few rounds in order to keep the continent.

The first thing to do in the MP is to gather all information for each player

(including one self) in a data structure for easier access to all information known about him from the IGs. This data structure is seen in Figure C.9.

```

PlayerName Name;
CloseToWinningRounds [CertaintyForWinningRound: 1,2,3,4,5];
EstimatedMissionWithCertainty [Mission,Certainty]
NextMoves [T1,T2,T3,...,Tn];
CloseToOwningContinent [Continent, Certainty: 1,2,3,4,5];

```

Figure C.9: The *PLAYER* data structure that keeps information on a player.

Apart from the already mentioned array to keep track of who is winning, it is also usable to keep track of which players are close to taking a continent and especially if any of the opponents are going to attack the AI's continents. This is seen in Figure C.10. How this array is updated is seen in Figure C.11.

```

PlayersWinningArray [Players];
MyContinentsArray [Continents];
MyContinentsInDangerArray [Continents];
PlayersTakingContinentArray [Continent,Players];

```

Figure C.10: The data maintained by the AI. *PlayersWinningArray* is an array of players that are close to winning the game. *MyContinentsArray* is an array of continents the AI occupies. *MyContinentsInDangerArray* is an array of continents that are currently threatened by other players, meaning that one or more of the territories in the continent is in the attack plan of a player. *PlayersTakingContinentArray* is an array over players that are close to conquering a specific continent.

It is now possible to determine whether or not to play preventive in the following turn. Playing preventive means taking importance points away from mission-specific goal. Note that this does not mean that the AI will not do any attacks. It means that AI will attack or fortify in order to prevent another player from winning, disregarding its own mission for the moment. A script for this decision can be seen in Figure C.12. A script, *MyContinentInDanger*, that determines whether or not territories within an occupied continent are threatened the next round is used to determine if an occupied continent needs reinforcements. This script compares the list of territories in an occupied continent with the territories in the opponents' next moves. This script should return true if one of the AI's continents is in any other player's next moves and enter which territory in the *MyContinentsInDangerArray* array.

Doing it this way means that the AI will only play preventive if an opponent is winning, if an opponent is close to taking a continent, or if one of its own continents is in danger of being conquered.

Redistributing Importance Points

With just 100 importance points available for the full distribution, it needs to be established how to redistribute points given in the initial distribution from one

```

FOR_EACH Continent
  FOR_MY_SELF
    IF
      IG_CloseToContinentRounds[1] == 1.0 OR
      IG_CloseToContinentRounds[2] == 1.0 OR
      IG_CloseToContinentRounds[3] == 1.0 OR
      IG_CloseToContinentRounds[4] == 1.0 OR
      IG_CloseToContinentRounds[5] == 1.0;
    THEN
      MyContinentArray.add(Continent);
FOR_EACH PLAYER
  IF
    IG_CloseToContinentRounds[1] > 0.75 OR
    IG_CloseToContinentRounds[2] > 0.8 OR
    IG_CloseToContinentRounds[3] > 0.85 OR
    IG_CloseToContinentRounds[4] > 0.9 OR
    IG_CloseToContinentRounds[5] == 1.0;
  THEN
    PlayersTakingContinentArray.add(Continent, Player);

```

Figure C.11: A script that adds continents the AI owns to its *MyContinentArray* array and adds players close to occupying a continent to the *PlayersTakingContinent* for that specific continent.

goal to another. This is why it was first established whether to play preventive or not. Playing preventive means that importance points will primarily be taken from mission-specific goals. However, playing preventive should not totally rule out the possibility to make an mission-specific attack plan in the RP. Therefore we will not just remove point from all mission specific goals and give them to preventive goals if the AI is playing preventive.

Releasing Importance Points

After the initial distribution, each goal gets to state a *claim* on importance points. Goals that do not require additional importance points release their assigned points to a pool of redistributive points. This will result in the AI releasing points for all goals and sub goals that it has no interest in.

Preventive Mode

If the AI is in preventive mode, preventive goals must have their claims met before mission-specific goals. The goals with the highest claims are considered the most important and should have their claims met first. The claims are met by first assigning importance points from the pool of re-distributional points to the highest claimer and down to the lowest claimer in preventive missions. If no more points are available in the pool of re-distributional points, the preventive goals start taking them from the mission-specific goals. Points must be taken from the least important goal and up to ensure that, if any, the most important mission-specific goals are passed on to the RP. If even the mission-specific goals' are not enough to meet the claims of the most important preventive goals, the must start taking from each other. Again, the most important ones needs have their claim fulfilled by taking from the ones with less importance and up. In this way, it is insured that the most important goals are passed to the RP. Pseudo code for this the re-distribution is seen in Figure C.13. The script uses a function, *MakeImportanceList()*, that builds a prioritized list based

```

IF
  PlayersWinningArray == EMPTY AND
  FOR_EACH Continent X:
    PlayersTakingContinentArray[X] == EMPTY
THEN
  PlayPreventive = FALSE;
ELSE
  IF
    PlayersWinningArray == [Only_MYSELF]
  THEN
    IF
      MyContinentsInDangerArray != EMPTY
    THEN
      PlayPreventive = TRUE;
    ELSE
      PlayPreventive = FALSE;
  ELSE
    PlayPreventive = TRUE;

```

Figure C.12: *The script that decides whether or not to play preventive the in the turn. It will play preventive if any players are close to taking a continent, or if any of the AI's continents are in RISK of being attacked, or if any player besides itself is winning.*

on the claims and the play mode. This list is essentially split in two: one part for each play mode. The play mode dictates which claims are most important. If the play mode is preventive, the preventive goal with the highest claim is on top of the list followed by the second most important preventive goal and so forth. The mission-specific goals are placed below the least important preventive goal with the same ordering (high claimers first and so forth). For mission-specific mode, the mission-specific goals top the list followed by the preventive goals

Mission-specific Mode

This mode behaves in the same way as preventive mode. The only difference is that in mission-specific mode, mission-specific goals meet their claims on behalf of preventive goals. If in either mode, the pool is not used up, the remaining importance points are used to scale up the importance of the goals used by the mode starting from the most important goal and down.

It will now be described how each goal make claims on importance points.

Conquer and Defend Continent

This goal is divided into smaller parts. There is one conquer and defend importance attached to each continent. It is an mission-specific goal, meaning that it is given higher priority if the AI is not in preventive mode. If the AI is close to owning an entire continent, this goal for that continent gets higher priority than conquering other continents, even higher if the continent in question is mission-specific to the AI.

Such a continent is likely to get higher priority when the “How Close an Opponent Is To Owning A Continent”-IG (calculated for the AI itself) gives the probabilities [0.70, 0.75, 0.80, 0.85, 0.90] or higher for the number of rounds from 1 to 5. This goal will then make a claim of 5 points if the probability

```

Pool = 0;
DoInitialDistribution();

//Make Claims. Each goal gets an claim assigned.
FOR_EACH GOAL
    MakeClaim();

//Release unclaimed points to pool
FOR_EACH GOAL
    IF Goal.Claim == InitialGoalImportance THEN
        Pool += InitialGoalImportance
        Goal.Claim = 0;

//Re-distribute importance points.
ImportanceList = MakeImportanceList();
INT InitialSizeOfList = SIZE_OF(ImportanceList);
INT i = 0;
WHILE (i < (SIZE_OF(ImportanceList)-1)) AND (ImportanceList.getItem[i].Importance !=
        ImportanceList.getItem[i].Claim) DO

    IF Pool == 0 THEN
        IF ImportanceList.getItem[SIZE_OF(ImportanceList)-1].Importance == 0 THEN
            ImportanceList.getItem[SIZE_OF(ImportanceList)-1].Importance--;
            ImportanceList.getItem[i].Importance++;
            ImportanceList.getItem[i].Claim--;
        ELSE
            ImportanceList.removeItem[SIZE_OF(ImportanceList)-1];
        ELSE
            Pool--;
            ImportanceList.getItem[i].Importance++;
    i++;

// Any point left in pool?
INT j = 0;
WHILE (Pool != 0) AND (j < SIZE_OF(ImportanceList)) do
    ImportanceList.getItem[j].Importance++;
    Pool--;
    j++;

```

Figure C.13: Script for redistributing importance points. If no points are available in the pool, points are taken from the goal at the bottom of an importance list. When this goal has no importance points left, it is removed from the list. This is repeated until all the top priorities have had their claims met.

in round 1 is higher than 0.7, if not, then a claim of 4 is made if round 2 is higher than 0.75 and so on. When assigning importance, all mission-specific continents will get a importance boost of 10 points.

If the continent in question is mission-specific to the AI the points assigned are multiplied by 2. A simple script for assigning importance points to a continent is seen in Figure C.14.

Conquer Continent

This mission-specific goal is given higher priority for a continent when the AI is close to completing its mission. This information is obtained by using the IG “How Close an Opponent is to Winning” on the AI with 100% certainty on the input mission. This should only be given higher priority if the IG tells the MP that it will probably complete its mission within this turn with over 0.9 certainty. The assignment of points is the same as in the conquer and defend goal only with the difference that there is this extra check on whether or not the AI is close to winning in the turn.

```

FOR_EACH Continent
  If
    ContinentIsMissionSpecific;
  then
    ImportanceClaim = 10;
  For_MYSELF
    if
      IG_CloseToContinentRounds[1] > 0.70
    then ImportanceClaim +=10;
    else if
      IG_CloseToContinentRounds[2] > 0.75
    then ImportanceClaim +=8;
    else if
      IG_CloseToContinentRounds[3] > 0.80
    then ImportanceClaim += 6;
    else if
      IG_CloseToContinentRounds[4] > 0.85
    then ImportanceClaim += 4;
    else if
      IG_CloseToContinentRounds[5] > 0.90
    then ImportanceClaim += 2;
    else
      Return ImportanceClaim;
  else
    For_MYSELF
      if
        IG_CloseToContinentRounds[1] > 0.70
      then ImportanceClaim +=5;
      else if
        IG_CloseToContinentRounds[2] > 0.75
      then ImportanceClaim +=4;
      else if
        IG_CloseToContinentRounds[3] > 0.80
      then ImportanceClaim += 3;
      else if
        IG_CloseToContinentRounds[4] > 0.85
      then ImportanceClaim += 2;
      else if
        IG_CloseToContinentRounds[5] > 0.90
      then ImportanceClaim += 1;
      else
        Return ImportanceClaim;

```

Figure C.14: Script to assign importance points to a continent under the Conquer and Defend goal

Defend Continent

This mission-specific goal is given higher priority for a continent if this continent is fully occupied by the AI and any of the occupied territories in that continent is part of an opponents' next moves. Again, the IG "How Close an Opponent Is To Owning A Continent" is used as the basis to decide how many importance points are assigned to the continent. In this goal we only care about continents where the probability for the AI conquering it within the next two rounds is higher than some threshold. This threshold is dependent on whether or not the continent is mission-specific. If it is not mission-specific the threshold is [0.7,0.8]. If it is mission-specific, the threshold is [0.6,0.7]. If the actual value is greater than the threshold the continent make a claim of 5 points for that continent. If any territory in the continent is included in any opponent's attack plan, the continent will make a claim of 2 points for each territory in the

attack plan. If the continent is also mission-specific, it will make a claim of 10 points for each territory in the opponent's attack plan. Territories located in several opponents' attack plans are only counted once since we are only interested in whether or not the continent is attacked.

Obstruct Continent

This preventive goal gets high importance if opponent is close to owning one or more continents with probabilities above $[0.9, 0.92, 0.94, 0.96, 0.98]$ and is also close to winning the game within the next five rounds with a distribution that includes 1.0 at any of the next three rounds, for example $[0.95, 0.97, 1.0, 1.0, 1.0]$. This situation will result in the MP getting paranoia and will make a claim of 20 points in the continents in the opponents' mission. If any of these continents are mission-specific to the AI another 5 points is claimed for this specific continent.

Obstruct and Defend Continent

This preventive goal is given high priority in two cases. The first case is if an opponent has taken, or is close to taking a continent. The distribution of how close the opponent is to occupying the territory should be larger than $[0.60, 0.70, 0.80, 0.9, 1.0]$ and make a claim of 10 points. The reason for this distribution is that the AI will have some time to counter the opponent getting foothold in the continent by reacting somewhat early to the threat of the opponent occupying the continent. The importance points claim on a continent, in the case that the continent is mission-specific to the AI, should be 15 points.

The other case is when the opponent is estimated to win the game within the next five rounds with the distribution larger than $[0.6, 0.7, 0.8, 0.9, 1.0]$ at a specific entry and his mission is estimated to contain continents he occupies or nearly occupies with the same probability as in the first case. If this is the case, all continents specified in the opponents mission should given a high obstruct and defend priority. The importance point claim on this continent should be 15. If any of the continents are mission-specific to the AI, a claim of 20 points is made to this continent.

Attack Player

This, both mission-specific and preventive, goal is mainly prioritized in two cases. The first being if it is the AIs mission to destroy another player. The other case is if an estimated opponent's mission is either one of the "Occupy 18" or "Occupy 24" territories and he is estimated to be close to fulfilling the mission with at distribution above $[0.6, 0.7, 0.8, 0.9, 1.0]$ on any entry.

If the AI has a "Destroy player X"-mission this goal must always get some importance points. However, this must not be give too many importance points, since this is an easily detectable mission. As a basis it makes a claim of 5 points on this player X. When "How close an opponent is to winning" (run on the AI itself) returns a probability distribution above $[0.8, 0.9, 1.0, 1.0, 1.0]$ this goal makes a claim of 15 points for the given player X since the AI is fairly close to fulfilling the mission.

The second case, where a player is estimated of having "Occupy 18" or "Occupy 24" and close to winning, then this player should be targeted. A

claim of 5 points is made on attacking this player.

18 territories with at least 2 armies.

This mission-specific goal is only given high priority if the AI is close to fulfilling the mission corresponding to this goal with a minimum distribution $[0.6, 0.7, 0.8, 0.9, 1.0]$ on all entries from the IG “How Close is An Opponent to Winning” run on the AI itself. In such a case it makes a claim of 15 points. If it is not close (the certainties are smaller than 0.6, 0.7 etc), then it makes a claim of 5 points, since it is its mission.

24 territories

This mission-specific goal is only given high priority if the AI is close to actually reaching it with a minimum distribution $[0.6, 0.7, 0.8, 0.9, 1.0]$ on all entries from the IG “How Close is An Opponent to Winning” run on the AI itself. It must of course not make no claim at all since it is the AI's mission. The claim is the same as for “18 territories with at least 2 armies”.

Concealing the Mission

The MP needs a way to conceal the mission it is trying to fulfill. This could have been a goal given to the RP, but this is done by producing a different importance distribution on the goals instead.

This is done by checking what the opponents' would think the AI's mission is. This is done by running the IG “Estimating an Opponent's mission” on itself. If the estimate is correct and above 75% certainty the AI should do some random actions to mislead the other players. However, the random moves should not include leaving mission-specific continents. Rather it should give a distribution on the goals with an even distribution on “Attack Player” which will leave it up to the RP whom to attack. If the AI's mission is a “Destroy Player X”-mission, the distribution should result in a lower importance on the player in mission. Still there should at least 10 importance points on defending each occupied, mission-specific continent.